ADVANCED PYTHON · PE-V4 · 20 BÀI ĐẦU

Bản đáp án · 20 bài đầu

Dành cho giảng viên và cố vấn buổi chạy thử · 20/20 bài · bản dựng 13/10/2026

KHÔNG PHÁT CHO HỌC VIÊN

Tập giấy này chứa lời giải mẫu, đáp án trắc nghiệm và đáp án mọi mini-game của 20 bài đầu. Phát cho học viên là phá buổi chạy thử. Mỗi trang đều có dòng cảnh báo ở đầu trang để một tờ rời cũng không bị hiểu nhầm.

Dùng vào việc gì. Kèm một buổi chạy thử có người học thật: đối chiếu code học viên với lời giải mẫu, biết bộ chấm đòi gì, trả lời được "sai ở đâu" mà không phải tự giải lại 20 bài.

Phạm vi. P01–P07 (process), T01–T06 (thread), G01–G06 (GUI), R02 (boss). 28 bài còn lại của giáo trình CỐ Ý không có ở đây: X01, X02, X03, X04, X05, N01, N02, N03, N04, N05, N06, PD01, PD02, PD03, PD04, PD05, PD06, PD07, PD08, PD09, M01, M02, M03, M04, M05, M06, DA01, DA02.

Mỗi bài bắt đầu ở một trang mới (trừ bài đầu của mỗi module, đi liền dưới tên module) và mang mã bài ở đầu trang. Bài nhiều code (R02) tràn sang vài trang — các trang đó vẫn mang đúng mã bài, nên một tờ bị xé ra vẫn biết là của bài nào.

Nguồn. Mọi dòng trong tài liệu được render từ bản build, không câu nào gõ tay: `out/lessons/*.json` (đề, trắc nghiệm, giải thích, ngộ nhận, trace, scaffold, env) · `out/qa.json` (lời giải mẫu, bộ ca chấm, các biến thể sai cố ý) · `out-advanced/lessons/*.json` (mini-game). Thiếu trường nào thì trang bài ghi là thiếu — tài liệu đáp án không được suy diễn.

Cái gì CỐ Ý không có ở đây. Phần lý thuyết và "điểm kiến thức" của bài không được lặp lại — chúng nằm ở sổ tay nội dung (`python-advanced-handbook.pdf`, cùng thư mục). Tập này chỉ in những gì cần để CHẤM và GIẢI THÍCH: đề, lời giải, bộ chấm, đáp án, bẫy, môi trường.

Đọc kỹ khối MÔI TRƯỜNG CHẤM (khung cam). Trong 20 bài này, subprocess / thread / tkinter đều là bản MÔ PHỎNG chạy trong trình duyệt. Thứ tự in của thread, việc cửa sổ không hiện ra, hay race condition không tái hiện được — đó là mô phỏng, KHÔNG phải lỗi của học viên.

Chấm nằm ở trình duyệt. Server không chấm lại gì; một bài "đạt" nghĩa là bộ chấm trong trình duyệt báo đạt. Xem AGENTS.md.

Màu. Khung xanh = đáp án. Khung đỏ = ngộ nhận / kiểu sai đã biết. Khung cam = môi trường mô phỏng. Chữ **ĐÚNG** đánh dấu lựa chọn đúng.

Bản dựng này có đủ 20/20 mã bài trong phạm vi.

Cách đọc một trang đáp án

Mọi bài theo đúng một thứ tự khối, để lúc đang kèm học viên thì tìm bằng vị trí chứ không phải đọc hết trang:

Khối	Là gì
Mục tiêu + ĐỀ BÀI	chép từ step_1.brief.goal và step_4.task — đủ để dùng trang này mà không cần mở sổ tay.
Hợp đồng hàm	chữ ký mà bộ chấm gọi tới; học viên đặt tên hàm khác là trượt hết ca.
LỜI GIẢI MẪU (khung xanh)	nguyên văn qa.json. Không phải lời giải DUY NHẤT đúng — bộ chấm chỉ gọi hàm và so giá trị.
BỘ CHẤM	từng ca: tên ca (chấm cái gì) và biểu thức gọi → giá trị mong đợi.
MÔI TRƯỜNG CHẤM (khung cam)	cái gì là mô phỏng. Đọc trước khi kết luận học viên sai.
NGỘ NHẬN (khung đỏ)	chỗ học viên thật sự vấp, theo chính tác giả bài.
Dự đoán / trắc nghiệm / phản ví dụ / tự giải thích	đáp án kèm giải thích nguyên văn JSON.
Trace + LẮP MẢNH	chuỗi trạng thái để giảng giải, mảnh đúng từng ô và bẫy từng ô.
ĐÁP ÁN MINI-GAME (khung xanh)	từng màn một: nối cặp, điền ô, sắp dòng, săn lỗi, sửa code, bảng theo dõi biến, đúng/sai, tự giải thích.
CÁC KIỂU SAI ĐÃ BIẾT (khung đỏ)	diff giữa lời giải mẫu và từng biến thể sai cố ý — code học viên thường rơi đúng vào đây.

CÚ PHÁP CỦA BỘ CHẤM XUẤT HIỆN TRONG CÁC CA

Biểu thức gọi chạy trong namespace của học viên và có sẵn đối tượng `_studio` của bộ chấm. Trong 20 bài này dùng tới: _studio.call, _studio.check, _studio.run, _studio.tk. Những dòng dưới trích từ chính bộ chấm (`frontend/public/static/js/studio-harness.py`):

```
_studio.run(vars=, files=, entry=, setup=, stdin=, fail_fork_at=, child_exit=) -> Result
_studio.call(fn, *args, **kw) -> Result      _studio.stdout(fn, ...) -> str
_studio.check('assert ...') -> True          _studio.expr('biểu thức')
_studio.tk                      = _studio.main.tk (TkState, xem dưới)
```

Trong khối code, dấu » ở cuối dòng nghĩa là dòng gốc dài hơn khổ giấy nên bị bọc xuống dòng dưới — không phải ký tự của chương trình.

BÀI NÀO CHẤM TRÊN BẢN MÔ PHỎNG

Rút máy móc từ chuỗi `step_4.env` của từng bài. Đây là bảng tra nhanh; chữ đầy đủ nằm ở khung cam trong trang bài.

Mã bài	Hệ thống bị mô phỏng
P01	subprocess, đồng hồ (time.sleep)
P02	subprocess, đồng hồ (time.sleep)
P04	subprocess, đồng hồ (time.sleep)
P05	subprocess, đồng hồ (time.sleep)
P06	subprocess, đồng hồ (time.sleep)
P07	multiprocessing, đồng hồ (time.sleep)
T01	threading, đồng hồ (time.sleep)
T02	threading, đồng hồ (time.sleep)
T03	threading, đồng hồ (time.sleep)
T04	đồng hồ (time.sleep)
T05	threading, đồng hồ (time.sleep)
T06	threading, đồng hồ (time.sleep)
G01	tkinter
G02	tkinter
G03	tkinter
G04	tkinter
G05	tkinter
G06	tkinter
R02	tkinter

Không có ghi chú mô phỏng trong env: P03.

Mục lục

Số trang là trang PDF. Mỗi bài bắt đầu ở một trang mới, trừ bài đầu của mỗi module (đi liền dưới tên module).

Cách đọc một trang đáp án 2

M1 · Tiến trình và subprocess 4

P01 · Program, process và tài nguyên . . . 4
P02 · Vòng đời, fork, exec và wait . . . 9
P03 · Scheduling và context switch . . . 13
P04 · stdin, stdout, stderr và pipe . . . 18
P05 · subprocess.run và kết quả . . . 23
P06 · Popen, communicate và timeout . . . 28
P07 · multiprocessing: Process và Queue . . . 32

M2 · Đa luồng 36

T01 · Thread, tài nguyên và concurrency . . . 36
T02 · Thread target, start và join . . . 40
T03 · Subclass Thread . . . 44
T04 · GIL, CPU-bound và I/O-bound . . . 48
T05 · Lock, race condition và deadlock . . . 52
T06 · Queue và tác vụ lưu nền . . . 58

M3 · GUI Tkinter và dự án 63

G01 · GUI, CLI và cửa sổ . . . 63
G02 · Widget và callback . . . 67
G03 · Dữ liệu nhập và lựa chọn . . . 72
G04 · Layout: pack, place, grid . . . 77
G05 · OOP và tách logic khỏi giao diện . . . 82
G06 · Event loop và worker nền . . . 87
R02 · Dự án PW5–PW9: lưu, shell, thread và GUI . . . 92

M1 · Tiến trình và subprocess

7 bài trong phạm vi: P01, P02, P03, P04, P05, P06, P07. Chấm trên bản mô phỏng: P01, P02, P04, P05, P06, P07.

P01 · Program, process và tài nguyên

Mã bài P01 · Trung cấp · M1 · Tiến trình và subprocess · 25 phút · 15 XP · thứ tự 1 · mức I

MỤC TIÊU BÀI

Phân biệt program với process; đọc PID và đi ngược cây process.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

TABLE mô phỏng kết quả lệnh ps: mỗi dòng (pid, ppid, program). Viết count_processes(table, program) đếm số PROCESS đang chạy program đó (mỗi dòng là một process); ancestors(table, pid) trả list PID đi ngược theo ppid: [pid, cha, ông, ...] tới khi ppid không còn trong bảng (pid không có trong bảng → ValueError); current_identity() trả (os.getpid(), os.getppid()) của chính process đang chạy.

Thực hành: Đếm theo DÒNG (mỗi dòng một process). Dựng dict pid → ppid một lần, rồi đi ngược theo ppid tới khi ppid không còn trong bảng.

Gọi (đề cho)	Kết quả mong đợi
`count_processes(TABLE, "python")`	`3`
`ancestors(TABLE, 410)`	`[410, 305, 120, 1]`
`ancestors(TABLE, 999)`	`ValueError`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`count_processes(table, program)`	Đếm số process đang chạy một program. · table: list (pid, ppid, program); program: str · int (0 nếu không có) · Không in, không sửa table.
`ancestors(table, pid)`	Đi ngược cây process theo ppid. · table như trên; pid: int · list[int] bắt đầu bằng pid; pid lạ → ValueError · Không phụ thuộc thứ tự dòng trong table.
`current_identity()`	Cho biết process hiện tại là ai, cha là ai. · không · tuple (os.getpid(), os.getppid()) · Không in.

LỜI GIẢI MẪU (qa.json · P01)

```
import os

# Mô phỏng kết quả lệnh ps: mỗi dòng (pid, ppid, program)
TABLE = [
    (1, 0, "init"),
    (120, 1, "bash"),
    (305, 120, "python"),
    (306, 120, "python"),
    (410, 305, "python"),
]


def count_processes(table, program):
    count = 0
    for pid, ppid, name in table:
        if name == program:
            count += 1
    return count


def ancestors(table, pid):
    parent_of = {}
    for row_pid, row_ppid, program in table:
        parent_of[row_pid] = row_ppid
    if pid not in parent_of:
        raise ValueError(f"không có process {pid} trong bảng")
    chain = [pid]
    while parent_of[pid] in parent_of:
        pid = parent_of[pid]
        chain.append(pid)
    return chain


def current_identity():
    return os.getpid(), os.getppid()


if __name__ == "__main__":
    print(count_processes(TABLE, "python"))
    print(ancestors(TABLE, 410))
    print(current_identity())
```

BỘ CHẤM — 9 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	TABLE: "python" có 3 process	`count_processes(TABLE, "python")` `→ 3`

2	Bảng khác: "sshd" có 1, "vim" có 0	`(count_processes([(1, 0, "systemd"), (900, 1, "sshd"), (77, 900, "bash"), »`
		`(5000, 77, "python")], "sshd"), count_processes([(1, 0, "systemd"), »`
		`(900, 1, "sshd"), (77, 900, "bash"), (5000, 77, "python")], "vim"))`
		`→ (1, 0)`

3	Một program, hai process	`count_processes([(1, 0, "init"), (7, 1, "code"), (8, 1, "code")], "code")` `→ 2`
4	ancestors(TABLE, 410)	`ancestors(TABLE, 410)` `→ [410, 305, 120, 1]`
5	Gốc cây: ancestors(TABLE, 1) → [1]	`ancestors(TABLE, 1)` `→ [1]`

6	PID không tăng dần	`ancestors([(1, 0, "systemd"), (900, 1, "sshd"), (77, 900, "bash"), (5000, »`
		`77, "python")], 5000)`
		`→ [5000, 77, 900, 1]`

7	Dòng con đứng trước dòng cha	`ancestors([(50, 7, "python"), (7, 1, "bash"), (1, 0, "init")], 50)` `→ [50, 7, 1]`
8	PID không có trong bảng → ValueError	`_studio.call(ancestors, TABLE, 999).error` `→ 'ValueError'`
9	current_identity() = (os.getpid(), os.getppid())	`current_identity()` `→ (__import__("os").getpid(), __import__("os").getppid())`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Đếm theo DÒNG (mỗi dòng một process). Dựng dict pid → ppid một lần, rồi đi ngược theo ppid tới khi ppid không còn trong bảng.

Cấu trúc. 1. count_processes: duyệt từng dòng (pid, ppid, name), cộng 1 khi name == program. 2. ancestors: dựng parent_of = {pid: ppid}; pid không có trong dict → raise ValueError. 3. chain = [pid]; khi parent_of[pid] còn trong bảng: pid = parent_of[pid], append. 4. current_identity: return os.getpid(), os.getppid() — không viết cứng số nào.

Lời giải từng phần. parent_of = {} for row_pid, row_ppid, program in table: parent_of[row_pid] = row_ppid ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — subprocess, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy subprocess, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• PID trong Studio là số GIẢ nhưng ổn định (process con luôn có PID khác cha); trên máy thật mỗi lần chạy là một số khác.

• Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng).

• Lệnh shell khác (ls, dir, ping...) không chạy được ở đây.

• Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

NGỘ NHẬN THƯỜNG GẶP: "Program và process là một"

Program là mã thụ động; process là một lần thực thi đang hoạt động. Một program có thể có nhiều process cùng lúc (mở ba cửa sổ python là ba process), mỗi process có PID, stack, heap, file descriptor riêng. Vì vậy đếm process phải đếm từng DÒNG trong bảng ps, không đếm tên program.

CÂU DỰ ĐOÁN — đáp án và giải thích

Chạy cùng một script hai lần liên tiếp. Hai lần đó có nhất thiết cùng PID không?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Có — cùng file thì cùng PID	PID gắn với process đang chạy, không gắn với program.
B	Có, nếu chạy trên cùng một máy	Cùng máy vẫn là hai lần thực thi khác nhau.
C ĐÚNG	**Không — mỗi lần chạy là một process mới, hệ điều hành cấp PID mới**	PID định danh process (một lần thực thi), không định danh file .py.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. os.getppid() trả về gì?

A	PID của chính process hiện tại	Đó là os.getpid().
B ĐÚNG	**PID của process cha đã tạo ra process hiện tại**	Nối ppid → pid ta đi ngược được cây process.
C	Số process con đang chạy	Không hàm os nào đếm con theo kiểu này.

Giải thích (nguyên văn JSON): Nối ppid → pid ta đi ngược được cây process.

2. File descriptor 0, 1, 2 của một process thường là gì?

A	Ba vùng nhớ stack, data, heap	File descriptor tham chiếu tài nguyên I/O, không phải vùng nhớ.
B	Ba process đầu tiên của hệ thống	Đó là PID, không phải file descriptor.
C ĐÚNG	**stdin, stdout, stderr**	Ba luồng chuẩn mỗi process có sẵn khi khởi động.

Giải thích (nguyên văn JSON): Ba luồng chuẩn mỗi process có sẵn khi khởi động.

3. Object tạo trong lúc chạy (vd. list(range(5))) nằm ở vùng nhớ nào của process?

A	Program counter	PC chỉ là vị trí lệnh đang chạy.
B ĐÚNG	**Heap — vùng cấp phát động**	Stack chỉ giữ khung lời gọi hàm và tham chiếu tới object.
C	File descriptor	File descriptor tham chiếu file, pipe, terminal.

Giải thích (nguyên văn JSON): Stack chỉ giữ khung lời gọi hàm và tham chiếu tới object.

CÂU PHẢN VÍ DỤ — đáp án

Bảng [(1, 0, "init"), (7, 1, "code"), (8, 1, "code")]: count_processes(bảng, "code") là?

Nhãn	Lựa chọn
A ĐÚNG	2
B	3
C	1

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao count_processes(TABLE, "python") là 3 chứ không phải 1?

Mẫu đạt: Ba dòng có program "python" là ba lần thực thi khác nhau với ba PID 305, 306, 410 — tức ba process, dù chỉ có một program "python". Đếm tên program khác nhau sẽ ra 1 và bỏ mất hai process đang chạy. Trường hợp đặc biệt: program không có dòng nào thì trả 0.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	ancestors(TABLE, 410) → chain = [410]
Bước 2	parent_of[410] là 305 → thêm 305
Bước 3	parent_of[305] là 120, parent_of[120] là 1 → thêm 120 rồi 1
Bước 4	parent_of[1] là 0, không có trong bảng → dừng, trả [410, 305, 120, 1]

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp chỗ so sánh khi đếm, chỗ dựng bảng cha và bước đi lên cây.

Yêu cầu thêm: Đổi sang bảng của máy khác (PID không tăng dần, dòng con đứng trước dòng cha) và một program không có process nào; thử cả PID không có trong bảng.

```
def count_processes(table, program):
    count = 0
    for pid, ppid, name in table:
        if {{slot1}}:
            count += 1
    return count


def ancestors(table, pid):
    parent_of = {}
    for row_pid, row_ppid, program in table:
        {{slot2}}
    if pid not in parent_of:
        raise ValueError(f"không có process {pid} trong bảng")
    chain = [pid]
    while parent_of[pid] in parent_of:
        pid = {{slot3}}
        chain.append(pid)
    return chain
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`name == program`
[Slot 2]	`parent_of[row_pid] = row_ppid`
[Slot 3]	`parent_of[pid]`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 2	`parent_of[row_ppid] = row_pid`	Ngược chiều: một cha nhiều con thì con sau ghi đè con trước.
ô 3	`row_ppid`	Biến của vòng for đã xong — chỉ là ppid của dòng cuối bảng.

ô 1	`name in program`	"py" in "python" là True — đếm nhầm program khác tên.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 3, ô 1, ô 2.

ĐÁP ÁN MINI-GAME (4 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [3, 2, 0, 1].

Bên trái	NỐI VỚI (đáp án)
ancestors(TABLE, 410)	**chain = [410]**
parent_of[410] là 305	**thêm 305**
parent_of[305] là 120, parent_of[120] là 1	**thêm 120 rồi 1**
parent_of[1] là 0, không có trong bảng	**dừng, trả [410, 305, 120, 1]**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`name == program`
ô 2	`parent_of[row_pid] = row_ppid`
ô 3	`parent_of[pid]`

Bẫy ở ô 2 — `parent_of[row_ppid] = row_pid`: Ngược chiều: một cha nhiều con thì con sau ghi đè con trước.
Bẫy ở ô 3 — `row_ppid`: Biến của vòng for đã xong — chỉ là ppid của dòng cuối bảng.
Bẫy ở ô 1 — `name in program`: "py" in "python" là True — đếm nhầm program khác tên.

parsons-2 · SẮP DÒNG (Parsons) — Ghép các dòng thành chương trình chạy đúng

Thứ tự đúng (theo mảng `lines`): `L12 → L13 → L14 → L15 → L16 → L17 → L20 → L21`. Thứ tự xáo lúc chơi là `order` = ['L15', 'D1', 'L14', 'L20', 'L13', 'L21', 'D0', 'L12', 'L17', 'L16'].

```
L12  def count_processes(table, program):
L13      count = 0
L14      for pid, ppid, name in table:
L15          if name == program:
L16              count += 1
L17      return count
L20  def ancestors(table, pid):
L21      parent_of = {}
```

Phần đầu/cuối đã cho sẵn, học viên không sắp: 11 dòng đầu + 18 dòng cuối.

DÒNG BẪY (không được dùng)	Vì sao sai
`            count = 1`	đếm program (có/không) thay vì đếm process
`        parent_of[row_ppid] = row_pid`	nhầm chiều pid ↔ ppid

spotError-3 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 17 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 10).
Lỗi: đếm program (có/không) thay vì đếm process
Sửa thành:

```
            count += 1
```

```
    10 | ]
    11 |
    12 |
    13 | def count_processes(table, program):
    14 |     count = 0
    15 |     for pid, ppid, name in table:
    16 |         if name == program:
>>  17 |             count = 1
    18 |     return count
    19 |
    20 |
    21 | def ancestors(table, pid):
    22 |     parent_of = {}
    23 |     for row_pid, row_ppid, program in table:
    24 |         parent_of[row_pid] = row_ppid
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — đếm program (có/không) thay vì đếm process

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
17	`            count += 1`	`            count = 1`

KIỂU SAI 2 — nhầm chiều pid ↔ ppid

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
24	`parent_of[row_pid] = row_ppid`	`parent_of[row_ppid] = row_pid`

KIỂU SAI 3 — nhầm getpid với getppid

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
35	`return os.getpid(), os.getppid()`	`return os.getpid(), os.getpid()`

Nguồn trang này: pe-v4 · 001-P01.json · 001-P01.json · qa.json[P01]

P02 · Vòng đời, fork, exec và wait

Mã bài P02 · Trung cấp · M1 · Tiến trình và subprocess · 35 phút · 15 XP · thứ tự 2 · mức I

MỤC TIÊU BÀI

Theo dõi vòng đời process; chờ con để lấy exit code.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết next_state(state, event) theo sơ đồ 5 trạng thái: new –admit→ ready, ready –dispatch→ running, running –interrupt→ ready, running –io_wait→ waiting, waiting –io_done→ ready, running –exit→ terminated; cặp (state, event) khác → ValueError. lifecycle(events) đã cấp, dùng next_state. Rồi viết run_child(code): tạo process con bằng subprocess.Popen([sys.executable, "-c", code]), chờ con kết thúc bằng wait() và trả exit code.

Thực hành: Viết sơ đồ thành dict {(state, event): trạng thái mới}; cặp không có trong dict thì raise ValueError. Với process con: Popen → wait() → trả giá trị của wait().

Gọi (đề cho)	Kết quả mong đợi
`next_state("waiting", "io_done")`	`"ready"`
`next_state("new", "dispatch")`	`ValueError`
`run_child("raise SystemExit(3)")`	`3`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`next_state(state, event)`	Một bước trong sơ đồ 5 trạng thái. · state, event: str · str trạng thái mới; cặp không hợp lệ → ValueError · Không in.
`run_child(code)`	Tạo process con chạy code bằng Python hiện tại và chờ nó. · code: str (mã Python) · int exit code của con · Con in gì thì in thẳng ra màn hình (không thu).

LỜI GIẢI MẪU (qa.json · P02)

```
import subprocess
import sys

# Sơ đồ 5 trạng thái (sự kiện ghi trên mũi tên):
#   new --admit--> ready --dispatch--> running --exit--> terminated
#   running --interrupt--> ready     (hết time slice, bị ngắt)
#   running --io_wait--> waiting     (chờ đọc đĩa, chờ mạng...)
#   waiting --io_done--> ready       (I/O xong: xếp hàng lại)

TRANSITIONS = {
    ("new", "admit"): "ready",
    ("ready", "dispatch"): "running",
    ("running", "interrupt"): "ready",
    ("running", "io_wait"): "waiting",
    ("waiting", "io_done"): "ready",
    ("running", "exit"): "terminated",
}


def next_state(state, event):
    if (state, event) not in TRANSITIONS:
        raise ValueError(f"không có chuyển {state} --{event}-->")
    return TRANSITIONS[(state, event)]


def lifecycle(events):
    states = ["new"]
    for event in events:
        states.append(next_state(states[-1], event))
    return states


def run_child(code):
    child = subprocess.Popen([sys.executable, "-c", code])
    return child.wait()


if __name__ == "__main__":
    print(lifecycle(["admit", "dispatch", "io_wait", "io_done", "dispatch", "exit"]))
    print(run_child("raise SystemExit(3)"))
```

BỘ CHẤM — 9 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	new --admit--> ready	`next_state("new", "admit")` → `'ready'`
2	I/O xong → ready, không phải running	`next_state("waiting", "io_done")` → `'ready'`

3	Bị ngắt (hết time slice) → ready	`next_state("running", "interrupt")` `→ 'ready'`

4	Vòng đời có chờ I/O và bị ngắt	`lifecycle(["admit", "dispatch", "interrupt", "dispatch", "io_wait", »`
		`  "io_done", "dispatch", "exit"])`
		`→ ['new', 'ready', 'running', 'ready', 'running', 'waiting', 'ready', »`
		`  'running', 'terminated']`

5	Chuyển không hợp lệ → ValueError	`[_studio.call(next_state, s, e).error for s, e in [("new", "dispatch"), »`
		`  ("waiting", "dispatch"), ("terminated", "admit"), ("ready", "exit")]]`
		`→ ['ValueError'] * 4`

6	run_child("pass") → 0	`run_child("pass")` `→ 0`
7	SystemExit(3) → 3, sys.exit(7) → 7	`(run_child("raise SystemExit(3)"), run_child("import sys; sys.exit(7)"))` `→ (3, 7)`
8	Ngoại lệ không bắt trong con → 1	`run_child("1/0")` `→ 1`
9	Dùng Popen rồi tự chờ (không gọi run)	`[c["kind"] for c in _studio.call(run_child, "pass").subprocess_calls]` `→ ['Popen']`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Viết sơ đồ thành dict {(state, event): trạng thái mới}; cặp không có trong dict thì raise ValueError. Với process con: Popen → wait() → trả giá trị của wait().

Cấu trúc. 1. Liệt kê đủ 6 cặp (state, event) hợp lệ vào dict TRANSITIONS. 2. next_state: cặp không có trong dict → raise ValueError; có thì trả trạng thái mới. 3. run_child: child = subprocess.Popen([sys.executable, "-c", code]). 4. return child.wait() — không đọc child.returncode trước khi chờ.

Lời giải từng phần. def run_child(code): child = subprocess.Popen([sys.executable, "-c", code]) ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — subprocess, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy subprocess, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng).
- Lệnh shell khác (ls, dir, ping...) không chạy được ở đây.
- Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

NGỘ NHẬN THƯỜNG GẶP: "exec() tạo ra một process mới"

exec() thay image chương trình của CHÍNH process đang chạy; PID không đổi. Tạo process mới là việc của fork() (UNIX) hay CreateProcess (Windows); subprocess trên Linux fork rồi exec trong con. Ngộ nhận đi kèm: đọc child.returncode ngay sau Popen để lấy exit code — lúc đó nó còn là None, phải wait() trước.

CÂU DỰ ĐOÁN — đáp án và giải thích

Ngay sau child = subprocess.Popen([...]) (chưa gọi wait/poll), child.returncode là gì?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	PID của con	PID nằm ở child.pid.
B ĐÚNG	**None — Popen trả object ngay, chưa ai thu trạng thái kết thúc của con**	returncode chỉ được điền khi poll()/wait()/communicate() thấy con đã kết thúc.
C	0	0 chỉ có sau khi con đã kết thúc thành công VÀ đã được thu.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Sau os.fork() thành công, giá trị trả về trong process CON là?

A ĐÚNG	**0**	Cha nhận PID của con, con nhận 0 — nhờ vậy if pid == 0 tách được hai nhánh.
B	PID của con	Đó là giá trị cha nhận.
C	PID của cha	Con muốn biết cha thì gọi os.getppid().

Giải thích (nguyên văn JSON): Cha nhận PID của con, con nhận 0 — nhờ vậy if pid == 0 tách được hai nhánh.

2. Process đang waiting (chờ đọc đĩa) và I/O vừa xong. Nó chuyển sang...

A ĐÚNG	**ready — xếp hàng chờ scheduler cho chạy tiếp**	CPU có thể đang chạy process khác; I/O xong chỉ có nghĩa là SẴN SÀNG.
B	terminated	Chờ I/O xong không có nghĩa là kết thúc.
C	running ngay lập tức	Phải qua ready và được scheduler chọn (dispatch).

Giải thích (nguyên văn JSON): CPU có thể đang chạy process khác; I/O xong chỉ có nghĩa là SẴN SÀNG.

3. exec() có luôn tạo PID mới không?

A	Có	exec không tạo thêm process.
B	Chỉ trên Windows	Windows không có exec theo nghĩa UNIX; tạo process bằng CreateProcess.
C ĐÚNG	**Không — nó thay chương trình của process hiện tại, PID giữ nguyên**	Tạo process mới là fork()/CreateProcess.

Giải thích (nguyên văn JSON): Tạo process mới là fork()/CreateProcess.

CÂU PHẢN VÍ DỤ — đáp án

run_child("import sys; sys.exit(7)") trả gì?

Nhãn	Lựa chọn
A	`None`
B	`0`
C ĐÚNG	`7`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao I/O xong thì process về ready chứ không về running?

Mẫu đạt: Trong lúc process chờ I/O, scheduler đã giao CPU cho process khác. Khi I/O xong, process chỉ SẴN SÀNG chạy tiếp nên phải xếp vào hàng ready chờ scheduler chọn lại (dispatch). Nhảy thẳng sang running là cướp CPU của process đang chạy — chỉ scheduler quyết định việc đó. Trường hợp đặc biệt: cặp không có trong sơ đồ (vd. waiting + dispatch) thì báo ValueError.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	Popen([sys.executable, "-c", code]) → con được tạo, cha chạy tiếp ngay
Bước 2	child.returncode lúc này → None: chưa ai thu trạng thái
Bước 3	child.wait() → cha chặn tới khi con terminated
Bước 4	wait() trả về → exit code: 0, n của SystemExit(n), 1 nếu lỗi

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp chuyển trạng thái hay nhầm, cách báo lỗi và lệnh chờ con.

Yêu cầu thêm: Thử chuỗi sự kiện có hai lần bị ngắt và một lần chờ I/O; thử con thoát bằng SystemExit(7) và con ném ZeroDivisionError.

```
TRANSITIONS = {
    ("new", "admit"): "ready",
    ("ready", "dispatch"): "running",
    ("running", "interrupt"): "ready",
    ("running", "io_wait"): "waiting",
    ("waiting", "io_done"): {{slot1}},
    ("running", "exit"): "terminated",
}


def next_state(state, event):
    if (state, event) not in TRANSITIONS:
        {{slot2}}
    return TRANSITIONS[(state, event)]


def run_child(code):
    child = subprocess.Popen([sys.executable, "-c", code])
    return {{slot3}}
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`"ready"`
[Slot 2]	`raise ValueError("invalid transition")`
[Slot 3]	`child.wait()`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 2	`return None`	Âm thầm trả None — lỗi chuyển trạng thái bị che mất.
ô 1	`"running"`	I/O xong chưa chắc có CPU: phải về ready chờ dispatch.
ô 3	`child.returncode`	Đọc ngay sau Popen là None: chưa ai chờ con.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 1, ô 3, ô 2.

ĐÁP ÁN MINI-GAME (4 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [2, 3, 0, 1].

Bên trái	NỐI VỚI (đáp án)
Popen([sys.executable, "-c", code])	**con được tạo, cha chạy tiếp ngay**
child.returncode lúc này	**None: chưa ai thu trạng thái**
child.wait()	**cha chặn tới khi con terminated**
wait() trả về	**exit code: 0, n của SystemExit(n), 1 nếu lỗi**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`"ready"`
ô 2	`raise ValueError("invalid transition")`
ô 3	`child.wait()`

Bẫy ở ô 2 — `return None`: Âm thầm trả None — lỗi chuyển trạng thái bị che mất.
Bẫy ở ô 1 — `"running"`: I/O xong chưa chắc có CPU: phải về ready chờ dispatch.
Bẫy ở ô 3 — `child.returncode`: Đọc ngay sau Popen là None: chưa ai chờ con.

parsons-2 · SẮP DÒNG (Parsons) — Ghép các dòng thành chương trình chạy đúng

Thứ tự đúng (theo mảng `lines`): `L28 → L29 → L32 → L33 → L34 → L37 → L38 → L39`. Thứ tự xáo lúc chơi là `order` = ['D1', 'L33', 'D0', 'L38', 'L34', 'L39', 'L29', 'L32', 'L28', 'L37'].

```
L28          states.append(next_state(states[-1], event))
L29      return states
L32  def run_child(code):
L33      child = subprocess.Popen([sys.executable, "-c", code])
L34      return child.wait()
L37  if __name__ == "__main__":
L38      print(lifecycle(["admit", "dispatch", "io_wait", "io_done", "dispatch", "exit"]))
L39      print(run_child("raise SystemExit(3)"))
```

Phần đầu/cuối đã cho sẵn, học viên không sắp: 27 dòng đầu + 0 dòng cuối.

DÒNG BẪY (không được dùng)	Vì sao sai
`return child.returncode`	đọc returncode trước khi chờ
`("waiting", "io_done"): "running",`	I/O xong nhảy thẳng vào running

spotError-3 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 35 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 28).
Lỗi: đọc returncode trước khi chờ
Sửa thành:

```
return child.wait()
```

```
28 |     for event in events:
29 |         states.append(next_state(states[-1], event))
30 |     return states
31 |
32 |
33 | def run_child(code):
34 |     child = subprocess.Popen([sys.executable, "-c", code])
>> 35 |     return child.returncode
36 |
37 |
38 | if __name__ == "__main__":
39 |     print(lifecycle(["admit", "dispatch", "io_wait", "io_done", "dispatch", "exit"]))
40 |     print(run_child("raise SystemExit(3)"))
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 2 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — đọc returncode trước khi chờ

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
35	`return child.wait()`	`return child.returncode`

KIỂU SAI 2 — I/O xong nhảy thẳng vào running

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
15	`("waiting", "io_done"): "ready",`	`("waiting", "io_done"): "running",`

Nguồn trang này: pe-v4 · 002-P02.json · 002-P02.json · qa.json[P02]

P03 · Scheduling và context switch

Mã bài P03 · Trung cấp · M1 · Tiến trình và subprocess · 35 phút · 15 XP · thứ tự 3 · mức I

MỤC TIÊU BÀI

Mô phỏng Round Robin và tính thời gian chờ.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết round_robin(jobs, quantum, switch=0): jobs là list (tên, burst) — burst nguyên dương, mọi job đến lúc 0, hàng ready theo đúng thứ tự list. Mỗi lượt lấy process đầu hàng, cho chạy min(quantum, phần còn lại) đơn vị và ghi (tên, bắt đầu, kết thúc) vào timeline; chưa xong thì về CUỐI hàng. Mỗi khi CPU chuyển sang một process KHÁC process vừa chạy, đồng hồ cộng thêm switch đơn vị (context switch, không ai làm việc). quantum <= 0 → ValueError; không sửa list jobs. Rồi viết waiting_times(jobs, timeline): dict tên → lúc hoàn thành − burst.

Thực hành: Hàng ready là deque các cặp (tên, phần còn lại). Mỗi lượt: popleft, cộng switch nếu đổi process, chạy min(quantum, phần còn lại), chưa xong thì append về cuối.

Gọi (đề cho)	Kết quả mong đợi
`round_robin([("P1", 5), ("P2", 3), ("P3", 1)], 2)`	`P1 0–2, P2 2–4, P3 4–5, P1 5–7, P2 7–8, P1 8–9`
`round_robin([("P1", 3), ("P2", 2)], 2, switch=1)`	`P1 0–2, P2 3–5, P1 6–7`
`round_robin([("A", 1)], 0)`	`ValueError`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`round_robin(jobs, quantum, switch=0)`	Mô phỏng Round Robin trên một lõi. · jobs: list (tên, burst > 0); quantum: int > 0; switch: int >= 0 · list (tên, bắt đầu, kết thúc) theo thứ tự chạy; quantum <= 0 → ValueError · Không sửa jobs, không in.
`waiting_times(jobs, timeline)`	Thời gian chờ của từng job. · jobs như trên; timeline do round_robin trả · dict tên → lúc hoàn thành − burst · Không in.

LỜI GIẢI MẪU (qa.json · P03)

```
from collections import deque


def round_robin(jobs, quantum, switch=0):
    if quantum <= 0:
        raise ValueError("quantum must be positive")
    ready = deque(jobs)
    clock = 0
    last = None
    timeline = []
    while ready:
        name, left = ready.popleft()
        if last is not None and name != last:
            clock += switch
        run = min(quantum, left)
        timeline.append((name, clock, clock + run))
        clock += run
        last = name
        if left > run:
            ready.append((name, left - run))
    return timeline


def waiting_times(jobs, timeline):
    finish = {}
    for name, start, end in timeline:
        finish[name] = end
    return {name: finish[name] - burst for name, burst in jobs}


if __name__ == "__main__":
    jobs = [("P1", 5), ("P2", 3), ("P3", 1)]
    timeline = round_robin(jobs, 2)
    print(timeline)
    print(waiting_times(jobs, timeline))
```

BỘ CHẤM — 10 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Ví dụ đề: quantum = 2	`round_robin([("P1", 5), ("P2", 3), ("P3", 1)], 2)` `→ [('P1', 0, 2), ('P2', 2, 4), ('P3', 4, 5), ('P1', 5, 7), ('P2', 7, 8), » ('P1', 8, 9)]`
2	quantum ≥ burst → như FCFS	`round_robin([("A", 4), ("B", 4)], 4)` `→ [('A', 0, 4), ('B', 4, 8)]`
3	quantum = 1, ba job	`round_robin([("X", 2), ("Y", 1), ("Z", 2)], 1)` `→ [('X', 0, 1), ('Y', 1, 2), ('Z', 2, 3), ('X', 3, 4), ('Z', 4, 5)]`

4	switch = 1 mỗi lần đổi process	`round_robin([("P1", 3), ("P2", 2)], 2, switch=1)` `→ [('P1', 0, 2), ('P2', 3, 5), ('P1', 6, 7)]`
5	Cùng một process chạy liền hai lượt: không tốn switch	`round_robin([("P1", 5), ("P2", 1)], 2, switch=1)` `→ [('P1', 0, 2), ('P2', 3, 4), ('P1', 5, 7), ('P1', 7, 8)]`

6	Thời gian chờ (ví dụ đề)	`waiting_times([("P1", 5), ("P2", 3), ("P3", 1)], round_robin([("P1", 5), »`
		`("P2", 3), ("P3", 1)], 2))`
		`→ {'P1': 4, 'P2': 5, 'P3': 4}`

7	Thời gian chờ tính cả overhead switch	`waiting_times([("P1", 3), ("P2", 2)], round_robin([("P1", 3), ("P2", 2)], »`
		`2, switch=1))`
		`→ {'P1': 4, 'P2': 3}`

8	Không có job → []	`round_robin([], 3)` `→ []`
9	quantum <= 0 → ValueError	`[_studio.call(round_robin, [("A", 1)], q).error for q in (0, -2)]` `→ ['ValueError', 'ValueError']`

10	Không sửa list jobs	`_studio.check('j = [("A", 3), ("B", 1)]\nround_robin(j, 1)\nassert j == »`
		`[("A", 3), ("B", 1)], "round_robin không được sửa jobs"')`
		`→ True`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Hàng ready là deque các cặp (tên, phần còn lại). Mỗi lượt: popleft, cộng switch nếu đổi process, chạy min(quantum, phần còn lại), chưa xong thì append về cuối.

Cấu trúc. 1. quantum <= 0 → raise ValueError trước khi chạy. 2. ready = deque(jobs) (bản sao, không sửa jobs); clock = 0; last = None; timeline = []. 3. Trong while ready: popleft; last khác None và khác tên hiện tại → clock += switch. 4. run = min(quantum, left); ghi (tên, clock, clock + run); clock += run; còn dư thì append về cuối. 5. waiting_times: lúc hoàn thành = kết thúc của lát CUỐI mang tên đó; chờ = hoàn thành − burst.

Lời giải từng phần. while ready: name, left = ready.popleft() ... run = min(quantum, left) timeline.append((name, clock, clock + run))

MÔI TRƯỜNG CHẤM

JSON của bài không khai step_4.env (ghi chú môi trường) — không có cảnh báo mô phỏng nào để đọc — chỗ này KHÔNG được suy diễn thêm.

NGỘ NHẬN THƯỜNG GẶP: "Hết quantum thì process vẫn chạy tiếp tới xong"

Đó là FCFS (non-preemptive). Round Robin là preemptive: hết time slice, scheduler NGẮT process, lưu ngữ cảnh và đưa nó về cuối hàng ready. Ngộ nhận thứ hai: context switch "miễn phí" — mỗi lần đổi process tốn thời gian lưu/nạp PCB mà không làm việc của ứng dụng, nên quantum quá nhỏ làm mọi job xong muộn hơn.

CÂU DỰ ĐOÁN — đáp án và giải thích

jobs = [("P1", 5), ("P2", 3), ("P3", 1)], quantum = 2. Process nào XONG ĐẦU TIÊN?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	P1 — nó đứng đầu hàng	Đứng đầu chỉ được chạy trước 2 đơn vị rồi bị ngắt.
B	P2	P2 cần 3 > quantum nên lượt đầu chưa xong.
C ĐÚNG	**P3, lúc 5 — nó chỉ cần 1 đơn vị, tới lượt ở lần thứ ba**	P1 0–2, P2 2–4, P3 4–5: P3 xong ngay trong lượt đầu của nó.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Concurrency có bắt buộc CPU nhiều lõi không?

A	Có, ít nhất bằng số process	Hệ điều hành chạy hàng trăm process trên vài lõi.
B	Có	Một lõi + time slice vẫn cho nhiều tác vụ cùng tiến triển.
C ĐÚNG	**Không — một lõi xen kẽ các tác vụ cũng tạo concurrency; nhiều lõi mới chạy song song thật**	Concurrency là cùng tiến triển; parallelism là cùng chạy một lúc.

Giải thích (nguyên văn JSON): Concurrency là cùng tiến triển; parallelism là cùng chạy một lúc.

2. Trong lúc context switch, CPU đang làm gì?

A	Chạy cả hai process cùng lúc	Một lõi chỉ chạy một luồng lệnh tại một thời điểm.
B ĐÚNG	**Lưu ngữ cảnh process cũ, nạp ngữ cảnh process mới (PCB) — không chạy code ứng dụng**	Vì thế switch là overhead.
C	Chạy tiếp process cũ	Process cũ vừa bị ngắt.

Giải thích (nguyên văn JSON): Vì thế switch là overhead.

3. Process phần lớn thời gian chờ đĩa/mạng gọi là gì?

A	Preemptive	Preemptive là kiểu scheduler, không phải kiểu process.
B ĐÚNG	**I/O-bound**	Nó nằm trong waiting nhiều hơn running.

C	CPU-bound	CPU-bound chủ yếu tính toán.

Giải thích (nguyên văn JSON): Nó nằm trong waiting nhiều hơn running.

CÂU PHẢN VÍ DỤ — đáp án

[("A", 4), ("B", 4)], quantum = 4: timeline là?

Nhãn	Lựa chọn
A ĐÚNG	`A 0–4, B 4–8`
B	`B 0–4, A 4–8`
C	`A 0–2, B 2–4, A 4–6, B 6–8`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Khi switch > 0, vì sao giảm quantum làm các job xong MUỘN hơn?

Mẫu đạt: Quantum nhỏ cắt mỗi job thành nhiều lượt hơn nên CPU đổi process nhiều lần hơn; mỗi lần đổi tốn switch đơn vị không làm việc của ai. Tổng burst giữ nguyên nhưng cộng thêm nhiều overhead, nên lúc hoàn thành và thời gian chờ đều tăng. Trường hợp đặc biệt: chỉ còn một process thì nó chạy các lượt liền nhau, không tốn switch.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	ready.popleft() → lấy process đầu hàng ready
Bước 2	đổi sang process khác → clock += switch (overhead)
Bước 3	run = min(quantum, left) → chạy tối đa một time slice
Bước 4	left > run → về cuối hàng với phần còn lại left - run

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp ba chỗ của vòng Round Robin.

Yêu cầu thêm: Thử quantum lớn hơn mọi burst (thành FCFS), quantum = 1, và switch = 1 khi một process phải chạy hai lượt liền nhau ở cuối.

```
from collections import deque


def round_robin(jobs, quantum, switch=0):
    if quantum <= 0:
        raise ValueError("quantum must be positive")
    ready = deque(jobs)
    clock = 0
    last = None
    timeline = []
    while ready:
        name, left = ready.popleft()
        if last is not None and name != last:
            {{slot1}}
        run = {{slot2}}
        timeline.append((name, clock, clock + run))
        clock += run
        last = name
        if left > run:
            {{slot3}}
    return timeline
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`clock += switch`
[Slot 2]	`min(quantum, left)`
[Slot 3]	`ready.append((name, left - run))`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 3	`ready.appendleft((name, left - run))`	Về ĐẦU hàng: process vừa bị ngắt lại chạy tiếp, process khác phải chờ mãi.
ô 1	`clock += 1`	Chi phí switch là tham số, không cố định 1.
ô 2	`left`	Chạy tới hết — thành FCFS, không còn preemptive.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 1, ô 3, ô 2.

ĐÁP ÁN MINI-GAME (4 màn, nguồn out-advanced · studio-lesson/v2)

traceTable-0 · BẢNG THEO DÕI BIẾN — Theo dõi biến: điền giá trị sau mỗi dòng

```
ready = ["A", "B", "C"]
clock = 0
quantum = 3
for turn in range(4):
```

```
    current = ready.pop(0)
    clock = clock + quantum
    print(current, "chạy tới thời điểm", clock)
    ready.append(current)
```

#	Dòng chạy	clock	quantum	turn
0	`clock = 0`	0	–	–
1	`quantum = 3`	0	**3**	–
2	`for turn in range(4):`	0	3	**0**
3	`current = ready.pop(0)`	0	3	0
4	`clock = clock + quantum`	**3**	3	0
5	`for turn in range(4):`	3	3	**1**

Ô IN XANH ĐẬM là ô học viên phải điền (4 ô); ô xám là giá trị bài cho sẵn. So sánh bỏ nháy và không phân biệt hoa/thường.

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`clock += switch`
ô 2	`min(quantum, left)`
ô 3	`ready.append((name, left - run))`

Bẫy ở ô 3 — `ready.appendleft((name, left - run))`: Về ĐẦU hàng: process vừa bị ngắt lại chạy tiếp, process khác phải chờ mãi.
Bẫy ở ô 1 — `clock += 1`: Chi phí switch là tham số, không cố định 1.
Bẫy ở ô 2 — `left`: Chạy tới hết — thành FCFS, không còn preemptive.

parsons-2 · SẮP DÒNG (Parsons) — Ghép các dòng thành chương trình chạy đúng

Thứ tự đúng (theo mảng `lines`): `L10 → L11 → L12 → L13 → L14 → L15 → L16 → L17`. Thứ tự xáo lúc chơi là `order` = ['L12', 'D1', 'L13', 'D0', 'L11', 'L10', 'L14', 'L15', 'L17', 'L16'].

```
L10      while ready:
L11          name, left = ready.popleft()
L12          if last is not None and name != last:
L13              clock += switch
L14          run = min(quantum, left)
L15          timeline.append((name, clock, clock + run))
L16          clock += run
L17          last = name
```

Phần đầu/cuối đã cho sẵn, học viên không sắp: 9 dòng đầu + 16 dòng cuối.

DÒNG BẪY (không được dùng)	Vì sao sai
`        run = left`	không ngắt khi hết quantum (thành FCFS)
`    return {name: finish[name] for name, burst in jobs}`	nhầm thời gian chờ với lúc hoàn thành

spotError-3 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 15 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 8).
Lỗi: không ngắt khi hết quantum (thành FCFS)
Sửa thành:

```
        run = min(quantum, left)
```

```
     8 |     clock = 0
     9 |     last = None
    10 |     timeline = []
    11 |     while ready:
    12 |         name, left = ready.popleft()
    13 |         if last is not None and name != last:
    14 |             clock += switch
>>  15 |         run = left
    16 |         timeline.append((name, clock, clock + run))
    17 |         clock += run
    18 |         last = name
    19 |         if left > run:
    20 |             ready.append((name, left - run))
    21 |     return timeline
    22 |
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — không ngắt khi hết quantum (thành FCFS)

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
15	`        run = min(quantum, left)`	`        run = left`

KIỂU SAI 2 — coi context switch là miễn phí

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
13	`        if last is not None and name != last:`	(xoá dòng)
14	`            clock += switch`	(xoá dòng)

KIỂU SAI 3 — nhầm thời gian chờ với lúc hoàn thành

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
28	`    return {name: finish[name] - burst for name, burst in jobs}`	`    return {name: finish[name] for name, burst in jobs}`

Nguồn trang này: pe-v4 · 003-P03.json · 003-P03.json · qa.json[P03]

P04 · stdin, stdout, stderr và pipe

Mã bài P04 · Trung cấp · M1 · Tiến trình và subprocess · 35 phút · 15 XP · thứ tự 4 · mức I

MỤC TIÊU BÀI

Phân biệt stdin, stdout, stderr và nối chúng với process con.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hai phía của một process. (1) read_scores(): đọc HẾT các dòng từ stdin (sys.stdin); dòng rỗng bỏ qua, dòng là số nguyên thì giữ (int), dòng khác thì in "skip: <dòng đã strip>" ra STDERR — không ra stdout; trả list số. (2) run_sum(lines): chạy sum_numbers.py (đã cấp) làm process con bằng [sys.executable, "sum_numbers.py"], đưa các dòng vào stdin của con qua input= (mỗi dòng kết thúc bằng "\n"), thu riêng stdout và stderr dạng chuỗi; trả (tổng: int, list các dòng cảnh báo).

Thực hành: Phía con: đọc sys.stdin, cảnh báo ra sys.stderr. Phía cha: input= đưa dữ liệu vào, capture_output=True tách hai pipe, text=True để có str.

Gọi (đề cho)	Kết quả mong đợi
`stdin: 5 / abc / -2`	`read_scores() → [5, -2]; stderr: skip: abc`
`run_sum(["10", "x", "20"])`	`(30, ["skip: x"])`
`run_sum([])`	`(0, [])`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`read_scores()`	Đọc số nguyên từ stdin của chính process. · không (đọc sys.stdin tới EOF) · list[int] · Dòng không phải số → "skip: <dòng>" ra stderr; stdout không in gì.
`run_sum(lines)`	Chạy sum_numbers.py làm process con và đọc kết quả. · lines: list[str] — mỗi phần tử một dòng · tuple (int tổng, list[str] cảnh báo) · Lệnh con: [sys.executable, "sum_numbers.py"]; dữ liệu qua input=.

LỜI GIẢI MẪU (qa.json · P04)

```
import subprocess
import sys


def read_scores():
    scores = []
    for line in sys.stdin:
        text = line.strip()
        if text == "":
            continue
        try:
            scores.append(int(text))
        except ValueError:
            print("skip:", text, file=sys.stderr)
    return scores


def run_sum(lines):
    data = "".join(line + "\n" for line in lines)
    result = subprocess.run([sys.executable, "sum_numbers.py"],
                            input=data, capture_output=True, text=True)
    return int(result.stdout), result.stderr.splitlines()


if __name__ == "__main__":
    print(run_sum(["10", "x", "20"]))
```

BỘ CHẤM — 8 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	stdin 7, 8 → [7, 8]	`_studio.run(stdin=['7', '8'], entry='read_scores()').value` `→ [7, 8]`
2	Bỏ dòng rỗng, giữ số âm, strip khoảng trắng	`_studio.run(stdin=['5', '', ' -2 ', '40'], entry='read_scores()').value` `→ [5, -2, 40]`

3	Cảnh báo ra stderr, stdout sạch	`(lambda r: (r.ns['_got'], r.ns['_err'].getvalue(), »`
		`r.stdout))(_studio.run(stdin=['5', 'abc', '3.5'], entry='import io, »`
		`contextlib\n_err = io.StringIO()\nwith »`
		`contextlib.redirect_stderr(_err):\n _got = read_scores()'))`
		`→ ([5], 'skip: abc\nskip: 3.5\n', '')`

4	stdin rỗng → []	`_studio.run(stdin=[], entry='read_scores()').value` `→ []`
5	run_sum: 10, x, 20	`run_sum(["10", "x", "20"])` `→ (30, ['skip: x'])`

6	run_sum: số âm, nhiều dòng hỏng, dòng rỗng	`run_sum(["-4", "a", "b", "9", ""])` `→ (5, ['skip: a', 'skip: b'])`
7	run_sum([]) → (0, [])	`run_sum([])` `→ (0, [])`
8	Lệnh con là list [sys.executable, "sum_numbers.py"]	`[list(c["args"]) for c in _studio.call(run_sum, ["1"]).subprocess_calls]` `→ [[__import__("sys").executable, "sum_numbers.py"]]`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Phía con: đọc sys.stdin, cảnh báo ra sys.stderr. Phía cha: input= đưa dữ liệu vào, capture_output=True tách hai pipe, text=True để có str.

Cấu trúc. 1. read_scores: for line in sys.stdin; text = line.strip(); bỏ dòng rỗng. 2. int(text) trong try; ValueError → print("skip:", text, file=sys.stderr). 3. run_sum: data = mỗi dòng + "\n"; subprocess.run([sys.executable, "sum_numbers.py"], input=data, capture_output=True, text=True). 4. Trả int(result.stdout) và result.stderr.splitlines().

Lời giải từng phần. except ValueError: print("skip:", text, file=sys.stderr) ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — subprocess, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy subprocess, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng).
- Lệnh shell khác (ls, dir, ping...) không chạy được ở đây.
- Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

FILE CẤP SẴN CHO BÀI · sum_numbers.py (module) — học viên không phải viết file này

```
import sys

# Đọc số nguyên từ stdin (mỗi dòng một số), in TỔNG ra stdout.
# Dòng không phải số nguyên: báo "skip: <dòng>" ra stderr.
total = 0
for line in sys.stdin:
    text = line.strip()
    if text == "":
        continue
    try:
        total += int(text)
    except ValueError:
        print("skip:", text, file=sys.stderr)
print(total)
```

NGỘ NHẬN THƯỜNG GẶP: "stderr cũng là stdout, in đâu cũng được"

stderr là luồng RIÊNG (fd 2). In cảnh báo ra stdout làm hỏng kết quả: chương trình đọc stdout của bạn (qua pipe, > out.txt, hay r.stdout) sẽ nhận cả "skip: abc" lẫn con số. Chỉ khi cấu hình gộp — 2>&1 trong shell, stderr=subprocess.STDOUT trong Python — hai luồng mới đi chung một đường.

CÂU DỰ ĐOÁN — đáp án và giải thích

Con chạy print(5) rồi print("warn", file=sys.stderr). Với capture_output=True, text=True, r.stdout là gì?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A ĐÚNG	**"5\n" — "warn" nằm riêng trong r.stderr**	Hai luồng được thu bằng hai pipe khác nhau.
B	"5\nwarn\n"	Chỉ gộp khi đặt stderr=subprocess.STDOUT.
C	b"5\n"	text=True đã đổi bytes thành str.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Lệnh shell ps aux | grep term làm gì với stdout của ps?

A	Ghi vào file tên grep	Ghi ra file là >.
B	Gộp vào stderr	Pipe không đụng tới stderr.
C ĐÚNG	**Nối nó vào stdin của grep qua một pipe**	| là pipe giữa hai process.

Giải thích (nguyên văn JSON): | là pipe giữa hai process.

2. subprocess.run(..., capture_output=True) KHÔNG có text=True. r.stdout có kiểu gì?

A	str	Cần text=True.
B ĐÚNG	**bytes**	Mặc định dữ liệu qua pipe là bytes; text=True (hoặc encoding=) mới cho str.
C	list các dòng	Muốn list thì tự splitlines().

Giải thích (nguyên văn JSON): Mặc định dữ liệu qua pipe là bytes; text=True (hoặc encoding=) mới cho str.

3. for line in sys.stdin dừng khi nào?

A ĐÚNG	**Khi hết dữ liệu (EOF): input= đã đọc hết, hoặc Ctrl+D (Linux) / Ctrl+Z rồi Enter (Windows)**	Dòng rỗng giữa chừng vẫn là một dòng "\n", không làm dừng vòng lặp.
B	Ở dòng rỗng đầu tiên	Dòng rỗng vẫn là một dòng.
C	Sau đúng một dòng	Vòng for đọc tới EOF.

Giải thích (nguyên văn JSON): Dòng rỗng giữa chừng vẫn là một dòng "\n", không làm dừng vòng lặp.

CÂU PHẢN VÍ DỤ — đáp án

run_sum(["4", "", "y", "6"]) trả gì?

Nhãn	Lựa chọn
A ĐÚNG	`(10, ["skip: y"])`
B	`(10, [])`
C	`(4, ["skip: y"])`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao "skip: abc" phải ra stderr chứ không ra stdout?

Mẫu đạt: stdout là KẾT QUẢ mà bên khác đọc tiếp (qua pipe, > file, hay r.stdout trong subprocess). Nếu cảnh báo lẫn vào stdout, bên đọc gặp "skip: abc" thay vì con số và int() báo lỗi. Đặt cảnh báo ở stderr giữ stdout sạch mà người dùng vẫn thấy cảnh báo. Trường hợp đặc biệt: không có dòng hỏng thì stderr rỗng.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	input=data → các dòng đi vào stdin (fd 0) của con
Bước 2	con print(total) → con số đi vào pipe stdout
Bước 3	con print("skip: x", file=sys.stderr) → đi vào pipe stderr riêng
Bước 4	text=True → r.stdout và r.stderr là str, không phải bytes

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp chỗ in cảnh báo và hai tham số của subprocess.run.

Yêu cầu thêm: Thử list rỗng, toàn dòng hỏng, số âm và dòng có khoảng trắng hai đầu; kiểm stdout chỉ chứa con số.

```
def read_scores():
    scores = []
    for line in sys.stdin:
        text = line.strip()
        if text == "":
            continue
        try:
            scores.append(int(text))
        except ValueError:
            {{slot1}}
    return scores


def run_sum(lines):
    data = "".join(line + "\n" for line in lines)
    result = subprocess.run([sys.executable, "sum_numbers.py"],
                            {{slot2}}, capture_output=True, {{slot3}})
    return int(result.stdout), result.stderr.splitlines()
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`print("skip:", text, file=sys.stderr)`
[Slot 2]	`input=data`
[Slot 3]	`text=True`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 2	`stdin=data`	stdin= nhận PIPE hoặc file object; chuỗi phải đi qua input=.
ô 1	`print("skip:", text)`	Ra stdout — lẫn vào kết quả.
ô 3	`stderr=subprocess.STDOUT`	Không dùng chung với capture_output — và gộp thì int() gặp chữ.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 2, ô 1, ô 3.

ĐÁP ÁN MINI-GAME (4 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [1, 2, 0, 3].

Bên trái	NỐI VỚI (đáp án)
input=data	**các dòng đi vào stdin (fd 0) của con**
con print(total)	**con số đi vào pipe stdout**
con print("skip: x", file=sys.stderr)	**đi vào pipe stderr riêng**
text=True	**r.stdout và r.stderr là str, không phải bytes**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`print("skip:", text, file=sys.stderr)`
ô 2	`input=data`
ô 3	`text=True`

Bẫy ở ô 2 — `stdin=data`: stdin= nhận PIPE hoặc file object; chuỗi phải đi qua input=.
Bẫy ở ô 1 — `print("skip:", text)`: Ra stdout — lẫn vào kết quả.
Bẫy ở ô 3 — `stderr=subprocess.STDOUT`: Không dùng chung với capture_output — và gộp thì int() gặp chữ.

parsons-2 · SẮP DÒNG (Parsons) — Ghép các dòng thành chương trình chạy đúng

Thứ tự đúng (theo mảng `lines`): `L9 → L10 → L11 → L12 → L13 → L14 → L17 → L18`. Thứ tự xáo lúc chơi là `order` = ['D1', 'L9', 'L17', 'L13', 'L11', 'L14', 'L12', 'L18', 'L10', 'D0'].

```
L9              continue
L10         try:
L11             scores.append(int(text))
L12         except ValueError:
L13             print("skip:", text, file=sys.stderr)
L14     return scores
L17 def run_sum(lines):
L18     data = "".join(line + "\n" for line in lines)
```

Phần đầu/cuối đã cho sẵn, học viên không sắp: 8 dòng đầu + 6 dòng cuối.

DÒNG BẪY (không được dùng)	Vì sao sai
`print("skip:", text)`	cảnh báo in ra stdout
`input=data, » capture_output=True)`	quên text=True

spotError-3 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 14 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 7).
Lỗi: cảnh báo in ra stdout
Sửa thành:

```
            print("skip:", text, file=sys.stderr)
```

```
     7 |     for line in sys.stdin:
     8 |         text = line.strip()
     9 |         if text == "":
    10 |             continue
    11 |         try:
    12 |             scores.append(int(text))
    13 |         except ValueError:
>>  14 |             print("skip:", text)
    15 |     return scores
    16 |
    17 |
    18 | def run_sum(lines):
    19 |     data = "".join(line + "\n" for line in lines)
    20 |     result = subprocess.run([sys.executable, "sum_numbers.py"],
    21 |                             input=data, capture_output=True, text=True)
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — cảnh báo in ra stdout

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
14	`print("skip:", text, file=sys.stderr)`	`print("skip:", text)`

KIỂU SAI 2 — quên text=True

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
21	`input=data, » capture_output=True, » text=True)`	`input=data, » capture_output=True)`

KIỂU SAI 3 — gộp stderr vào stdout

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai

21	`input=data, »`
	`capture_output=True, »`
	`text=True)`
		`input=data, »`
		`stdout=subprocess.PIPE, »`
		`stderr=subprocess.STDOUT, »`
		`text=True)`

Nguồn trang này: pe-v4 · 004-P04.json · 004-P04.json · qa.json[P04]

P05 · subprocess.run và kết quả

Mã bài P05 · Trung cấp · M1 · Tiến trình và subprocess · 40 phút · 15 XP · thứ tự 5 · mức I

MỤC TIÊU BÀI

Chạy process con bằng subprocess.run và đọc đúng kết quả.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết ba hàm, mỗi hàm chạy code Python trong một process con bằng [sys.executable, "-c", code]. exit_code(code) trả exit code (int) của con. output_of(code) trả stdout của con dạng str bằng subprocess.check_output (con thoát mã khác 0 thì để CalledProcessError bay ra). grade(code, expected, limit=2) là máy chấm mini: chạy con với capture_output=True, text=True, timeout=limit — quá giờ trả "TIMEOUT"; exit code khác 0 trả "ERROR <mã>"; stdout bỏ khoảng trắng hai đầu bằng expected trả "PASS"; còn lại "FAIL".

Thực hành: Một lần run() là đủ cho grade: capture_output=True, text=True, timeout=limit. Bắt TimeoutExpired, rồi kiểm returncode, cuối cùng mới so stdout.

Gọi (đề cho)	Kết quả mong đợi
`grade("print(2+3)", "5")`	`"PASS"`
`grade("raise SystemExit(2)", "")`	`"ERROR 2"`
`grade("import time; time.sleep(5)", "", 1)`	`"TIMEOUT"`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`exit_code(code)`	Exit code của process con chạy code. · code: str · int · Con in thẳng ra màn hình (không thu).
`output_of(code)`	stdout của process con. · code: str · str; con thoát mã khác 0 → CalledProcessError · Dùng subprocess.check_output(..., text=True).
`grade(code, expected, limit=2)`	Máy chấm mini: chạy con, so đầu ra. · code, expected: str; limit: giây · "TIMEOUT" | "ERROR <mã>" | "PASS" | "FAIL" · Một lần subprocess.run(capture_output=True, text=True, timeout=limit).

LỜI GIẢI MẪU (qa.json · P05)

```
import subprocess
import sys


def exit_code(code):
    return subprocess.run([sys.executable, "-c", code]).returncode


def output_of(code):
    return subprocess.check_output([sys.executable, "-c", code], text=True)


def grade(code, expected, limit=2):
    try:
        result = subprocess.run([sys.executable, "-c", code],
                                capture_output=True, text=True, timeout=limit)
    except subprocess.TimeoutExpired:
        return "TIMEOUT"
    if result.returncode != 0:
        return f"ERROR {result.returncode}"
    if result.stdout.strip() == expected:
        return "PASS"
    return "FAIL"


if __name__ == "__main__":
    print(grade("print(2+3)", "5"))
```

BỘ CHẤM — 12 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	exit_code("pass") → 0	`exit_code("pass")` `→ 0`
2	exit_code: SystemExit(4) → 4, 1/0 → 1	`(exit_code("raise SystemExit(4)"), exit_code("1/0"))` `→ (4, 1)`
3	output_of trả stdout (str)	`output_of("print(6 * 7)")` `→ '42\n'`
4	output_of: nhiều dòng	`output_of("for i in range(3): print(i)")` `→ '0\n1\n2\n'`
5	output_of: con thoát mã 5 → CalledProcessError	`_studio.call(output_of, "raise SystemExit(5)").error` `→ 'CalledProcessError'`

6	grade: PASS và FAIL	`(grade("print(2+3)", "5"), grade("print(6)", "5"))` `→ ('PASS', 'FAIL')`
7	grade: in đúng rồi thoát mã 3 → ERROR 3	`grade("print(5); raise SystemExit(3)", "5")` `→ 'ERROR 3'`
8	grade: ngoại lệ trong con → ERROR 1	`grade("1/0", "x")` `→ 'ERROR 1'`
9	grade: quá giờ → TIMEOUT	`grade("import time; time.sleep(5)", "", 1)` `→ 'TIMEOUT'`
10	grade: ngủ ít hơn limit mặc định vẫn chấm	`grade("import time; time.sleep(1); print('ok')", "ok")` `→ 'PASS'`
11	grade: nhiều dòng, chỉ bỏ khoảng trắng hai đầu	`grade("print(1); print(2)", "1\n2")` `→ 'PASS'`

12	Lệnh con là list [sys.executable, "-c", code], không phải chuỗi	`(lambda a: (isinstance(a, (list, tuple)), »`
		`list(a)))(_studio.call(exit_code, "raise »`
		`SystemExit(4)").subprocess_calls[0]["args"])`
		`→ (True, [__import__("sys").executable, "-c", "raise SystemExit(4)"])`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Một lần run() là đủ cho grade: capture_output=True, text=True, timeout=limit. Bắt TimeoutExpired, rồi kiểm returncode, cuối cùng mới so stdout.

Cấu trúc. 1. exit_code: return subprocess.run([sys.executable, "-c", code]).returncode. 2. output_of: return subprocess.check_output([sys.executable, "-c", code], text=True). 3. grade: try run(..., capture_output=True, text=True, timeout=limit); except subprocess.TimeoutExpired → "TIMEOUT". 4. returncode != 0 → f"ERROR {result.returncode}"; stdout.strip() == expected → "PASS"; còn lại "FAIL".

Lời giải từng phần. def exit_code(code): return subprocess.run([sys.executable, "-c", code]).returncode ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — subprocess, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy subprocess, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng).
- Lệnh shell khác (ls, dir, ping...) không chạy được ở đây.
- Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

NGỘ NHẬN THƯỜNG GẶP: "check_output trả exit code"

check_output trả STDOUT của con. Viết if subprocess.check_output(...) == 0 là so chuỗi/bytes với 0 — luôn False; còn khi con thoát mã khác 0, check_output ném CalledProcessError chứ không trả mã. Lấy exit code bằng run(...).returncode, hoặc error.returncode trong except. Ngộ nhận anh em: tưởng exit code khác 0 tự thành lỗi — run() không ném gì nếu thiếu check=True.

CÂU DỰ ĐOÁN — đáp án và giải thích

subprocess.check_output([sys.executable, "-c", "print(1)"], text=True) trả về gì?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	0 — exit code	Đó là ngộ nhận phổ biến nhất về check_output.
B ĐÚNG	**"1\n" — stdout của con**	check_output trả đầu ra; exit code nằm ở run().returncode hoặc trong exception.
C	Một CompletedProcess	Đó là kiểu trả về của run().

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. subprocess.run([sys.executable, "-c", "raise SystemExit(3)"]) (không có check) thì...

A	Ném SystemExit trong process cha	SystemExit xảy ra trong process CON; cha chỉ thấy exit code.
B ĐÚNG	**Trả CompletedProcess với returncode = 3, không ném lỗi**	Chỉ check=True mới biến mã khác 0 thành CalledProcessError.
C	Ném CalledProcessError	Cần check=True.

Giải thích (nguyên văn JSON): Chỉ check=True mới biến mã khác 0 thành CalledProcessError.

2. Vì sao viết [sys.executable, "-c", code] thay vì chuỗi "python -c " + code?

A	Hai cách như nhau	Code có dấu cách hay dấu ngoặc kép sẽ bị tách sai.
B ĐÚNG	**List giao từng đối số nguyên vẹn, không cần shell tách chuỗi; sys.executable chắc chắn là Python đang chạy**	Chuỗi không có shell=True bị coi là TÊN chương trình trên Linux; "python" có thể là bản khác, hoặc stub của Microsoft Store trên Windows.
C	Chuỗi chạy chậm hơn	Không phải chuyện tốc độ.

Giải thích (nguyên văn JSON): Chuỗi không có shell=True bị coi là TÊN chương trình trên Linux; "python" có thể là bản khác, hoặc stub của Microsoft Store trên Windows.

3. Con chạy 1/0 (không bắt lỗi). returncode là?

A	0	Có lỗi thì không phải 0.
B	ZeroDivisionError	Cha không nhận object exception của con, chỉ nhận mã thoát.
C ĐÚNG	**1 — ngoại lệ không bắt làm Python thoát mã 1, traceback ra stderr**	Traceback nằm trong result.stderr nếu có capture.

Giải thích (nguyên văn JSON): Traceback nằm trong result.stderr nếu có capture.

CÂU PHẢN VÍ DỤ — đáp án

grade("print(5); raise SystemExit(3)", "5") trả gì?

Nhãn	Lựa chọn
A	`PASS`
B	`FAIL`
C ĐÚNG	`ERROR 3`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: grade kiểm returncode TRƯỚC khi so stdout. Vì sao?

Mẫu đạt: Con có thể in đúng "5" rồi mới gặp lỗi và thoát mã khác 0 — chỉ so stdout sẽ chấm PASS một chương trình hỏng. Exit code là tín hiệu thành công/thất bại chính thức của process; chỉ khi nó bằng 0 mới so đầu ra. Trường hợp đặc biệt: chạy quá limit thì không có exit code bình thường, trả TIMEOUT.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	run([sys.executable, "-c", code]) → cha chờ con kết thúc
Bước 2	quá limit giây → TimeoutExpired → "TIMEOUT"
Bước 3	returncode != 0 → "ERROR <mã>"
Bước 4	stdout.strip() == expected → "PASS", khác thì "FAIL"

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp máy chấm mini.

Yêu cầu thêm: Thử code in nhiều dòng, code in đúng rồi mới thoát mã 3, và code ngủ time.sleep(5) với limit = 1.

```
def grade(code, expected, limit=2):
    try:
        result = subprocess.run([sys.executable, "-c", code],
                                capture_output=True, text=True, {{slot1}})
    except {{slot2}}:
        return "TIMEOUT"
    if {{slot3}}:
        return f"ERROR {result.returncode}"
    if result.stdout.strip() == expected:
        return "PASS"
    return "FAIL"
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`timeout=limit`
[Slot 2]	`subprocess.TimeoutExpired`
[Slot 3]	`result.returncode != 0`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 3	`result.stderr`	Có chữ trong stderr chưa chắc là lỗi; tín hiệu chính thức là returncode.
ô 1	`check=True`	Mã khác 0 thành CalledProcessError — không ai bắt, và mất timeout.
ô 2	`subprocess.CalledProcessError`	Không có check=True thì run() không ném lỗi này; quá giờ là TimeoutExpired.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 3, ô 2, ô 1.

ĐÁP ÁN MINI-GAME (4 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [3, 2, 1, 0].

Bên trái	NỐI VỚI (đáp án)
run([sys.executable, "-c", code])	**cha chờ con kết thúc**

quá limit giây	**TimeoutExpired → "TIMEOUT"**
returncode != 0	**"ERROR <mã>"**
stdout.strip() == expected	**"PASS", khác thì "FAIL"**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`timeout=limit`
ô 2	`subprocess.TimeoutExpired`
ô 3	`result.returncode != 0`

Bẫy ở ô 3 — `result.stderr`: Có chữ trong stderr chưa chắc là lỗi; tín hiệu chính thức là returncode.
Bẫy ở ô 1 — `check=True`: Mã khác 0 thành CalledProcessError — không ai bắt, và mất timeout.
Bẫy ở ô 2 — `subprocess.CalledProcessError`: Không có check=True thì run() không ném lỗi này; quá giờ là TimeoutExpired.

parsons-2 · SẮP DÒNG (Parsons) — Ghép các dòng thành chương trình chạy đúng

Thứ tự đúng (theo mảng `lines`): `L4 → L5 → L8 → L9 → L12 → L13 → L14 → L15`. Thứ tự xáo lúc chơi là `order` = ['L15', 'L9', 'L14', 'D0', 'L12', 'L8', 'L4', 'L13', 'D1', 'L5'].

```
L4   def exit_code(code):
L5       return subprocess.run([sys.executable, "-c", code]).returncode
L8   def output_of(code):
L9       return subprocess.check_output([sys.executable, "-c", code], text=True)
L12  def grade(code, expected, limit=2):
L13      try:
L14          result = subprocess.run([sys.executable, "-c", code],
L15                                  capture_output=True, text=True, timeout=limit)
```

Phần đầu/cuối đã cho sẵn, học viên không sắp: 3 dòng đầu + 10 dòng cuối.

DÒNG BẪY (không được dùng)	Vì sao sai
`return subprocess.check_output([sys.executable, "-c", code])`	check_output tưởng trả exit code
`return subprocess.run(sys.executable + " -c " + code).returncode`	ghép lệnh thành một chuỗi thay vì list

spotError-3 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 6 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 1).
Lỗi: check_output tưởng trả exit code
Sửa thành:

```
return subprocess.run([sys.executable, "-c", code]).returncode
```

```
 1 | import subprocess
 2 | import sys
 3 |
 4 |
 5 | def exit_code(code):
>>  6 |     return subprocess.check_output([sys.executable, "-c", code])
 7 |
 8 |
 9 | def output_of(code):
10 |     return subprocess.check_output([sys.executable, "-c", code], text=True)
11 |
12 |
13 | def grade(code, expected, limit=2):
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 4 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — check_output tưởng trả exit code

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
6	`return subprocess.run([sys.executable, "-c", code]).returncode`	`return subprocess.check_output([sys.executable, "-c", code])`

KIỂU SAI 2 — ghép lệnh thành một chuỗi thay vì list

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
6	`return subprocess.run([sys.executable, "-c", code]).returncode`	`return subprocess.run(sys.executable + " -c " + code).returncode`

KIỂU SAI 3 — quên text=True

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
16	`capture_output=True, text=True, timeout=limit)`	`capture_output=True, timeout=limit)`

KIỂU SAI 4 — đoán lỗi qua stdout thay vì returncode

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
19	`    if result.returncode != 0:`	`    if result.stdout == "":`

Nguồn trang này: pe-v4 · 005-P05.json · 005-P05.json · qa.json[P05]

P06 · Popen, communicate và timeout

Mã bài P06 · Trung cấp · M1 · Tiến trình và subprocess · 40 phút · 15 XP · thứ tự 6 · mức I

MỤC TIÊU BÀI

Điều khiển process con bằng Popen: gửi input, chờ có giới hạn, dọn dẹp khi quá giờ.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết talk(code, data, limit=2): mở process con bằng subprocess.Popen([sys.executable, "-c", code]) với stdin, stdout, stderr đều là subprocess.PIPE và text=True; gửi data vào stdin và chờ bằng communicate(data, timeout=limit). Xong kịp thì trả ("ok", stdout, returncode). Quá giờ (TimeoutExpired) thì KHÔNG bỏ mặc con: kill() rồi communicate() thêm một lần để thu nốt phần đã in và thu hồi con; trả ("timeout", phần stdout thu được, returncode).

Thực hành: Popen với ba PIPE + text=True; communicate(data, timeout=limit) trong try; except TimeoutExpired: kill(), communicate() lần hai, trả "timeout".

Gọi (đề cho)	Kết quả mong đợi
`talk("print(input().upper())", "hello\n")`	`("ok", "HELLO\n", 0)`
`Con in "start" rồi ngủ 10 giây, limit = 1`	`("timeout", "start\n", mã khác 0)`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`talk(code, data, limit=2)`	Gửi data cho process con, chờ có giới hạn, dọn dẹp khi quá giờ. · code, data: str; limit: giây · ("ok" | "timeout", stdout: str, returncode: int) · Quá giờ: kill() rồi communicate() lần hai — không để con chạy tiếp.

LỜI GIẢI MẪU (qa.json · P06)

```
import subprocess
import sys


def talk(code, data, limit=2):
    proc = subprocess.Popen([sys.executable, "-c", code],
                            stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                            stderr=subprocess.PIPE, text=True)
    try:
        out, err = proc.communicate(data, timeout=limit)
    except subprocess.TimeoutExpired:
        proc.kill()
        out, err = proc.communicate()
        return "timeout", out, proc.returncode
    return "ok", out, proc.returncode


if __name__ == "__main__":
    print(talk("print(input().upper())", "hello\n"))
```

BỘ CHẤM — 7 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Gửi stdin, nhận stdout	`talk("print(input().upper())", "hello\n")` `→ ('ok', 'HELLO\n', 0)`
2	Con đọc nhiều dòng stdin	`talk("import sys; print(sum(int(x) for x in sys.stdin))", "1\n2\n39\n")` `→ ('ok', '42\n', 0)`
3	Trả đúng exit code của con	`talk("import sys; print('bye'); sys.exit(3)", "")` `→ ('ok', 'bye\n', 3)`
4	Quá giờ → "timeout"	`talk('import time; time.sleep(10)', "", 1)[0]` `→ 'timeout'`
5	Thu nốt phần con đã in trước khi bị kill	`talk("import time; print('start', flush=True); time.sleep(10)", "", 1)[:2]` `→ ('timeout', 'start\n')`

6	Sau kill, con đã kết thúc (returncode khác None và khác 0)	`(lambda rc: rc is not None and rc != 0)(talk('import time; »`
		`  time.sleep(10)', "", 1)[2])`
		`→ True`

7	Ngủ ngắn hơn limit thì vẫn "ok"	`talk("import time; time.sleep(1); print('done')", "", 3)` `→ ('ok', 'done\n', 0)`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Popen với ba PIPE + text=True; communicate(data, timeout=limit) trong try; except TimeoutExpired: kill(), communicate() lần hai, trả "timeout".

Cấu trúc. 1. proc = subprocess.Popen([sys.executable, "-c", code], stdin=PIPE, stdout=PIPE, stderr=PIPE, text=True). 2. try: out, err = proc.communicate(data, timeout=limit). 3. except subprocess.TimeoutExpired: proc.kill(); out, err = proc.communicate(); return "timeout", out, proc.returncode. 4. Xong kịp: return "ok", out, proc.returncode.

Lời giải từng phần. try: out, err = proc.communicate(data, timeout=limit) except subprocess.TimeoutExpired: ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — subprocess, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy subprocess, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng).
- Lệnh shell khác (ls, dir, ping...) không chạy được ở đây.
- Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

NGỘ NHẬN THƯỜNG GẶP: "Bắt TimeoutExpired là xong việc"

Với Popen.communicate(timeout=...), TimeoutExpired không tự kill process: con vẫn chạy, vẫn giữ pipe và CPU; bỏ mặc nó là rò process. Phải kill() (hoặc terminate()) rồi gọi communicate() thêm một lần để thu phần output còn lại và thu hồi con — lúc đó returncode mới có giá trị.

CÂU DỰ ĐOÁN — đáp án và giải thích

p.communicate(timeout=1) ném TimeoutExpired. Lúc đó process con thế nào?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Đã bị kill tự động	Đó là hành vi của run(timeout=...), không phải Popen.communicate.
B	Đã kết thúc bình thường	Nếu đã kết thúc thì communicate trả kết quả chứ không ném.
C ĐÚNG	**Vẫn đang chạy — phải tự kill() rồi communicate() lại**	TimeoutExpired chỉ báo cha đã chờ hết giờ; nó không đụng vào con.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Popen() và run() khác nhau chính ở đâu?

A	Popen không chạy được Python	Cả hai nhận cùng kiểu args.
B	run chạy song song, Popen chạy tuần tự	Ngược lại: Popen không chờ.
C ĐÚNG	**run chờ xong rồi trả kết quả; Popen trả object ngay để bạn tự điều khiển (gửi input, poll, kill...)**	run được xây trên Popen.

Giải thích (nguyên văn JSON): run được xây trên Popen.

2. Ngay sau proc = Popen(...) (con đang chờ input), proc.poll() trả?

A	Chặn tới khi con xong	Đó là wait().
B ĐÚNG	**None — con còn chạy**	poll không chặn; None nghĩa là chưa kết thúc.
C	0	0 chỉ có khi con đã kết thúc thành công.

Giải thích (nguyên văn JSON): poll không chặn; None nghĩa là chưa kết thúc.

3. Muốn communicate trả stdout dạng str (không phải bytes) cần gì?

A	Không cần gì	Mặc định stdout không bị thu và pipe là bytes.
B ĐÚNG	**stdout=subprocess.PIPE và text=True khi tạo Popen**	Thiếu PIPE thì phần stdout là None; thiếu text=True thì là bytes.
C	Gọi communicate(text=True)	communicate không có tham số text.

Giải thích (nguyên văn JSON): Thiếu PIPE thì phần stdout là None; thiếu text=True thì là bytes.

CÂU PHẢN VÍ DỤ — đáp án

talk("import sys; print(len(sys.stdin.read().split()))", "a b c\n") trả gì?

Nhãn	Lựa chọn
A	`("ok", "1\n", 0)`
B ĐÚNG	`("ok", "3\n", 0)`
C	`("timeout", "", 1)`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Sau TimeoutExpired, vì sao phải gọi communicate() THÊM một lần sau kill()?

Mẫu đạt: kill() chỉ yêu cầu hệ điều hành dừng con. communicate() lần hai đọc nốt dữ liệu còn trong pipe (vd. "start" con đã in trước khi bị giết) và chờ con thật sự kết thúc để thu hồi, nhờ đó returncode có giá trị và không còn process mồ côi. Trường hợp đặc biệt: con xong kịp thì không vào nhánh này.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	Popen(...) → con chạy, cha nhận proc ngay
Bước 2	communicate(data, timeout=limit) → gửi data, đọc tới khi con xong
Bước 3	quá limit → TimeoutExpired, con VẪN chạy
Bước 4	kill() rồi communicate() → thu nốt output, returncode có giá trị

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp mẫu chuẩn "timeout → kill → communicate".

Yêu cầu thêm: Thử con đọc nhiều dòng từ stdin, con in một dòng rồi ngủ lâu, và con ngủ ngắn hơn limit.

```
def talk(code, data, limit=2):
    proc = subprocess.Popen([sys.executable, "-c", code],
                            stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                            stderr=subprocess.PIPE, text=True)
    try:
        out, err = {{slot1}}
    except subprocess.TimeoutExpired:
        {{slot2}}
        out, err = {{slot3}}
        return "timeout", out, proc.returncode
    return "ok", out, proc.returncode
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`proc.communicate(data, timeout=limit)`
[Slot 2]	`proc.kill()`
[Slot 3]	`proc.communicate()`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 3	`proc.communicate(data)`	Không được gửi input lần hai sau khi đã bắt đầu giao tiếp (ValueError).
ô 1	`proc.wait(timeout=limit)`	wait() không gửi data, không đọc pipe; nó trả exit code chứ không phải (out, err).
ô 2	`pass`	Con vẫn chạy — TimeoutExpired không tự kill.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 2, ô 3, ô 1.

ĐÁP ÁN MINI-GAME (4 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [1, 2, 3, 0].

Bên trái	NỐI VỚI (đáp án)
Popen(...)	**con chạy, cha nhận proc ngay**
communicate(data, timeout=limit)	**gửi data, đọc tới khi con xong**
quá limit	**TimeoutExpired, con VẪN chạy**
kill() rồi communicate()	**thu nốt output, returncode có giá trị**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`proc.communicate(data, timeout=limit)`
ô 2	`proc.kill()`
ô 3	`proc.communicate()`

Bẫy ở ô 3 — `proc.communicate(data)`: Không được gửi input lần hai sau khi đã bắt đầu giao tiếp (ValueError).
Bẫy ở ô 1 — `proc.wait(timeout=limit)`: wait() không gửi data, không đọc pipe; nó trả exit code chứ không phải (out, err).
Bẫy ở ô 2 — `pass`: Con vẫn chạy — TimeoutExpired không tự kill.

parsons-2 · SẮP DÒNG (Parsons) — Ghép các dòng thành chương trình chạy đúng

Thứ tự đúng (theo mảng `lines`): `L4 → L5 → L6 → L7 → L8 → L9 → L10 → L11`. Thứ tự xáo lúc chơi là `order` = ['L8', 'L7', 'L11', 'D1', 'L4', 'D0', 'L10', 'L6', 'L5', 'L9'].

```
L4   def talk(code, data, limit=2):
L5       proc = subprocess.Popen([sys.executable, "-c", code],
L6                               stdin=subprocess.PIPE, stdout=subprocess.PIPE,
L7                               stderr=subprocess.PIPE, text=True)
L8       try:
L9           out, err = proc.communicate(data, timeout=limit)
L10      except subprocess.TimeoutExpired:
L11          proc.kill()
```

Phần đầu/cuối đã cho sẵn, học viên không sắp: 3 dòng đầu + 6 dòng cuối.

DÒNG BẪY (không được dùng)	Vì sao sai

stderr=subprocess.PIPE)	quên text=True
return "timeout", "", proc.returncode	tưởng TimeoutExpired đã kill con

spotError-3 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 8 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 1).

Lỗi: quên text=True

Sửa thành:

```
                          stderr=subprocess.PIPE, text=True)
```

```
    1 | import subprocess
    2 | import sys
    3 |
    4 |
    5 | def talk(code, data, limit=2):
    6 |     proc = subprocess.Popen([sys.executable, "-c", code],
    7 |                             stdin=subprocess.PIPE, stdout=subprocess.PIPE,
>>  8 |                             stderr=subprocess.PIPE)
    9 |     try:
   10 |         out, err = proc.communicate(data, timeout=limit)
   11 |     except subprocess.TimeoutExpired:
   12 |         proc.kill()
   13 |         out, err = proc.communicate()
   14 |         return "timeout", out, proc.returncode
   15 |     return "ok", out, proc.returncode
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — quên text=True

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
8	stderr=subprocess.PIPE, » text=True)	stderr=subprocess.PIPE)

KIỂU SAI 2 — tưởng TimeoutExpired đã kill con

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
12	proc.kill()	return "timeout", "", proc.returncode
13	out, err = proc.communicate()	(xoá dòng)
14	return "timeout", out, proc.returncode	(xoá dòng)

KIỂU SAI 3 — không bắt TimeoutExpired

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
9	try:	out, err = proc.communicate(data, timeout=limit)
10	out, err = proc.communicate(data, timeout=limit)	(xoá dòng)
11	except subprocess.TimeoutExpired:	(xoá dòng)
12	proc.kill()	(xoá dòng)
13	out, err = proc.communicate()	(xoá dòng)
14	return "timeout", out, proc.returncode	(xoá dòng)

Nguồn trang này: pe-v4 · 006-P06.json · 006-P06.json · qa.json[P06]

P07 · multiprocessing: Process và Queue

Mã bài P07 · Trung cấp · M1 · Tiến trình và subprocess · 35 phút · 15 XP · thứ tự 7 · mức I

MỤC TIÊU BÀI

Chạy hàm ở process riêng và gửi kết quả về qua Queue.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết square(n, results): tính n * n và put cặp (n, n * n) vào Queue results. Viết run_workers(numbers): tạo một Queue, tạo một Process cho MỖI số (target=square, args=(n, results)), start hết, lấy đủ len(numbers) kết quả bằng results.get(), join hết, trả dict {n: n * n}. Lời gọi tạo process ở cấp ngoài cùng của file phải nằm trong if __name__ == "__main__":.

Thực hành: Worker cấp module chỉ tính rồi put vào Queue; cha tạo Process cho từng số, start hết, get đủ, join hết. Lời gọi tạo process ở cấp ngoài cùng nằm dưới main guard.

Gọi (đề cho)	Kết quả mong đợi
`run_workers([2, 3, 4])`	`{2: 4, 3: 9, 4: 16}`
`run_workers([])`	`{}`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`square(n, results)`	Worker chạy trong process con. · n: int; results: Queue · None (kết quả đi qua Queue) · results.put((n, n * n)).
`run_workers(numbers)`	Mỗi số một process, gom kết quả qua Queue. · numbers: list[int] · dict {n: n * n} · Tạo len(numbers) Process, start hết, get đủ, join hết.

LỜI GIẢI MẪU (qa.json · P07)

```
from multiprocessing import Process, Queue


def square(n, results):
    results.put((n, n * n))


def run_workers(numbers):
    results = Queue()
    workers = [Process(target=square, args=(n, results)) for n in numbers]
    for worker in workers:
        worker.start()
    squares = {}
    for _ in numbers:
        n, value = results.get()
        squares[n] = value
    for worker in workers:
        worker.join()
    return squares


if __name__ == "__main__":
    print(run_workers([2, 3, 4]))
```

BỘ CHẤM — 6 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	run_workers([2, 3, 4])	`run_workers([2, 3, 4])` `→ {2: 4, 3: 9, 4: 16}`
2	Dữ liệu khác, có số âm	`run_workers([7, -3, 10])` `→ {7: 49, -3: 9, 10: 100}`
3	List rỗng → {}	`run_workers([])` `→ {}`

4	square gửi đúng một cặp qua Queue	`_studio.check('from multiprocessing import Queue\nq = Queue()\nsquare(6, »`
		`  q)\nassert q.get() == (6, 36), "square phải put cặp (n, n * n)"')`
		`→ True`

5	Mỗi số một Process; target là hàm; start hết rồi join hết	`(lambda r: (r.process_count, r.process_targets_callable, »`
		`  r.process_starts_before_joins, »`
		`  r.all_processes_joined))(_studio.call(run_workers, [1, 2, 3]))`
		`→ (3, True, True, True)`

6	Import file không tự tạo process (main guard)	`(lambda r: (r.stdout, r.process_count))(_studio.run(entry='None'))` `→ ('', 0)`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Worker cấp module chỉ tính rồi put vào Queue; cha tạo Process cho từng số, start hết, get đủ, join hết. Lời gọi tạo process ở cấp ngoài cùng nằm dưới main guard.

Cấu trúc. 1. def square(n, results): results.put((n, n * n)). 2. results = Queue(); workers = [Process(target=square, args=(n, results)) for n in numbers]. 3. start từng worker; lặp len(numbers) lần results.get() và ghi vào dict. 4. join từng worker rồi trả dict; print(run_workers(...)) ở cuối file đặt trong if __name__ == "__main__":.

Lời giải từng phần. def square(n, results): results.put((n, n * n)) ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — multiprocessing, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy multiprocessing, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG multiprocessing: target của mỗi Process chạy ngay khi start() trên một BẢN SAO dữ liệu (đổi biến toàn cục trong con không ảnh hưởng cha, đúng như process thật);
- Queue/Value/Manager là kênh dùng chung.
- Thời gian đo bằng đồng hồ ảo: các process chạy song song nên tổng thời gian ≈ process lâu nhất.
- Trên Windows nhớ đặt code tạo process trong if __name__ == "__main__": — chạy thật trên máy để thấy khác biệt.

NGỘ NHẬN THƯỜNG GẶP: "Process cũng chia sẻ biến như thread"

Các thread trong cùng một process dùng chung bộ nhớ; các process thì không. Worker ghi vào dict toàn cục hay append vào list của cha chỉ sửa BẢN SAO trong con — cha nhận dict rỗng. Kết quả phải đi qua IPC: Queue, Pipe, Value/Manager. Và subprocess với multiprocessing không phải hai tên của cùng một API.

CÂU DỰ ĐOÁN — đáp án và giải thích

Worker chạy counter += 1 trong process con (counter là biến toàn cục bằng 0). Sau join, cha in counter được mấy?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A ĐÚNG	**0 — con sửa bản sao của nó, cha không thấy**	Process có không gian địa chỉ riêng; gửi về phải qua Queue/Pipe.
B	1	Đó là hành vi của thread (chung bộ nhớ), không phải process.
C	Báo lỗi	Không lỗi — chỉ là hai bản biến khác nhau.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. subprocess và multiprocessing có phải hai tên cho cùng một API không?

A ĐÚNG	**Không — subprocess chạy chương trình khác; multiprocessing chạy hàm Python ở process riêng**	Cả hai liên quan process nhưng mục đích và API khác nhau.
B	Có	Hai module, hai bộ API.
C	multiprocessing là bản cũ của subprocess	Không có quan hệ cũ/mới.

Giải thích (nguyên văn JSON): Cả hai liên quan process nhưng mục đích và API khác nhau.

2. Vì sao không dùng target=lambda n: n * n cho Process?

A ĐÚNG	**Process con phải tìm/pickle lại target theo tên ở cấp module; lambda không có tên như vậy**	Định nghĩa worker bằng def ở cấp module.
B	Process không nhận hàm	Process nhận hàm qua target.
C	lambda chạy chậm hơn	Không phải chuyện tốc độ.

Giải thích (nguyên văn JSON): Định nghĩa worker bằng def ở cấp module.

3. Bỏ if __name__ == "__main__": trên Windows thì...

A	Chỉ chạy chậm hơn	Không phải chuyện tốc độ.
B ĐÚNG	**Process con import lại file và chạy lại đoạn tạo process — Python báo RuntimeError (bootstrapping)**	Main guard chặn code tạo process chạy lại khi file bị import.
C	Chạy bình thường	Trên Windows (spawn) thì không.

Giải thích (nguyên văn JSON): Main guard chặn code tạo process chạy lại khi file bị import.

CÂU PHẢN VÍ DỤ — đáp án

run_workers([-3, 5]) trả gì?

Nhãn	Lựa chọn
A	`{-3: -9, 5: 25}`
B ĐÚNG	`{-3: 9, 5: 25}`
C	`{}`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao worker phải put kết quả vào Queue thay vì ghi vào dict toàn cục?

Mẫu đạt: Mỗi process con có bộ nhớ riêng (bản sao), nên ghi vào dict toàn cục chỉ sửa bản của con; cha vẫn thấy dict rỗng. Queue là kênh IPC: con put cặp (n, n * n), dữ liệu được pickle chuyển sang cha, cha get đủ len(numbers) lần. Trường hợp đặc biệt: numbers rỗng thì không tạo process nào và trả {}.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	Queue() → kênh IPC dùng chung giữa cha và con
Bước 2	Process(target=square, args=(n, results)) → mô tả process, chưa chạy
Bước 3	start() → square chạy ở process con, put (n, n * n)
Bước 4	cha get() đủ len(numbers) lần → join hết → dict

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp ba chỗ: gửi kết quả, tạo Process, nhận kết quả.

Yêu cầu thêm: Thử một số, list rỗng và số âm; thử cho worker ghi vào biến toàn cục để thấy cha không nhận được gì.

```
from multiprocessing import Process, Queue


def square(n, results):
    {{slot1}}


def run_workers(numbers):
    results = Queue()
    workers = [{{slot2}} for n in numbers]
    for worker in workers:
        worker.start()
    squares = {}
    for _ in numbers:
        n, value = {{slot3}}
        squares[n] = value
    for worker in workers:
        worker.join()
    return squares
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`results.put((n, n * n))`
[Slot 2]	`Process(target=square, args=(n, results))`
[Slot 3]	`results.get()`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 3	`results`	Queue là kênh, không phải kết quả — phải get() từng cặp.
ô 1	`return n * n`	Giá trị return của target bị bỏ đi — cha không nhận được.
ô 2	`Process(target=square(n, results))`	Gọi square ngay trong cha; Process nhận target=None.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 3, ô 2, ô 1.

ĐÁP ÁN MINI-GAME (4 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [2, 1, 3, 0].

Bên trái	NỐI VỚI (đáp án)
Queue()	**kênh IPC dùng chung giữa cha và con**
Process(target=square, args=(n, results))	**mô tả process, chưa chạy**
start()	**square chạy ở process con, put (n, n * n)**
cha get() đủ len(numbers) lần	**join hết → dict**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`results.put((n, n * n))`
ô 2	`Process(target=square, args=(n, results))`
ô 3	`results.get()`

Bẫy ở ô 3 — `results`: Queue là kênh, không phải kết quả — phải get() từng cặp.
Bẫy ở ô 1 — `return n * n`: Giá trị return của target bị bỏ đi — cha không nhận được.
Bẫy ở ô 2 — `Process(target=square(n, results))`: Gọi square ngay trong cha; Process nhận target=None.

parsons-2 · SẮP DÒNG (Parsons) — Ghép các dòng thành chương trình chạy đúng

Thứ tự đúng (theo mảng `lines`): L13 → L14 → L15 → L16 → L17 → L18 → L21 → L22. Thứ tự xáo lúc chơi là `order` = ['L15', 'L17', 'L14', 'L13', 'D0', 'L16', 'L22', 'L21', 'L18'].

```
L13      for _ in numbers:
L14          n, value = results.get()
L15          squares[n] = value
L16      for worker in workers:
L17          worker.join()
L18      return squares
L21  if __name__ == "__main__":
L22      print(run_workers([2, 3, 4]))
```

Phần đầu/cuối đã cho sẵn, học viên không sắp: 12 dòng đầu + 0 dòng cuối.

DÒNG BẪY (không được dùng)	Vì sao sai
`workers = [Process(target=square(n, results)) for n in numbers]`	gọi sẵn target trong process cha

spotError-3 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 22 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 15).
Lỗi: thiếu main guard
Sửa thành:

```
if __name__ == "__main__":
    print(run_workers([2, 3, 4]))
```

```
    15 |         n, value = results.get()
    16 |         squares[n] = value
    17 |     for worker in workers:
    18 |         worker.join()
    19 |     return squares
    20 |
    21 |
>>  22 | print(run_workers([2, 3, 4]))
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — thiếu main guard

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
22	`if __name__ == "__main__":`	`print(run_workers([2, 3, 4]))`
23	`    print(run_workers([2, 3, 4]))`	(xoá dòng)

KIỂU SAI 2 — ghi vào dict toàn cục thay vì Queue

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
—	(không có dòng)	
—	(không có dòng)	`SQUARES = {}`
5	`    results.put((n, n * n))`	`    SQUARES[n] = n * n`
13	`    squares = {}`	(xoá dòng)
14	`    for _ in numbers:`	(xoá dòng)
15	`        n, value = results.get()`	(xoá dòng)
16	`        squares[n] = value`	(xoá dòng)
19	`    return squares`	`    return dict(SQUARES)`

KIỂU SAI 3 — gọi sẵn target trong process cha

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
10	`    workers = [Process(target=square, args=(n, results)) for n in numbers]`	`    workers = [Process(target=square(n, results)) for n in numbers]`

Nguồn trang này: pe-v4 · 007-P07.json · 007-P07.json · qa.json[P07]

M2 · Đa luồng

6 bài trong phạm vi: T01, T02, T03, T04, T05, T06. Chấm trên bản mô phỏng: T01, T02, T03, T04, T05, T06.

T01 · Thread, tài nguyên và concurrency

Mã bài T01 · Nâng cao · M2 · Đa luồng · 25 phút · 20 XP · thứ tự 8 · mức N

MỤC TIÊU BÀI

Phân biệt phần thread dùng chung và phần mỗi thread có riêng.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Một process có thể chạy nhiều thread. Viết scope_of(part): trả "shared" cho phần mọi thread trong process dùng chung ("code", "global", "heap", "file") và "per-thread" cho phần mỗi thread có riêng ("stack", "pc" — program counter, "registers"); tên khác → ValueError. Viết thêm who_am_i() trả tên của thread ĐANG chạy dòng lệnh đó (threading.current_thread().name).

Thực hành: Gom tên tài nguyên vào hai tuple, scope_of chỉ việc tra; who_am_i đọc tên thread LÚC ĐƯỢC GỌI, không viết cứng.

Gọi (đề cho)	Kết quả mong đợi
`scope_of("heap")`	`"shared"`
`scope_of("stack")`	`"per-thread"`
`who_am_i() trong Thread(name="saver")`	`"saver"`
`scope_of("gpu")`	`ValueError`

JSON của bài không khai step_4.contracts (hợp đồng hàm) — chỗ này KHÔNG được suy diễn thêm.

LỜI GIẢI MẪU (qa.json · T01)

```
import threading

SHARED = ("code", "global", "heap", "file")
PER_THREAD = ("stack", "pc", "registers")


def scope_of(part):
    if part in SHARED:
        return "shared"
    if part in PER_THREAD:
        return "per-thread"
    raise ValueError("unknown part: " + str(part))


def who_am_i():
    return threading.current_thread().name


if __name__ == "__main__":
    print(scope_of("heap"))
    print(who_am_i())
```

BỘ CHẤM — 8 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	heap → "shared" (thread không copy heap)	`scope_of("heap")` `→ 'shared'`
2	stack → "per-thread"	`scope_of("stack")` `→ 'per-thread'`
3	global và file dùng chung	`[scope_of("global"), scope_of("file")]` `→ ['shared', 'shared']`
4	code dùng chung; pc, registers riêng	`[scope_of("code"), scope_of("pc"), scope_of("registers")]` `→ ['shared', 'per-thread', 'per-thread']`
5	Tên lạ → ValueError	`_studio.call(scope_of, "gpu").error` `→ 'ValueError'`
6	Gọi ở chương trình chính → "MainThread"	`who_am_i()` `→ 'MainThread'`

7	Gọi trong Thread(name="saver") → "saver"	`_studio.call(lambda box: [(lambda w: (w.start(), »`
		`w.join()))(__import__("threading").Thread(target=lambda: »`
		`box.append(who_am_i()), name=n)) for n in ['saver']] and box, []).value`
		`→ ['saver']`

8	Hai thread khác tên → hai kết quả	`_studio.call(lambda box: [(lambda w: (w.start(), »`
		`w.join()))(__import__("threading").Thread(target=lambda: »`
		`box.append(who_am_i()), name=n)) for n in ['a', 'b']] and box, »`
		`[]).value`
		`→ ['a', 'b']`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Gom tên tài nguyên vào hai tuple, scope_of chỉ việc tra; who_am_i đọc tên thread LÚC ĐƯỢC GỌI, không viết cứng.

Cấu trúc. 1. SHARED = ("code", "global", "heap", "file"); PER_THREAD = ("stack", "pc", "registers"). 2. scope_of: thuộc SHARED → "shared"; thuộc PER_THREAD → "per-thread"; còn lại raise ValueError. 3. who_am_i trả threading.current_thread().name.

Lời giải từng phần. def scope_of(part): if part in SHARED: return "shared" ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — threading, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy threading, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O.

• Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo.

• Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần.

NGỘ NHẬN THƯỜNG GẶP: "Mỗi thread có bản sao heap và biến toàn cục riêng, giống process con"

Đó là hành vi của process: process con làm việc trên vùng nhớ riêng nên sửa list không ảnh hưởng cha. Thread trong cùng process dùng chung code, biến toàn cục, heap và file đang mở; chỉ stack, program counter và thanh ghi là của riêng từng thread. Vì dùng chung nên trao đổi dữ liệu rẻ — và cũng vì dùng chung mà sinh ra race condition (T05).

CÂU DỰ ĐOÁN — đáp án và giải thích

Worker thread append vào list toàn cục shared_log rồi kết thúc. Sau join(), main thread in shared_log thấy gì?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Lúc thấy lúc không, tuỳ máy	Thứ tự chạy không xác định, nhưng sau join() worker chắc chắn đã xong.
B	Không thấy — worker sửa trên bản sao của riêng nó	Đó là process con: nó có vùng nhớ riêng. Thread thì không.
C ĐÚNG	**Thấy phần tử worker vừa thêm — hai thread dùng chung heap của process**	List là object trên heap; mọi thread trong process cùng trỏ tới nó.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Hai thread trong cùng process có nhất thiết copy toàn bộ heap không?

A	Có — mỗi thread một bản heap	Đó là process con, không phải thread.
B	Chỉ copy khi thread ghi vào heap	Copy-on-write là chuyện của fork process.
C ĐÚNG	**Không — chúng dùng chung heap của process**	Chỉ stack, program counter và thanh ghi là riêng.

Giải thích (nguyên văn JSON): Chỉ stack, program counter và thanh ghi là riêng.

2. Hai thread cùng chạy hàm f; trong f có biến cục bộ total. Hai biến total đó...

A	Gây NameError ở thread thứ hai	Mỗi thread có lời gọi f riêng, không lỗi gì.
B	Là một biến dùng chung	Chỉ biến toàn cục và object trên heap mới dùng chung.
C ĐÚNG	**Là hai biến khác nhau — mỗi lời gọi nằm trên stack của thread mình**	Biến cục bộ sống trên stack.

Giải thích (nguyên văn JSON): Biến cục bộ sống trên stack.

3. Cái giá nào đi kèm multi-thread?

A	Các thread không chia sẻ được dữ liệu	Ngược lại: chia sẻ là điểm mạnh của thread.
B ĐÚNG	**Thứ tự chạy không xác định, race condition, deadlock — phải tự đồng bộ**	Cái giá của việc dùng chung bộ nhớ.
C	Mỗi thread tốn một bản sao bộ nhớ của process	Thread dùng chung bộ nhớ của process.

Giải thích (nguyên văn JSON): Cái giá của việc dùng chung bộ nhớ.

CÂU PHẢN VÍ DỤ — đáp án

Thread(target=lambda: print(who_am_i()), name="bg") được start — dòng in ra là gì?

Nhãn	Lựa chọn
A	`MainThread`
B ĐÚNG	`bg`
C	`Thread-1`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao cùng một hàm who_am_i lại trả hai tên khác nhau ở hai thread, trong khi hai thread cùng thấy một list toàn cục?

Mẫu đạt: Code của hàm dùng chung, nhưng mỗi thread là một dòng thực thi riêng với stack và program counter riêng; current_thread() trả về thread ĐANG chạy lệnh đó, nên worker "saver" nhận "saver", main nhận "MainThread". List toàn cục là object trên heap dùng chung, nên thay đổi của thread này thread kia thấy được. Trường hợp đặc biệt: tên lạ như "gpu" không thuộc nhóm nào → ValueError.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	scope_of("heap") → "shared": mọi thread thấy cùng object
Bước 2	scope_of("stack") → "per-thread": biến cục bộ không lẫn
Bước 3	who_am_i() ở chương trình chính → "MainThread"
Bước 4	who_am_i() trong Thread(name="saver") → "saver"

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp hai nhóm tài nguyên và lời gọi đọc tên thread hiện tại.

Yêu cầu thêm: Gọi who_am_i trong hai thread đặt tên khác nhau; thử một tên tài nguyên lạ để thấy ValueError.

```
import threading

SHARED = {{slot1}}
PER_THREAD = {{slot2}}


def scope_of(part):
    if part in SHARED:
        return "shared"
    if part in PER_THREAD:
        return "per-thread"
    raise ValueError("unknown part: " + str(part))


def who_am_i():
    return {{slot3}}
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`("code", "global", "heap", "file")`
[Slot 2]	`("stack", "pc", "registers")`
[Slot 3]	`threading.current_thread().name`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 2	`("stack", "pc", "registers", "global")`	Biến toàn cục dùng chung — chỉ process con mới có bản riêng.
ô 3	`threading.main_thread().name`	Luôn là main thread, không phải thread đang chạy.
ô 3	`"MainThread"`	Viết cứng: gọi trong worker vẫn trả MainThread.
ô 1	`("code", "global", "file")`	Thiếu heap: các thread dùng chung heap, không copy.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 2, ô 3, ô 1.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [3, 2, 0, 1].

Bên trái	NỐI VỚI (đáp án)
scope_of("heap")	**"shared": mọi thread thấy cùng object**
scope_of("stack")	**"per-thread": biến cục bộ không lẫn**
who_am_i() ở chương trình chính	**"MainThread"**
who_am_i() trong Thread(name="saver")	**"saver"**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`("code", "global", "heap", "file")`
ô 2	`("stack", "pc", "registers")`
ô 3	`threading.current_thread().name`

Bẫy ở ô 2 — `("stack", "pc", "registers", "global")`: Biến toàn cục dùng chung — chỉ process con mới có bản riêng.

Bẫy ở ô 3 — `threading.main_thread().name`: Luôn là main thread, không phải thread đang chạy.
Bẫy ở ô 3 — `"MainThread"`: Viết cứng: gọi trong worker vẫn trả MainThread.
Bẫy ở ô 1 — `("code", "global", "file")`: Thiếu heap: các thread dùng chung heap, không copy.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 3, 4 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 1).
Lỗi: heap riêng cho từng thread
Sửa thành:

```
SHARED = ("code", "global", "heap", "file")
PER_THREAD = ("stack", "pc", "registers")
```

```
     1 | import threading
     2 |
>>   3 | SHARED = ("code", "global", "file")
>>   4 | PER_THREAD = ("stack", "pc", "registers", "heap")
     5 |
     6 |
     7 | def scope_of(part):
     8 |     if part in SHARED:
     9 |         return "shared"
    10 |     if part in PER_THREAD:
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: viết cứng tên thread
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
16	`    return "MainThread"`	`    return threading.current_thread().name`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao cùng một hàm who_am_i lại trả hai tên khác nhau ở hai thread, trong khi hai thread cùng thấy một list toàn cục?

Câu hỏi của màn: Vì sao cùng một hàm who_am_i lại trả hai tên khác nhau ở hai thread, trong khi hai thread cùng thấy một list toàn cục?
Lời giải thích mẫu: Code của hàm dùng chung, nhưng mỗi thread là một dòng thực thi riêng với stack và program counter riêng; current_thread() trả về thread ĐANG chạy lệnh đó, nên worker "saver" nhận "saver", main nhận "MainThread". List toàn cục là object trên heap dùng chung, nên thay đổi của thread này thread kia thấy được. Trường hợp đặc biệt: tên lạ như "gpu" không thuộc nhóm nào → ValueError.
Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — heap riêng cho từng thread

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
3	`SHARED = ("code", "global", "heap", "file")`	`SHARED = ("code", "global", "file")`
4	`PER_THREAD = ("stack", "pc", "registers")`	`PER_THREAD = ("stack", "pc", "registers", "heap")`

KIỂU SAI 2 — viết cứng tên thread

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
16	`    return threading.current_thread().name`	`    return "MainThread"`

KIỂU SAI 3 — biến toàn cục copy cho mỗi thread (nhầm với process)

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
3	`SHARED = ("code", "global", "heap", "file")`	`SHARED = ("code", "heap", "file")`
4	`PER_THREAD = ("stack", "pc", "registers")`	`PER_THREAD = ("stack", "pc", "registers", "global")`

Nguồn trang này: pe-v4 · 008-T01.json · 008-T01.json · qa.json[T01]

T02 · Thread target, start và join

Mã bài T02 · Nâng cao · M2 · Đa luồng · 30 phút · 20 XP · thứ tự 9 · mức N

MỤC TIÊU BÀI

Tạo nhiều Thread với target/args, start hết rồi join hết.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

download(url, sizes) đã cấp: "tải" một trang trong PAGES và ghi số ký tự vào dict sizes. Viết download_all(urls): mỗi url một threading.Thread(target=download, args=(url, sizes)); vòng thứ nhất start TẤT CẢ, vòng thứ hai join TẤT CẢ, rồi mới trả sizes. urls rỗng → {} và không tạo thread nào.

Thực hành: Mỗi url một Thread MỚI (target=download, args=(url, sizes)); vòng thứ nhất start hết, vòng thứ hai join hết, rồi mới return sizes.

Gọi (đề cho)	Kết quả mong đợi
`["home.html", "news.html"]`	`{"home.html": 11, "news.html": 13}` — hai thread
`["home.html", "missing.html"]`	`{"home.html": 11, "missing.html": 0}`
`[]`	`{}` — không tạo thread nào

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`download_all(urls)`	Tải nhiều trang, mỗi trang một thread. · urls: list tên trang (có thể rỗng, có thể trùng) · dict {url: số ký tự} · Tạo len(urls) Thread, start hết rồi join hết; không in.

LỜI GIẢI MẪU (qa.json · T02)

```
import threading

PAGES = {
    "home.html": "<h1>Hi</h1>",
    "news.html": "<p>Python</p>",
    "empty.html": "",
}


def download(url, sizes):
    # Giả lập tải trang (cấp sẵn): ngoài đời đây là lúc chờ mạng.
    sizes[url] = len(PAGES.get(url, ""))


def download_all(urls):
    sizes = {}
    threads = []
    for url in urls:
        worker = threading.Thread(target=download, args=(url, sizes))
        threads.append(worker)
    for worker in threads:
        worker.start()
    for worker in threads:
        worker.join()
    return sizes


if __name__ == "__main__":
    print(download_all(["home.html", "news.html"]))
```

BỘ CHẤM — 8 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Hai trang → đúng kích thước	`download_all(["home.html", "news.html"])` `→ {'home.html': 11, 'news.html': 13}`
2	Trang rỗng và trang không có → 0	`download_all(["empty.html", "missing.html"])` `→ {'empty.html': 0, 'missing.html': 0}`

3	Mỗi url một Thread (kể cả url trùng)	`_studio.call(download_all, ["home.html", "news.html", »`
		`"home.html"]).thread_count`
		`→ 3`

4	target là hàm download, không gọi sẵn	`(lambda r: (r.targets_callable, [t.target_name for t in »`
		`r.threads]))(_studio.call(download_all, ["home.html", "news.html"]))`
		`→ (True, ['download', 'download'])`

5	Start hết rồi mới join	`_studio.call(download_all, ["home.html", "news.html"]).starts_before_joins` `→ True`

6	Join đủ mọi worker	`_studio.call(download_all, ["home.html", "news.html", »`
		`"empty.html"]).all_joined`
		`→ True`

7	urls rỗng → {} và không tạo thread	`(lambda r: (r.value, r.thread_count))(_studio.call(download_all, []))` `→ ({}, 0)`

8	Dữ liệu khác (PAGES khác)	`_studio.run(vars={'PAGES': {'x.html': 'abc', 'y.html': 'hello'}}, » entry='download_all(["x.html", "y.html", "z.html"])').value → {'x.html': 3, 'y.html': 5, 'z.html': 0}`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Mỗi url một Thread MỚI (target=download, args=(url, sizes)); vòng thứ nhất start hết, vòng thứ hai join hết, rồi mới return sizes.

Cấu trúc. 1. Tạo dict sizes và list threads rỗng. 2. Mỗi url: threading.Thread(target=download, args=(url, sizes)) rồi append vào threads. 3. Vòng thứ nhất start từng worker; vòng thứ hai join từng worker; cuối cùng return sizes.

Lời giải từng phần. for url in urls: worker = threading.Thread(target=download, args=(url, sizes)) threads.append(worker) ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — threading, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy threading, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O.

• Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo.

• Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần.

NGỘ NHẬN THƯỜNG GẶP: "join() là lệnh cho thread chạy"

join chỉ CHỜ. Thread chỉ chạy khi start(); join một thread chưa start → RuntimeError: cannot join thread before it is started. Ngược lại, gọi run() trực tiếp thì target có chạy nhưng ở thread hiện tại — không có thread mới, không chồng lấp gì cả.

CÂU DỰ ĐOÁN — đáp án và giải thích

Ba trang, mỗi trang chờ mạng 1 giây. Viết "start rồi join ngay" trong CÙNG một vòng lặp thì tổng thời gian khoảng bao nhiêu?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Gần 0 giây vì thread chạy nền	join vẫn chờ từng worker xong.
B	Khoảng 1 giây	Chỉ đúng khi start hết rồi mới join.
C ĐÚNG	**Khoảng 3 giây — join chặn nên worker sau chỉ start khi worker trước xong**	Thành chạy lần lượt; start hết rồi mới join thì khoảng 1 giây.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Gọi t.run() có tạo thread mới không?

A	Có, nhưng không join được	Không có thread mới nào để join.
B	Có, run() và start() như nhau	Chỉ start() tạo dòng thực thi mới.
C ĐÚNG	**Không — target chạy ngay trong thread đang gọi; muốn thread mới phải t.start()**	run() chỉ là hàm gọi target.

Giải thích (nguyên văn JSON): run() chỉ là hàm gọi target.

2. Thread(target=save, args=("data.pkl")) — chuyện gì xảy ra khi start?

A	Chạy bình thường	Thread gọi save(*args) = save("d", "a", "t", ...).
B	Lỗi ngay lúc tạo Thread	Thread chỉ gọi target(*args) khi chạy.
C ĐÚNG	**Lỗi TypeError trong thread: args là chuỗi nên bị tách thành 8 ký tự**	("data.pkl") chỉ là chuỗi trong ngoặc; cần ("data.pkl",).

Giải thích (nguyên văn JSON): ("data.pkl") chỉ là chuỗi trong ngoặc; cần ("data.pkl",).

3. Worker daemon=True đang ghi file thì chương trình chính kết thúc. Điều gì có thể xảy ra?

A ĐÚNG	**Worker bị dừng giữa chừng, file ghi dở**	Daemon không giữ chương trình sống; cần join nếu việc phải xong.
B	Python báo lỗi DaemonError	Không có lỗi nào như vậy.
C	Chương trình chờ worker ghi xong	Đó là thread thường (daemon=False).

Giải thích (nguyên văn JSON): Daemon không giữ chương trình sống; cần join nếu việc phải xong.

CÂU PHẢN VÍ DỤ — đáp án

download_all(["a.html", "b.html", "a.html"]) tạo mấy Thread?

Nhãn	Lựa chọn
A ĐÚNG	3

B	2
C	1

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao download_all phải start hết rồi mới join hết, và vì sao chỉ trả sizes sau vòng join?

Mẫu đạt: start hết trước để các worker cùng chờ mạng chồng lấp nhau; join ngay sau từng start biến chúng thành chạy lần lượt. sizes là dict dùng chung mà các worker ghi vào; chỉ sau khi join hết mới chắc mọi worker đã ghi xong. Trường hợp đặc biệt: urls rỗng → không tạo thread nào, trả {}.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	Thread(target=download, args=(url, sizes)) → mô tả việc, chưa chạy
Bước 2	worker.start() → download chạy trong thread mới
Bước 3	worker.join() → main chờ worker đó xong
Bước 4	return sizes → đọc kết quả sau khi join hết

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp ba chỗ: tạo Thread đúng target/args, start, join.

Yêu cầu thêm: Đổi danh sách trang (có trang không tồn tại, có list rỗng); kiểm số thread bằng số url và kết quả chỉ đọc sau join.

```
def download_all(urls):
    sizes = {}
    threads = []
    for url in urls:
        worker = {{slot1}}
        threads.append(worker)
    for worker in threads:
        {{slot2}}
    for worker in threads:
        {{slot3}}
    return sizes
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`threading.Thread(target=download, args=(url, sizes))`
[Slot 2]	`worker.start()`
[Slot 3]	`worker.join()`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 1	`threading.Thread(target=download, args=url)`	args phải là tuple đủ hai đối số (url, sizes).
ô 2	`worker.run()`	run() chạy target trong thread hiện tại, không tạo thread mới.
ô 3	`time.sleep(1)`	Ngủ không phải chờ: không biết worker đã xong chưa.
ô 1	`threading.Thread(target=download(url, sizes))`	Gọi download ngay ở thread chính; Thread nhận target=None.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 3, ô 1, ô 2.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [3, 2, 0, 1].

Bên trái	NỐI VỚI (đáp án)
Thread(target=download, args=(url, sizes))	**mô tả việc, chưa chạy**
worker.start()	**download chạy trong thread mới**
worker.join()	**main chờ worker đó xong**
return sizes	**đọc kết quả sau khi join hết**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`threading.Thread(target=download, args=(url, sizes))`
ô 2	`worker.start()`
ô 3	`worker.join()`

Bẫy ở ô 1 — `threading.Thread(target=download, args=url)`: args phải là tuple đủ hai đối số (url, sizes).
Bẫy ở ô 2 — `worker.run()`: run() chạy target trong thread hiện tại, không tạo thread mới.

Bẫy ở ô 3 — `time.sleep(1)`: Ngủ không phải chờ: không biết worker đã xong chưa.
Bẫy ở ô 1 — `threading.Thread(target=download(url, sizes))`: Gọi download ngay ở thread chính; Thread nhận target=None.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 19 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 12).
Lỗi: gọi sẵn target
Sửa thành:

```
        worker = threading.Thread(target=download, args=(url, sizes))
```

```
    12 |     sizes[url] = len(PAGES.get(url, ""))
    13 |
    14 |
    15 | def download_all(urls):
    16 |     sizes = {}
    17 |     threads = []
    18 |     for url in urls:
>>  19 |         worker = threading.Thread(target=download(url, sizes))
    20 |         threads.append(worker)
    21 |     for worker in threads:
    22 |         worker.start()
    23 |     for worker in threads:
    24 |         worker.join()
    25 |     return sizes
    26 |
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: run() thay vì start()
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
22	`worker.run()`	`worker.start()`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao download_all phải start hết rồi mới join hết, và vì sao chỉ trả sizes sau vòng join?

Câu hỏi của màn: Vì sao download_all phải start hết rồi mới join hết, và vì sao chỉ trả sizes sau vòng join?
Lời giải thích mẫu: start hết trước để các worker cùng chờ mạng chồng lấp nhau; join ngay sau từng start biến chúng thành chạy lần lượt. sizes là dict dùng chung mà các worker ghi vào; chỉ sau khi join hết mới chắc mọi worker đã ghi xong. Trường hợp đặc biệt: urls rỗng → không tạo thread nào, trả {}.
Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — gọi sẵn target

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
19	`worker = threading.Thread(target=download, » args=(url, sizes))`	`worker = threading.Thread(target=download(url, » sizes))`

KIỂU SAI 2 — run() thay vì start()

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
22	`worker.start()`	`worker.run()`

KIỂU SAI 3 — start và join trong cùng một vòng

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
21	`for worker in threads:`	(xoá dòng)
23	`for worker in threads:`	(xoá dòng)

Nguồn trang này: pe-v4 · 009-T02.json · 009-T02.json · qa.json[T02]

T03 · Subclass Thread

Mã bài T03 · Nâng cao · M2 · Đa luồng · 30 phút · 20 XP · thứ tự 10 · mức N

MỤC TIÊU BÀI

Tạo worker có trạng thái bằng class con của Thread.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết class SumWorker(threading.Thread): __init__(self, numbers) gọi super().__init__() TRƯỚC, rồi lưu self.numbers (list) và self.total = None; override run(self) để gán self.total = sum(self.numbers). Viết parallel_sum(chunks): mỗi khối một SumWorker, start hết, join hết, trả tổng các total. Không có khối nào → 0.

Thực hành: Kế thừa threading.Thread, gọi super().__init__() đầu tiên, lưu trạng thái vào self; việc tính đặt trong run(). parallel_sum tạo worker, start hết, join hết rồi cộng total.

Gọi (đề cho)	Kết quả mong đợi
`parallel_sum([[1, 2, 3], [10, 20]])`	`36`
`SumWorker([1, 2]).total trước start`	`None`
`parallel_sum([])`	`0 — không tạo worker`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`SumWorker(numbers) — class con của » threading.Thread`	Một worker tự giữ dữ liệu và kết quả của nó. · numbers: iterable các số (list, tuple, range...) · object Thread; total = None tới khi run() chạy xong · run() gán self.total = sum(numbers); không in.
`parallel_sum(chunks)`	Cộng nhiều khối số, mỗi khối một SumWorker. · chunks: list các khối số (có thể rỗng) · int: tổng mọi total · Tạo len(chunks) worker, start hết rồi join hết.

LỜI GIẢI MẪU (qa.json · T03)

```
import threading


class SumWorker(threading.Thread):
    def __init__(self, numbers):
        super().__init__()
        self.numbers = list(numbers)
        self.total = None

    def run(self):
        self.total = sum(self.numbers)


def parallel_sum(chunks):
    workers = [SumWorker(chunk) for chunk in chunks]
    for worker in workers:
        worker.start()
    for worker in workers:
        worker.join()
    return sum(worker.total for worker in workers)


if __name__ == "__main__":
    print(parallel_sum([[1, 2, 3], [10, 20]]))
```

BỘ CHẤM — 7 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	SumWorker là một Thread	`isinstance(SumWorker([1]), __import__("threading").Thread)` `→ True`
2	Chưa start → total là None	`SumWorker([1, 2]).total` `→ None`
3	start + join → total đúng	`(lambda w: (w.start(), w.join(), w.total)[2])(SumWorker([4, 5, 6]))` `→ 15`
4	parallel_sum ba khối	`parallel_sum([[1, 2], [3], [4, 5, 6]])` `→ 21`
5	Mỗi khối một worker, start hết rồi join hết	`(lambda r: (r.thread_count, r.starts_before_joins, » r.all_joined))(_studio.call(parallel_sum, [[1], [2], [3]]))` `→ (3, True, True)`
6	Khối rỗng góp 0; nhận cả range/tuple	`parallel_sum([[], range(5), (1, 1)])` `→ 12`
7	Không có khối → 0, không tạo worker	`(lambda r: (r.value, r.thread_count))(_studio.call(parallel_sum, []))` `→ (0, 0)`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Kế thừa threading.Thread, gọi super().__init__() đầu tiên, lưu trạng thái vào self; việc tính đặt trong run(). parallel_sum tạo worker, start hết, join hết rồi cộng total.

Cấu trúc. 1. class SumWorker(threading.Thread); __init__ gọi super().__init__() rồi gán self.numbers, self.total = None. 2. def run(self): self.total = sum(self.numbers). 3. parallel_sum: một SumWorker cho mỗi khối; start hết; join hết; trả sum các worker.total.

Lời giải từng phần. class SumWorker(threading.Thread): def __init__(self, numbers): super().__init__() ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — threading, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy threading, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O.

• Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo.

• Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần.

NGỘ NHẬN THƯỜNG GẶP: "Override start() cho tiện, vì start mới là thứ chạy thread"

start() là của Thread: nó dựng thread mới rồi gọi run(). Override start() thì không còn thread nào được tạo, và join() sau đó báo RuntimeError: cannot join thread before it is started. Việc của worker đặt trong run(); nơi dùng gọi start().

CÂU DỰ ĐOÁN — đáp án và giải thích

class W(threading.Thread) override run(). Chạy w = W(); w.start(); w.join() — run() chạy mấy lần, ở thread nào?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Hai lần: ở start và ở join	join chỉ chờ, không chạy lại gì.
B ĐÚNG	**Một lần, trong thread mới do start() tạo**	start() gọi run() trong thread con.
C	Không lần nào — phải tự gọi w.run()	Gọi run() trực tiếp thì chạy ở thread hiện tại.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Method nào được override, method nào được gọi để khởi chạy?

A ĐÚNG	**Override run(); gọi start()**	start() tạo thread rồi gọi run() bên trong nó.
B	Override __init__; gọi join()	join chỉ chờ thread đã start.
C	Override start(); gọi run()	Như vậy không có thread mới nào.

Giải thích (nguyên văn JSON): start() tạo thread rồi gọi run() bên trong nó.

2. class W(Thread) có __init__ riêng nhưng quên super().__init__(). Gọi W(...).start() thì...

A ĐÚNG	**RuntimeError: thread.__init__() not called**	Phần khởi tạo của Thread chưa chạy (Studio mô phỏng báo AttributeError ... started — cùng nguyên nhân).
B	Chạy bình thường	Thread kiểm tra và từ chối start.
C	run() chạy ở main thread	Không chạy gì cả — start() báo lỗi trước.

Giải thích (nguyên văn JSON): Phần khởi tạo của Thread chưa chạy (Studio mô phỏng báo AttributeError ... started — cùng nguyên nhân).

3. Đọc w.total ngay sau w.start(), chưa join, thì...

A	Lỗi AttributeError	total đã gán None trong __init__.
B ĐÚNG	**Có thể vẫn là None — worker chưa chắc đã chạy xong**	Đọc kết quả sau join().
C	Luôn đã có kết quả	start() không chờ worker.

Giải thích (nguyên văn JSON): Đọc kết quả sau join().

CÂU PHẢN VÍ DỤ — đáp án

parallel_sum([[2, 3], [], [5]]) trả về?

Nhãn	Lựa chọn
A	`5`
B ĐÚNG	`10`
C	`None`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao total khởi tạo None trong __init__ và chỉ được tính trong run()?

Mẫu đạt: __init__ chạy ở thread tạo object (main thread), còn run() chạy trong thread mới sau start(). Tính tổng trong __init__ là làm việc ở main thread, worker không còn gì để làm. Để None tới khi run() xong giúp phân biệt "chưa chạy" với "tổng bằng 0"; nơi gọi đọc total sau join. Trường hợp đặc biệt: khối rỗng → total = 0, không phải None.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	SumWorker([1, 2]) → __init__: lưu numbers, total = None
Bước 2	worker.start() → Thread gọi run() trong thread mới
Bước 3	run() → self.total = 3
Bước 4	worker.join() → đọc total an toàn

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp lớp cha, lời gọi khởi tạo của Thread, tên method được override và lệnh khởi chạy.

Yêu cầu thêm: Thử khối rỗng, nhiều khối, và đọc total trước khi start để thấy None.

```
import threading


class SumWorker({{slot1}}):
    def __init__(self, numbers):
        {{slot2}}
        self.numbers = list(numbers)
        self.total = None

    def {{slot3}}(self):
        self.total = sum(self.numbers)


def parallel_sum(chunks):
    workers = [SumWorker(chunk) for chunk in chunks]
    for worker in workers:
        {{slot4}}
    for worker in workers:
        worker.join()
    return sum(worker.total for worker in workers)
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`threading.Thread`
[Slot 2]	`super().__init__()`
[Slot 3]	`run`
[Slot 4]	`worker.start()`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 4	`worker.run()`	Chạy run() ngay trong main thread.
ô 2	`self.total = sum(self.numbers)`	Tính ở __init__ = làm việc ở main thread; lại thiếu super().__init__().
ô 1	`threading.Thread()`	Kế thừa CLASS Thread, không phải một object Thread.
ô 3	`start`	Override start thì không còn thread nào được tạo.

Ngân hàng còn 4 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 4, ô 3, ô 2, ô 1.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [2, 0, 1, 3].

Bên trái	NỐI VỚI (đáp án)
SumWorker([1, 2])	**__init__: lưu numbers, total = None**
worker.start()	**Thread gọi run() trong thread mới**
run()	**self.total = 3**
worker.join()	**đọc total an toàn**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`threading.Thread`
ô 2	`super().__init__()`
ô 3	`run`

ô 4	`worker.start()`

Bẫy ở ô 4 — `worker.run()`: Chạy run() ngay trong main thread.
Bẫy ở ô 2 — `self.total = sum(self.numbers)`: Tính ở __init__ = làm việc ở main thread; lại thiếu super().__init__().
Bẫy ở ô 1 — `threading.Thread()`: Kế thừa CLASS Thread, không phải một object Thread.
Bẫy ở ô 3 — `start`: Override start thì không còn thread nào được tạo.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 10 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 3).
Lỗi: override start() thay vì run()
Sửa thành:

```
    def run(self):
```

```
 3 |
 4 | class SumWorker(threading.Thread):
 5 |     def __init__(self, numbers):
 6 |         super().__init__()
 7 |         self.numbers = list(numbers)
 8 |         self.total = None
 9 |
>> 10 |     def start(self):
11 |         self.total = sum(self.numbers)
12 |
13 |
14 | def parallel_sum(chunks):
15 |     workers = [SumWorker(chunk) for chunk in chunks]
16 |     for worker in workers:
17 |         worker.start()
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: quên super().__init__()
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
6	`(không có dòng)`	`        super().__init__()`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao total khởi tạo None trong __init__ và chỉ được tính trong run()?

Câu hỏi của màn: Vì sao total khởi tạo None trong __init__ và chỉ được tính trong run()?
Lời giải thích mẫu: __init__ chạy ở thread tạo object (main thread), còn run() chạy trong thread mới sau start(). Tính tổng trong __init__ là làm việc ở main thread, worker không còn gì để làm. Để None tới khi run() xong giúp phân biệt "chưa chạy" với "tổng bằng 0"; nơi gọi đọc total sau join. Trường hợp đặc biệt: khối rỗng → total = 0, không phải None.
Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 4 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — override start() thay vì run()

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
10	`    def run(self):`	`    def start(self):`

KIỂU SAI 2 — quên super().__init__()

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
6	`        super().__init__()`	`(xoá dòng)`

KIỂU SAI 3 — tính tổng ngay trong __init__

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
8	`        self.total = None`	`        self.total = sum(self.numbers)`

KIỂU SAI 4 — gọi run() thay vì start()

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
17	`        worker.start()`	`        worker.run()`

Nguồn trang này: pe-v4 · 010-T03.json · 010-T03.json · qa.json[T03]

T04 · GIL, CPU-bound và I/O-bound

Mã bài T04 · Nâng cao · M2 · Đa luồng · 30 phút · 20 XP · thứ tự 11 · mức N

MỤC TIÊU BÀI

Ước lượng thời gian theo workload, chọn thread hay process.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Mỗi task là (kind, seconds): "cpu" = tính Python thuần, "io" = chờ (mạng, đĩa). Viết wall_time(tasks, strategy) theo mô hình lý tưởng (mỗi task một worker, đủ lõi, bỏ qua chi phí tạo): "serial" = cộng mọi task; "thread" (CPython có GIL) = max(tổng giây CPU, task I/O dài nhất); "process" = task dài nhất; strategy khác → ValueError; không có task → 0. Viết best_strategy(tasks): cách có wall_time nhỏ nhất, bằng nhau thì chọn cách đứng trước trong serial → thread → process (rẻ hơn).

Thực hành: Tách giây CPU và giây I/O; serial cộng hết, thread lấy max(tổng CPU, I/O dài nhất), process lấy task dài nhất. best_strategy so lần lượt serial → thread → process, chỉ đổi khi NHANH HƠN hẳn.

Gọi (đề cho)	Kết quả mong đợi
`wall_time([("io", 2), ("io", 3), ("io", 1)], "thread")`	`3`
`wall_time([("cpu", 2), ("cpu", 3)], "thread")`	`5 — GIL, không nhanh hơn tuần tự`
`wall_time([("cpu", 2), ("cpu", 3)], "process")`	`3`
`best_strategy([("cpu", 1)])`	`"serial" — bằng nhau thì chọn cách đơn giản`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`wall_time(tasks, strategy)`	Ước lượng thời gian chạy lý tưởng của một workload. · tasks: list (kind, seconds), kind "cpu"/"io"; strategy: "serial" | "thread" | "process" · số giây (int/float); strategy lạ → ValueError · Không chạy gì thật, không in. Mô hình: mỗi task một worker, đủ lõi, bỏ qua chi phí tạo.
`best_strategy(tasks)`	Chọn cách chạy nhanh nhất; bằng nhau thì chọn cách đơn giản hơn. · tasks: như wall_time · "serial" | "thread" | "process" · So lần lượt serial → thread → process; chỉ đổi khi NHANH HƠN hẳn.

LỜI GIẢI MẪU (qa.json · T04)

```
def wall_time(tasks, strategy):
    cpu = [seconds for kind, seconds in tasks if kind == "cpu"]
    io = [seconds for kind, seconds in tasks if kind == "io"]
    if strategy == "serial":
        return sum(cpu) + sum(io)
    if strategy == "thread":
        return max(sum(cpu), max(io, default=0))
    if strategy == "process":
        return max(cpu + io, default=0)
    raise ValueError("unknown strategy: " + str(strategy))


def best_strategy(tasks):
    best = "serial"
    for strategy in ("thread", "process"):
        if wall_time(tasks, strategy) < wall_time(tasks, best):
            best = strategy
    return best


if __name__ == "__main__":
    work = [("io", 2), ("io", 3), ("cpu", 4)]
    print(wall_time(work, "serial"), wall_time(work, "thread"), wall_time(work, "process"))
    print(best_strategy(work))
```

BỘ CHẤM — 9 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Toàn I/O, thread → lần chờ dài nhất	`wall_time([("io", 2), ("io", 3), ("io", 1)], "thread")` `→ 3`
2	Toàn CPU, thread → cộng dồn (GIL)	`wall_time([("cpu", 2), ("cpu", 3)], "thread")` `→ 5`
3	Toàn CPU, process → task dài nhất	`wall_time([("cpu", 2), ("cpu", 3)], "process")` `→ 3`

4	Trộn: serial và thread	`[wall_time([("cpu", 4), ("io", 3), ("io", 5)], s) for s in ("serial", »`
		`"thread")]`
		`→ [12, 5]`

5	Số thực: CPU 0.1 + 0.2 với I/O 0.25	`[round(wall_time([("cpu", 0.1), ("cpu", 0.2), ("io", 0.25)], s), 6) for s »`
		`in ("serial", "thread", "process")]`
		`→ [0.55, 0.3, 0.25]`

6	Không có task → 0	`[wall_time([], s) for s in ("serial", "thread", "process")]` `→ [0, 0, 0]`
7	strategy lạ → ValueError	`_studio.call(wall_time, [("io", 1)], "gpu").error` `→ 'ValueError'`
8	best_strategy theo workload	`[best_strategy(t) for t in ([("io", 2), ("io", 2)], [("cpu", 2), ("cpu", » 2)], [("cpu", 3), ("io", 2), ("io", 2)], [("cpu", 2), ("cpu", 2), » ("io", 1)])]` `→ ['thread', 'process', 'thread', 'process']`
9	Bằng nhau → chọn cách đơn giản hơn	`[best_strategy([("cpu", 1)]), best_strategy([])]` `→ ['serial', 'serial']`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Tách giây CPU và giây I/O; serial cộng hết, thread lấy max(tổng CPU, I/O dài nhất), process lấy task dài nhất. best_strategy so lần lượt serial → thread → process, chỉ đổi khi NHANH HƠN hẳn.

Cấu trúc. 1. cpu = các seconds có kind "cpu"; io = các seconds có kind "io". 2. serial: sum(cpu) + sum(io); thread: max(sum(cpu), max(io, default=0)); process: max(cpu + io, default=0). 3. strategy lạ → raise ValueError. 4. best_strategy: best = "serial"; với "thread" rồi "process", nếu wall_time nhỏ hơn hẳn thì đổi best.

Lời giải từng phần. if strategy == "thread": return max(sum(cpu), max(io, default=0)) ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• Bài này không chạy thread: bộ chấm dùng ĐỒNG HỒ ẢO (số giây ghi trong từng task), vì trình duyệt chạy Python trong một luồng duy nhất và thời gian đo thật thì mỗi lần một khác.

• Muốn thấy GIL thật, chạy trên máy: hai thread cùng tính sum(range(10**7)) rồi so với hai thread cùng time.sleep(1), đo bằng time.perf_counter().

NGỘ NHẬN THƯỜNG GẶP: "Thêm thread thì chương trình Python nào cũng nhanh hơn"

Chỉ đúng khi thời gian chủ yếu là CHỜ (I/O). Với tính toán Python thuần, GIL cho mỗi lúc một thread chạy bytecode nên thêm thread không giảm thời gian, có khi còn chậm hơn vì chuyển ngữ cảnh. Chọn theo loại workload, không theo số thread.

CÂU DỰ ĐOÁN — đáp án và giải thích

Hai task tính Python thuần, mỗi task 3 giây, chạy bằng 2 thread trên máy 4 lõi (CPython có GIL). Tổng thời gian khoảng bao nhiêu?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	≈ 1.5 giây	Không có cơ chế nào chia đôi một task.
B	≈ 3 giây — mỗi thread một lõi	Đúng với 2 process, không đúng với thread khi có GIL.
C ĐÚNG	**≈ 6 giây — hai thread thay nhau giữ GIL**	Phần CPU cộng dồn; muốn ≈ 3 giây phải dùng 2 process.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Có GIL thì race condition của ứng dụng có tự biến mất không?

A	Có — mỗi lúc chỉ một thread chạy	Thread có thể bị đổi giữa hai bytecode của cùng một câu lệnh.
B ĐÚNG	**Không — GIL bảo vệ interpreter, không bảo vệ các bước đọc-sửa-ghi của bạn**	counter += 1 vẫn có thể bị xen giữa; cần Lock (T05).
C	Chỉ biến mất khi dùng list	List không có cơ chế khoá nào cho bạn.

Giải thích (nguyên văn JSON): counter += 1 vẫn có thể bị xen giữa; cần Lock (T05).

2. Tải 50 trang web, mỗi trang chủ yếu chờ mạng. Lựa chọn hợp lý?

A	Không nên song song	Chờ tuần tự 50 trang là lãng phí.
B	Process — vì thread bị GIL	GIL được nhả trong lúc chờ I/O; process tốn hơn mà không nhanh hơn.
C ĐÚNG	**Thread — các lần chờ mạng chồng lên nhau**	Thread đang chờ I/O nhả GIL.

Giải thích (nguyên văn JSON): Thread đang chờ I/O nhả GIL.

3. Phát biểu nào về NumPy đúng?

A	NumPy không dùng được trong thread	Dùng được bình thường.
B ĐÚNG	**Một số phép tính native nhả GIL, nhưng không suy ra được mọi code dùng NumPy đều chạy song song**	Phải xét từng thao tác, đo thật trên máy.
C	NumPy luôn bỏ qua GIL	Code Python bao quanh lời gọi NumPy vẫn cần GIL.

Giải thích (nguyên văn JSON): Phải xét từng thao tác, đo thật trên máy.

CÂU PHẢN VÍ DỤ — đáp án

best_strategy([("cpu", 2), ("cpu", 2), ("io", 1)]) trả về?

Nhãn	Lựa chọn
A	`thread`
B	`serial`
C ĐÚNG	`process`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao wall_time(..., "thread") cộng phần CPU nhưng lấy max phần I/O?

Mẫu đạt: Với GIL, mỗi lúc chỉ một thread chạy bytecode nên các task CPU phải thay nhau: thời gian cộng dồn. Thread đang chờ I/O nhả GIL nên các lần chờ diễn ra cùng lúc và chồng lên cả lúc thread khác tính; vì vậy thời gian là max(tổng CPU, lần chờ dài nhất). Trường hợp đặc biệt: không có task nào → 0.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	serial → cộng mọi task: 2 + 3 + 4 = 9
Bước 2	thread → max(tổng CPU, I/O dài nhất) = max(4, 3) = 4
Bước 3	process → task dài nhất = 4
Bước 4	best_strategy → nhanh nhất; bằng nhau chọn cách rẻ hơn

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp công thức của ba cách chạy theo mô hình GIL.

Yêu cầu thêm: Thử workload toàn CPU, toàn I/O và trộn; xem khi nào thread thắng, khi nào cần process.

```
def wall_time(tasks, strategy):
    cpu = [seconds for kind, seconds in tasks if kind == "cpu"]
    io = [seconds for kind, seconds in tasks if kind == "io"]
    if strategy == "serial":
        return {{slot1}}
    if strategy == "thread":
        return {{slot2}}
    if strategy == "process":
        return {{slot3}}
    raise ValueError("unknown strategy: " + str(strategy))
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`sum(cpu) + sum(io)`
[Slot 2]	`max(sum(cpu), max(io, default=0))`
[Slot 3]	`max(cpu + io, default=0)`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 2	`max(sum(cpu), sum(io))`	Các lần chờ I/O chồng lên nhau: lấy lần dài nhất, không cộng.
ô 2	`sum(cpu) + max(io, default=0)`	Lúc thread này chờ I/O, thread khác vẫn tính được — không nối tiếp.
ô 3	`sum(cpu + io) / len(tasks)`	Không có phép chia đều: mỗi task chạy trọn thời gian của nó.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 1, ô 2, ô 3.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

traceTable-0 · BẢNG THEO DÕI BIẾN — Theo dõi biến: điền giá trị sau mỗi dòng

```
tasks = [("io", 2), ("io", 3), ("cpu", 4)]
cpu_total = sum(s for kind, s in tasks if kind == "cpu")
io_longest = max(s for kind, s in tasks if kind == "io")
serial = sum(s for kind, s in tasks)
threads = max(cpu_total, io_longest)
processes = max(s for kind, s in tasks)
print("serial:", serial, "| thread:", threads, "| process:", processes)
```

#	Dòng chạy	cpu_total	io_longest	serial
0	`cpu_total = sum(s for kind, s in » tasks i`	4	–	–
1	`io_longest = max(s for kind, s in » tasks`	4	3	–
2	`serial = sum(s for kind, s in tasks)`	4	3	9
3	`threads = max(cpu_total, io_longest)`	4	3	9

4	`processes = max(s for kind, s in » tasks)`	4	3	9

Ô IN XANH ĐẬM là ô học viên phải điền (2 ô); ô xám là giá trị bài cho sẵn. So sánh bỏ nháy và không phân biệt hoa/thường.

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`sum(cpu) + sum(io)`
ô 2	`max(sum(cpu), max(io, default=0))`
ô 3	`max(cpu + io, default=0)`

Bẫy ở ô 2 — `max(sum(cpu), sum(io))`: Các lần chờ I/O chồng lên nhau: lấy lần dài nhất, không cộng.
Bẫy ở ô 2 — `sum(cpu) + max(io, default=0)`: Lúc thread này chờ I/O, thread khác vẫn tính được — không nối tiếp.
Bẫy ở ô 3 — `sum(cpu + io) / len(tasks)`: Không có phép chia đều: mỗi task chạy trọn thời gian của nó.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 7 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 1).
Lỗi: quên GIL: coi thread chạy song song cả phần CPU
Sửa thành:

```
        return max(sum(cpu), max(io, default=0))
```

```
    1 | def wall_time(tasks, strategy):
    2 |     cpu = [seconds for kind, seconds in tasks if kind == "cpu"]
    3 |     io = [seconds for kind, seconds in tasks if kind == "io"]
    4 |     if strategy == "serial":
    5 |         return sum(cpu) + sum(io)
    6 |     if strategy == "thread":
>>  7 |         return max(cpu + io, default=0)
    8 |     if strategy == "process":
    9 |         return max(cpu + io, default=0)
   10 |     raise ValueError("unknown strategy: " + str(strategy))
   11 |
   12 |
   13 | def best_strategy(tasks):
   14 |     best = "serial"
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: tưởng GIL chặn cả lúc chờ I/O
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
7	`return sum(cpu) + sum(io)`	`return max(sum(cpu), max(io, default=0))`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao wall_time(..., "thread") cộng phần CPU nhưng lấy max phần I/O?

Câu hỏi của màn: Vì sao wall_time(..., "thread") cộng phần CPU nhưng lấy max phần I/O?
Lời giải thích mẫu: Với GIL, mỗi lúc chỉ một thread chạy bytecode nên các task CPU phải thay nhau: thời gian cộng dồn. Thread đang chờ I/O nhả GIL nên các lần chờ diễn ra cùng lúc và chồng lên cả lúc thread khác tính; vì vậy thời gian là max(tổng CPU, lần chờ dài nhất). Trường hợp đặc biệt: không có task nào → 0.
Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — quên GIL: coi thread chạy song song cả phần CPU

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
7	`return max(sum(cpu), max(io, default=0))`	`return max(cpu + io, default=0)`

KIỂU SAI 2 — tưởng GIL chặn cả lúc chờ I/O

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
7	`return max(sum(cpu), max(io, default=0))`	`return sum(cpu) + sum(io)`

KIỂU SAI 3 — bằng nhau vẫn đổi sang cách tốn hơn

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
16	`if wall_time(tasks, strategy) < wall_time(tasks, » best):`	`if wall_time(tasks, strategy) <= » wall_time(tasks, best):`

Nguồn trang này: pe-v4 · 011-T04.json · 011-T04.json · qa.json[T04]

T05 · Lock, race condition và deadlock

Mã bài T05 · Nâng cao · M2 · Đa luồng · 40 phút · 20 XP · thứ tự 12 · mức N

MỤC TIÊU BÀI

Bảo vệ dữ liệu chung bằng Lock và tránh deadlock.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Account có self.balance và self.lock = threading.Lock(); run_deposits (cấp sẵn) cho mỗi khoản nạp một thread. Viết deposit(amount): đọc-cộng-ghi balance trong with self.lock. Viết withdraw(amount): trong CÙNG một with self.lock, nếu amount > balance thì raise ValueError("insufficient funds"), ngược lại trừ. Viết safe_plan(plan, order): True nếu plan (các lock một thread khoá lồng nhau, theo thứ tự) đi đúng thứ tự chung order — tăng dần, không khoá lại lock đang giữ; tên không có trong order → ValueError.

Thực hành: Mọi lần đọc/ghi self.balance nằm trong with self.lock; withdraw kiểm tra và trừ trong CÙNG một khối. safe_plan đổi tên lock thành vị trí trong order rồi kiểm dãy vị trí tăng hẳn.

Gọi (đề cho)	Kết quả mong đợi
`run_deposits(Account(0), [10, 20, 30])`	`60`
`Account(5).withdraw(9)`	`ValueError; balance vẫn 5; khoá đã nhả`
`safe_plan(["db", "log"], ["db", "cache", "log"])`	`True`
`safe_plan(["log", "db"], ["db", "cache", "log"])`	`False — ngược thứ tự chung`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`Account.deposit(self, amount)`	Nạp tiền an toàn khi nhiều thread cùng gọi. · self; amount: số cần cộng · None · Đọc-cộng-ghi self.balance trong with self.lock; lỗi giữa chừng vẫn nhả khoá.
`Account.withdraw(self, amount)`	Rút tiền: kiểm tra rồi trừ như MỘT bước. · self; amount: số cần trừ · None; amount > balance → ValueError("insufficient funds") · Kiểm tra và trừ trong cùng một with self.lock; rút đúng bằng số dư là hợp lệ.
`safe_plan(plan, order)`	Kiểm một thread có khoá theo đúng thứ tự đã thống nhất không. · plan: list tên lock theo thứ tự thread khoá; order: list tên lock theo thứ tự chung · bool; tên lock không có trong order → ValueError · Không khoá gì thật, không in.

LỜI GIẢI MẪU (qa.json · T05)

```
import threading


class Account:
    def __init__(self, balance=0):
        self.balance = balance
        self.lock = threading.Lock()

    def deposit(self, amount):
        with self.lock:
            self.balance += amount

    def withdraw(self, amount):
        with self.lock:
            if amount > self.balance:
                raise ValueError("insufficient funds")
            self.balance -= amount


def run_deposits(account, amounts):
    threads = [threading.Thread(target=account.deposit, args=(amount,)) for amount in amounts]
    for worker in threads:
        worker.start()
    for worker in threads:
        worker.join()
    return account.balance


def safe_plan(plan, order):
    positions = [order.index(name) for name in plan]
    for i in range(1, len(positions)):
        if positions[i] <= positions[i - 1]:
            return False
    return True


if __name__ == "__main__":
    account = Account(100)
    print(run_deposits(account, [10, 20, 30]))
    print(safe_plan(["db", "log"], ["db", "cache", "log"]))
```

BỘ CHẤM — 13 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Ba thread nạp 10, 20, 30 → 60	`run_deposits(Account(0), [10, 20, 30])` `→ 60`
2	Không có khoản nạp → giữ số dư đầu	`run_deposits(Account(7), [])` `→ 7`
3	deposit đọc và ghi balance khi đang giữ self.lock	`_studio.check('class Probe:\n def __init__(self):\n self.held = » False\n\n def acquire(self, blocking=True, timeout=-1):\n » assert not self.held, "khoá lại self.lock khi đang giữ nó — Lock thật » sẽ treo ở đây"\n self.held = True\n return True\n\n » def release(self):\n self.held = False\n\n def » locked(self):\n return self.held\n\n def __enter__(self):\n » self.acquire()\n return self\n\n def __exit__(self, *exc):\n » self.release()\n return False\n\n\nclass Spy(Account):\n log » = None\n\n @property\n def balance(self):\n if self.log » is not None:\n self.log.append(self.lock.held)\n » return self._value\n\n @balance.setter\n def balance(self, » value):\n if self.log is not None:\n » self.log.append(self.lock.held)\n self._value = value\n\n\na = » Spy(10)\na.lock = Probe()\na.log = []\na.deposit(5)\nassert a.log, » "deposit không đọc/ghi self.balance"\nassert all(a.log), "deposit đọc » hoặc ghi balance khi CHƯA giữ self.lock"\nassert not a.lock.held, » "deposit xong mà chưa nhả self.lock"\nassert a._value == 15, "balance » sau deposit(5) phải là 15"\n')` `→ True`
4	withdraw: kiểm tra VÀ trừ cùng trong self.lock	`_studio.check('class Probe:\n def __init__(self):\n self.held = » False\n\n def acquire(self, blocking=True, timeout=-1):\n » assert not self.held, "khoá lại self.lock khi đang giữ nó — Lock thật » sẽ treo ở đây"\n self.held = True\n return True\n\n » def release(self):\n self.held = False\n\n def » locked(self):\n return self.held\n\n def __enter__(self):\n » self.acquire()\n return self\n\n def __exit__(self, *exc):\n » self.release()\n return False\n\n\nclass Spy(Account):\n log » = None\n\n @property\n def balance(self):\n if self.log » is not None:\n self.log.append(self.lock.held)\n » return self._value\n\n @balance.setter\n def balance(self, » value):\n if self.log is not None:\n » self.log.append(self.lock.held)\n self._value = value\n\n\na = » Spy(10)\na.lock = Probe()\na.log = []\na.withdraw(4)\nassert » len(a.log) >= 2, "withdraw phải đọc balance để kiểm tra rồi ghi » lại"\nassert all(a.log), "phép KIỂM TRA số dư và phép TRỪ phải cùng » nằm trong with self.lock"\nassert not a.lock.held, "withdraw xong mà » chưa nhả self.lock"\nassert a._value == 6, "balance sau withdraw(4) » phải là 6"\n')` `→ True`
5	Rút quá số dư → ValueError, số dư không đổi	`(lambda a: (_studio.call(a.withdraw, 9).error, a.balance))(Account(5))` `→ ('ValueError', 5)`
6	Sau ValueError, khoá đã được nhả	`(lambda a: (_studio.call(a.withdraw, 9).error, a.lock.locked()))(Account(5))` `→ ('ValueError', False)`
7	Nạp sai kiểu → TypeError, khoá vẫn được nhả	`(lambda a: (_studio.call(a.deposit, "10").error, » a.lock.locked()))(Account(0))` `→ ('TypeError', False)`
8	Rút đúng bằng số dư → 0	`(lambda a: (a.withdraw(5), a.balance)[1])(Account(5))` `→ 0`
9	safe_plan: đúng thứ tự (kể cả bỏ qua lock giữa, plan rỗng)	`[safe_plan(["db", "log"], ["db", "cache", "log"]), safe_plan(["cache"], » ["db", "cache"]), safe_plan([], ["db"])]` `→ [True, True, True]`
10	safe_plan: ngược thứ tự → False	`[safe_plan(["log", "db"], ["db", "cache", "log"]), safe_plan(["db", "log", » "cache"], ["db", "cache", "log"])]` `→ [False, False]`
11	safe_plan: khoá lại lock đang giữ → False	`safe_plan(["db", "db"], ["db", "log"])` `→ False`
12	safe_plan theo order đã thống nhất, không theo bảng chữ cái	`safe_plan(["log", "cache"], ["log", "cache", "db"])` `→ True`
13	safe_plan: lock lạ → ValueError	`_studio.call(safe_plan, ["db", "x"], ["db", "log"]).error` `→ 'ValueError'`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Mọi lần đọc/ghi self.balance nằm trong with self.lock; withdraw kiểm tra và trừ trong CÙNG một khối. safe_plan đổi tên lock thành vị trí trong order rồi kiểm dãy vị trí tăng hẳn.

Cấu trúc. 1. deposit: with self.lock: self.balance += amount. 2. withdraw: with self.lock: nếu amount > self.balance → raise ValueError("insufficient funds"); ngược lại self.balance -= amount. 3. safe_plan: positions = [order.index(name) for name in plan]; có vị trí nào ≤ vị trí trước nó → False; còn lại True.

Lời giải từng phần. def deposit(self, amount): with self.lock: self.balance += amount ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — threading, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy threading, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O.

• Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo.

• Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần.

• Vì các thread không bao giờ xen kẽ ở đây, race condition không tái hiện được: bộ chấm thay self.lock bằng một khoá "ghi nhật ký" để kiểm mọi lần đọc/ghi balance đều nằm trong lúc giữ khoá.

• Khoá lại một Lock đang giữ thì trên máy thật chương trình treo mãi.

• Trên máy cũng đừng chờ n += 1 trần lộ race: từ CPython 3.10 thread chỉ bị đổi ở một số điểm (gọi hàm, cuối vòng lặp...), nên 2 thread × 1 triệu lần += thường vẫn ra đúng 2.000.000 — hiếm lộ, KHÔNG phải nguyên tử.

• Muốn thấy race thật, chạy python race_demo.py (mục "File trong thư mục bài"): file chèn time.sleep(0) giữa đọc và ghi nên thường ra dưới 200; bọc ba dòng đó trong with lock: thì luôn 200.

FILE CẤP SẴN CHO BÀI · race_demo.py (module) — học viên không phải viết file này

```
# Chạy trên MÁY: python race_demo.py — Studio chạy thread lần lượt nên không thấy race.
import threading
import time

balance = 0


def deposit_100():
    global balance
    for _ in range(100):
        seen = balance       # đọc
        time.sleep(0)        # nhường lượt: thread kia chen vào ĐÚNG giữa đọc và ghi
        balance = seen + 1   # ghi: đè mất phần thread kia vừa cộng


if __name__ == "__main__":
    workers = [threading.Thread(target=deposit_100) for _ in range(2)]
    for worker in workers:
        worker.start()
    for worker in workers:
        worker.join()
    print("Mong đợi 200, thực tế", balance)
```

NGỘ NHẬN THƯỜNG GẶP: "Có GIL rồi nên balance += amount đã an toàn"

GIL chỉ bảo đảm mỗi lúc một thread chạy bytecode; += gồm nhiều bytecode (đọc, cộng, ghi) và thread có thể bị đổi giữa chúng. Trạng thái của ứng dụng phải tự bảo vệ bằng Lock. Lưu ý: Studio chạy thread lần lượt nên không tái hiện được race — bộ chấm kiểm cấu trúc: mọi lần đọc/ghi balance phải nằm trong lúc giữ self.lock.

CÂU DỰ ĐOÁN — đáp án và giải thích

Hai thread cùng chạy deposit(50) trên balance = 100, KHÔNG có lock. Kết quả có thể là?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Luôn 200	Không có gì bảo đảm hai lần đọc-sửa-ghi không xen nhau.
B ĐÚNG	**150 hoặc 200 — tuỳ hai thread xen nhau thế nào**	Đọc-sửa-ghi bị xen giữa thì mất một lần nạp.
C	Luôn 150	Có lúc mỗi thread chạy trọn phần của mình thì ra 200.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Vì sao with lock dễ quản lý hơn acquire/release rời nhau?

A	Cho nhiều thread cùng giữ khoá	Lock luôn chỉ cho một thread giữ.
B	Chạy nhanh hơn	Tốc độ như nhau; khác ở chỗ an toàn khi có lỗi.
C ĐÚNG	**Tự release khi rời khối, kể cả khi có exception**	Không bao giờ để lock kẹt vì quên release.

Giải thích (nguyên văn JSON): Không bao giờ để lock kẹt vì quên release.

2. Thread 1 khoá lock_a rồi lock_b; thread 2 khoá lock_b rồi lock_a. Rủi ro gì?

A ĐÚNG	**Deadlock — mỗi bên giữ một lock và chờ lock kia**	Sửa: cả hai khoá theo cùng thứ tự lock_a rồi lock_b.
B	Không sao vì cuối cùng đều release	Có thể không bên nào tới được lệnh release.
C	Chỉ chậm hơn một chút	Hai bên chờ nhau mãi, không tiến triển.

Giải thích (nguyên văn JSON): Sửa: cả hai khoá theo cùng thứ tự lock_a rồi lock_b.

3. Nên đặt gì vào critical section?

A	Cả hàm, kể cả in và ghi file, cho chắc	Giữ lâu làm các thread khác chờ vô ích.
B ĐÚNG	**Chỉ các bước đọc-sửa-ghi dữ liệu dùng chung — giữ khoá ngắn nhất có thể**	Thread khác phải chờ suốt lúc bạn giữ khoá.
C	Không cần, vì GIL đã khoá	GIL không bảo vệ trạng thái của ứng dụng.

Giải thích (nguyên văn JSON): Thread khác phải chờ suốt lúc bạn giữ khoá.

CÂU PHẢN VÍ DỤ — đáp án

safe_plan(["cache", "log"], ["db", "cache", "log"]) trả về?

Nhãn	Lựa chọn
A	`False`
B	`ValueError`
C ĐÚNG	`True`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao withdraw phải đặt cả phép kiểm tra lẫn phép trừ trong cùng một with self.lock?

Mẫu đạt: Nếu kiểm tra ngoài khoá, hai thread có thể cùng thấy đủ tiền rồi lần lượt trừ, số dư âm. Đặt kiểm tra và trừ trong một lần giữ khoá thì thread kia chỉ kiểm tra sau khi lần trừ trước đã xong. with còn tự nhả khoá khi raise ValueError. Trường hợp đặc biệt: rút đúng bằng số dư thì được, số dư về 0.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	deposit(5) → with self.lock: đọc, cộng, ghi balance
Bước 2	withdraw(9) khi balance = 5 → ValueError, with nhả khoá
Bước 3	run_deposits(Account(0), [10, 20, 30]) → 60
Bước 4	safe_plan(["log", "db"], ["db", "cache", "log"]) → False

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp critical section của deposit, điều kiện của withdraw và phép so vị trí của safe_plan.

Yêu cầu thêm: Thử rút quá số dư, nạp dữ liệu sai kiểu, và một kế hoạch khoá ngược thứ tự; sau mỗi lỗi khoá phải được nhả.

```
class Account:
    def __init__(self, balance=0):
        self.balance = balance
        self.lock = threading.Lock()

    def deposit(self, amount):
        {{slot1}}
            self.balance += amount

    def withdraw(self, amount):
        with self.lock:
            if {{slot2}}:
                raise ValueError("insufficient funds")
            self.balance -= amount


def safe_plan(plan, order):
    positions = [order.index(name) for name in plan]
    for i in range(1, len(positions)):
        if {{slot3}}:
            return False
    return True
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`with self.lock:`
[Slot 2]	`amount > self.balance`
[Slot 3]	`positions[i] <= positions[i - 1]`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 1	`self.lock.acquire()`	Không có release đi kèm: khoá kẹt ngay sau lần gọi đầu.
ô 2	`amount >= self.balance`	Rút đúng bằng số dư là hợp lệ.
ô 1	`with threading.Lock():`	Mỗi lần gọi một Lock MỚI — không thread nào phải chờ ai.
ô 3	`positions[i] < positions[i - 1]`	Cho qua ["db", "db"]: Lock thường không khoá lại được khi đang giữ.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 2, ô 1, ô 3.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

traceTable-0 · BẢNG THEO DÕI BIẾN — Theo dõi biến: điền giá trị sau mỗi dòng

```
balance = 100
read_a = balance
read_b = balance
balance = read_a + 50
balance = read_b + 50
print("Mong đợi 200, thực tế", balance)
```

#	Dòng chạy	balance	read_a	read_b
0	`balance = 100`	100	–	–
1	`read_a = balance`	100	**100**	–
2	`read_b = balance`	100	100	**100**
3	`balance = read_a + 50`	**150**	100	100

Ô IN XANH ĐẬM là ô học viên phải điền (3 ô); ô xám là giá trị bài cho sẵn. So sánh bỏ nháy và không phân biệt hoa/thường.

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`with self.lock:`
ô 2	`amount > self.balance`
ô 3	`positions[i] <= positions[i - 1]`

Bẫy ở ô 1 — `self.lock.acquire()`: Không có release đi kèm: khoá kẹt ngay sau lần gọi đầu.
Bẫy ở ô 2 — `amount >= self.balance`: Rút đúng bằng số dư là hợp lệ.
Bẫy ở ô 1 — `with threading.Lock()`:: Mỗi lần gọi một Lock MỚI — không thread nào phải chờ ai.
Bẫy ở ô 3 — `positions[i] < positions[i - 1]`: Cho qua ["db", "db"]: Lock thường không khoá lại được khi đang giữ.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 10 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 3).
Lỗi: tạo Lock mới mỗi lần gọi
Sửa thành:

```
        with self.lock:
```

```
     3 |
     4 | class Account:
     5 |     def __init__(self, balance=0):
     6 |         self.balance = balance
     7 |         self.lock = threading.Lock()
     8 |
     9 |     def deposit(self, amount):
>>  10 |         with threading.Lock():
    11 |             self.balance += amount
    12 |
    13 |     def withdraw(self, amount):
    14 |         with self.lock:
    15 |             if amount > self.balance:
    16 |                 raise ValueError("insufficient funds")
    17 |             self.balance -= amount
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: acquire/release rời nhau: lỗi thì khoá kẹt
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
14	`        self.lock.acquire()`	`        with self.lock:`
15	`        if amount > self.balance:`	`            if amount > self.balance:`
16	`            raise ValueError("insufficient funds")`	`                raise ValueError("insufficient funds")`
17	`        self.balance -= amount`	`            self.balance -= amount`
—	`        self.lock.release()`	(xoá dòng)

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao withdraw phải đặt cả phép kiểm tra lẫn phép trừ trong cùng một with self.lock?

Câu hỏi của màn: Vì sao withdraw phải đặt cả phép kiểm tra lẫn phép trừ trong cùng một with self.lock?
Lời giải thích mẫu: Nếu kiểm tra ngoài khoá, hai thread có thể cùng thấy đủ tiền rồi lần lượt trừ, số dư âm. Đặt kiểm tra và trừ trong một lần giữ khoá thì thread kia chỉ kiểm tra sau khi lần trừ trước đã xong. with còn tự nhả khoá khi raise ValueError. Trường hợp đặc biệt: rút đúng bằng số dư thì được, số dư về 0.
Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 5 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — tạo Lock mới mỗi lần gọi

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
10	`        with self.lock:`	`        with threading.Lock():`

KIỂU SAI 2 — acquire/release rời nhau: lỗi thì khoá kẹt

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
14	`        with self.lock:`	`        self.lock.acquire()`
15	`            if amount > self.balance:`	`        if amount > self.balance:`
16	`                raise ValueError("insufficient funds")`	`            raise ValueError("insufficient funds")`
17	`            self.balance -= amount`	`        self.balance -= amount`
—	(không có dòng)	`        self.lock.release()`

KIỂU SAI 3 — không khoá — tin GIL làm += an toàn

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
10	`        with self.lock:`	`        self.balance += amount`
11	`            self.balance += amount`	(xoá dòng)

KIỂU SAI 4 — kiểm tra số dư ngoài khoá

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
—	(không có dòng)	`        if amount > self.balance:`
—	(không có dòng)	`            raise ValueError("insufficient funds")`
15	`            if amount > self.balance:`	(xoá dòng)
16	`                raise ValueError("insufficient funds")`	(xoá dòng)

KIỂU SAI 5 — cho khoá lại lock đang giữ

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
32	`        if positions[i] <= positions[i - 1]:`	`        if positions[i] < positions[i - 1]:`

Nguồn trang này: pe-v4 · 012-T05.json · 012-T05.json · qa.json[T05]

T06 · Queue và tác vụ lưu nền

Mã bài T06 · Nâng cao · M2 · Đa luồng · 40 phút · 20 XP · thứ tự 13 · mức N

MỤC TIÊU BÀI

Giao việc lưu cho worker nền qua queue.Queue.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hoàn thành class BackgroundSaver (self.jobs = queue.Queue(), self.lines = [] đã có). submit(record): put BẢN SAO record (dict(record)) vào self.jobs — chưa lưu gì. _work(): lấy từng job bằng self.jobs.get_nowait() tới khi gặp queue.Empty thì return; mỗi job thêm dòng "name,mark" vào self.lines rồi gọi self.jobs.task_done(). flush(): tạo Thread MỚI với target=self._work, start, join, trả len(self.lines).

Thực hành: submit chỉ put bản sao; mọi việc lưu nằm trong _work chạy ở worker. Lấy bằng get_nowait() trong try/except queue.Empty; mỗi job xong gọi task_done(). flush tạo Thread mới mỗi lần.

Gọi (đề cho)	Kết quả mong đợi
`submit An 8, Binh 7; flush()`	`2; lines = ["An,8", "Binh,7"]`
`flush() khi chưa submit`	`0`
`sửa record sau submit rồi flush`	`dòng đã lưu vẫn là giá trị lúc submit`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`BackgroundSaver.submit(self, record)`	Giao một bản ghi cho worker nền. · record: dict có "name", "mark" · None · put BẢN SAO vào self.jobs; chưa lưu gì — việc lưu là của worker.
`BackgroundSaver.flush(self)`	Chạy worker nền xả hết việc trong queue. · self · int: tổng số dòng trong self.lines · Tạo Thread MỚI (target=self._work), start, join. Worker lấy bằng get_nowait() tới queue.Empty, mỗi job thêm "name,mark" vào self.lines rồi task_done().

LỜI GIẢI MẪU (qa.json · T06)

```
import queue
import threading


class BackgroundSaver:
    def __init__(self):
        self.jobs = queue.Queue()
        self.lines = []

    def submit(self, record):
        self.jobs.put(dict(record))

    def _work(self):
        while True:
            try:
                record = self.jobs.get_nowait()
            except queue.Empty:
                return
            self.lines.append(f"{record['name']},{record['mark']}")
            self.jobs.task_done()

    def flush(self):
        worker = threading.Thread(target=self._work)
        worker.start()
        worker.join()
        return len(self.lines)


if __name__ == "__main__":
    saver = BackgroundSaver()
    saver.submit({"name": "An", "mark": 8})
    saver.submit({"name": "Binh", "mark": 7})
    print(saver.flush(), saver.lines)
```

BỘ CHẤM — 8 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Lưu đúng thứ tự FIFO	`(lambda s: (s.submit({"name": 'Lan', "mark": 9}), s.submit({"name": 'Minh', "mark": 6}), s.submit({"name": 'Nga', "mark": 10}), s.flush(), s.lines,)[-1])(BackgroundSaver())` → `['Lan,9', 'Minh,6', 'Nga,10']`
2	flush trả số dòng đã lưu	`(lambda s: (s.submit({"name": 'Lan', "mark": 9}), s.submit({"name": 'Minh', "mark": 6}), s.submit({"name": 'Nga', "mark": 10}), s.flush(),)[-1])(BackgroundSaver())` → `3`

3	Chưa flush thì chưa lưu (việc nằm ở worker)	`(lambda s: (s.submit({"name": 'Lan', "mark": 9}), s.submit({"name": 'Minh', "mark": 6}), (s.lines, s.jobs.qsize()),)[-1])(BackgroundSaver())` → `([], 2)`
4	Sửa record sau submit không đổi dữ liệu đã giao	`(lambda s, r: (s.submit(r), r.update(mark=0), s.flush(), s.lines)[-1])(BackgroundSaver(), {"name": "Oanh", "mark": 8})` → `['Oanh,8']`
5	Chưa submit gì → flush trả 0	`(lambda s: (s.flush(), (s.lines, s.jobs.empty()),)[-1])(BackgroundSaver())` → `([], True)`
6	Mỗi get có đúng một task_done (gọi thêm → ValueError)	`(lambda s: (s.submit({"name": 'Lan', "mark": 9}), s.submit({"name": 'Minh', "mark": 6}), s.flush(), _studio.call(s.jobs.task_done).error,)[-1])(BackgroundSaver())` → `'ValueError'`
7	flush dùng một worker thread: start, join, target là method	`(lambda r: (r.thread_count, r.all_joined, r.targets_callable, r.value))((lambda s: (s.submit({"name": 'Phu', "mark": 5}), _studio.call(s.flush),)[-1])(BackgroundSaver()))` → `(1, True, True, 1)`
8	flush hai lần: mỗi lần một Thread mới	`(lambda s: (s.submit({"name": 'An', "mark": 1}), s.flush(), s.submit({"name": 'Binh', "mark": 2}), s.flush(), s.lines,)[-1])(BackgroundSaver())` → `['An,1', 'Binh,2']`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. submit chỉ put bản sao; mọi việc lưu nằm trong _work chạy ở worker. Lấy bằng get_nowait() trong try/except queue.Empty; mỗi job xong gọi task_done(). flush tạo Thread mới mỗi lần.

Cấu trúc. 1. submit: self.jobs.put(dict(record)). 2. _work: while True → try get_nowait() / except queue.Empty: return → append f"{name},{mark}" → task_done(). 3. flush: worker = threading.Thread(target=self._work); start; join; return len(self.lines).

Lời giải từng phần. def _work(self): while True: try: record = self.jobs.get_nowait() except queue.Empty: return ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — threading, đồng hồ (time.sleep) · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy threading, đồng hồ (time.sleep) thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O.

• Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo.

• Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần.

• Vì worker chạy ngay lúc start(), bài này cho nơi gọi submit HẾT việc rồi mới flush, và worker xả queue bằng get_nowait() — không bao giờ phải chờ.

• Trên máy thật, worker chạy suốt thường dùng get() + sentinel None như ví dụ minh hoạ.

NGỘ NHẬN THƯỜNG GẶP: "q.get() khi queue rỗng sẽ trả None"

get() mặc định CHỜ tới khi có phần tử — với queue không ai put thêm, chương trình treo. Muốn không chờ thì dùng get_nowait() và bắt queue.Empty; muốn worker dừng thì gửi một sentinel (None) có chủ đích.

CÂU DỰ ĐOÁN — đáp án và giải thích

Đoạn code sau in gì?

```
import queue

q = queue.Queue()
q.put("a")
q.put("b")
print(q.get(), q.get())
```

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A ĐÚNG	**a b — vào trước ra trước (FIFO)**	Queue lấy phần tử ở đầu hàng đợi.
B	b a	Đó là LifoQueue (ngăn xếp), không phải Queue.
C	a a	get() lấy phần tử RA khỏi queue.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. q.get() khi queue rỗng có mặc định trả None ngay không?

A	Có, raise queue.Empty	Đó là get_nowait() (hoặc get(block=False)).
B ĐÚNG	**Không; mặc định nó chờ**	Chờ tới khi có thread khác put.
C	Có	Đó là hiểu nhầm phổ biến — None chỉ có khi ai đó put None.

Giải thích (nguyên văn JSON): Chờ tới khi có thread khác put.

2. Worker get() 3 việc nhưng chỉ gọi task_done() 2 lần. jobs.join() sẽ...

A	Raise ValueError	ValueError chỉ khi task_done nhiều hơn số việc.
B ĐÚNG	**Chờ mãi — vẫn còn một việc chưa báo xong**	Mỗi get() cần đúng một task_done().
C	Trả về ngay	join chờ tới khi mọi việc đã task_done.

Giải thích (nguyên văn JSON): Mỗi get() cần đúng một task_done().

3. Vì sao submit đưa dict(record) chứ không đưa record?

A	Queue không nhận dict	Queue nhận mọi object.
B	Để chạy nhanh hơn	Sao chép còn tốn thêm một chút.
C ĐÚNG	**Nơi gọi có thể sửa record trước khi worker lưu; bản sao giữ đúng dữ liệu lúc giao**	Queue chỉ giữ tham chiếu tới object.

Giải thích (nguyên văn JSON): Queue chỉ giữ tham chiếu tới object.

CÂU PHẢN VÍ DỤ — đáp án

Submit 2 record rồi flush; submit thêm 1 record rồi flush lần nữa. flush lần hai trả?

Nhãn	Lựa chọn
A	`RuntimeError`
B ĐÚNG	`3`
C	`1`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao _work dùng get_nowait() và dừng ở queue.Empty, còn worker chạy suốt lại cần get() và sentinel?

Mẫu đạt: flush chỉ start worker khi mọi job đã được submit, nên queue rỗng nghĩa là hết việc: get_nowait() raise queue.Empty và worker kết thúc, không bao giờ phải chờ. Worker chạy suốt thì queue rỗng chỉ là "tạm chưa có việc", nên dùng get() để chờ và một sentinel None để biết khi nào dừng. Trường hợp đặc biệt: flush khi chưa submit gì → Empty ngay, trả 0.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	submit(record) → put bản sao vào self.jobs
Bước 2	flush() → Thread(target=self._work).start()
Bước 3	get_nowait() → lấy theo FIFO, lưu, task_done()
Bước 4	queue.Empty → worker kết thúc, join xong

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Lắp bản sao khi submit, cách lấy việc không chờ, lời báo xong việc và Thread của flush.

Yêu cầu thêm: Thử flush khi chưa submit gì, flush hai lần liên tiếp, và sửa record sau khi submit.

```
    def submit(self, record):
        self.jobs.put({{slot1}})

    def _work(self):
        while True:
            try:
                record = {{slot2}}
            except queue.Empty:
                return
            self.lines.append(f"{record['name']},{record['mark']}")
            {{slot3}}

    def flush(self):
        worker = {{slot4}}
        worker.start()
        worker.join()
        return len(self.lines)
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`dict(record)`
[Slot 2]	`self.jobs.get_nowait()`
[Slot 3]	`self.jobs.task_done()`
[Slot 4]	`threading.Thread(target=self._work)`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 3	`self.jobs.join()`	join chờ các task_done — gọi trong worker là tự chờ chính mình.
ô 2	`self.jobs.get()`	Queue rỗng thì get() chờ mãi, không bao giờ tới queue.Empty.
ô 1	`record`	Giao chính object của nơi gọi — sửa sau đó là đổi luôn dữ liệu sắp lưu.
ô 4	`threading.Thread(target=self._work())`	Gọi _work ngay ở thread chính.

Ngân hàng còn 4 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 4, ô 3, ô 2, ô 1.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [0, 1, 3, 2].

Bên trái	NỐI VỚI (đáp án)
submit(record)	**put bản sao vào self.jobs**
flush()	**Thread(target=self._work).start()**
get_nowait()	**lấy theo FIFO, lưu, task_done()**
queue.Empty	**worker kết thúc, join xong**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`dict(record)`
ô 2	`self.jobs.get_nowait()`
ô 3	`self.jobs.task_done()`
ô 4	`threading.Thread(target=self._work)`

Bẫy ở ô 3 — `self.jobs.join()`: join chờ các task_done — gọi trong worker là tự chờ chính mình.
Bẫy ở ô 2 — `self.jobs.get()`: Queue rỗng thì get() chờ mãi, không bao giờ tới queue.Empty.
Bẫy ở ô 1 — `record`: Giao chính object của nơi gọi — sửa sau đó là đổi luôn dữ liệu sắp lưu.
Bẫy ở ô 4 — `threading.Thread(target=self._work())`: Gọi _work ngay ở thread chính.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 11 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 4).
Lỗi: đưa thẳng record, không sao chép
Sửa thành:

```
        self.jobs.put(dict(record))
```

```
     4 |
     5 | class BackgroundSaver:
     6 |     def __init__(self):
     7 |         self.jobs = queue.Queue()
     8 |         self.lines = []
     9 |
    10 |     def submit(self, record):
>>  11 |         self.jobs.put(record)
    12 |
    13 |     def _work(self):
    14 |         while True:
    15 |             try:
    16 |                 record = self.jobs.get_nowait()
    17 |             except queue.Empty:
    18 |                 return
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: quên task_done()
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
20	(không có dòng)	`self.jobs.task_done()`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao _work dùng get_nowait() và dừng ở queue.Empty, còn worker chạy suốt lại cần get() và sentinel?

Câu hỏi của màn: Vì sao _work dùng get_nowait() và dừng ở queue.Empty, còn worker chạy suốt lại cần get() và sentinel?
Lời giải thích mẫu: flush chỉ start worker khi mọi job đã được submit, nên queue rỗng nghĩa là hết việc: get_nowait() raise queue.Empty và worker kết thúc, không bao giờ phải chờ. Worker chạy suốt thì queue rỗng chỉ là "tạm chưa có việc", nên dùng get() để chờ và một sentinel None để biết khi nào dừng. Trường hợp đặc biệt: flush khi chưa submit gì → Empty ngay, trả 0.
Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 4 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — đưa thẳng record, không sao chép

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
11	`        self.jobs.put(dict(record))`	`        self.jobs.put(record)`

KIỂU SAI 2 — quên task_done()

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
20	`            self.jobs.task_done()`	(xoá dòng)

KIỂU SAI 3 — tạo Thread một lần trong __init__

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
—	(không có dòng)	`        self.worker = threading.Thread(target=self._work)`
23	`        worker = threading.Thread(target=self._work)`	`        self.worker.start()`
24	`        worker.start()`	`        self.worker.join()`
25	`        worker.join()`	(xoá dòng)

KIỂU SAI 4 — lưu ngay trong submit, không qua worker

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
11	`        self.jobs.put(dict(record))`	`        self.lines.append(f"{record['name']},{record['mar » k']}")`

Nguồn trang này: pe-v4 · 013-T06.json · 013-T06.json · qa.json[T06]

M3 · GUI Tkinter và dự án

7 bài trong phạm vi: G01, G02, G03, G04, G05, G06, R02. Chấm trên bản mô phỏng: G01, G02, G03, G04, G05, G06, R02.

G01 · GUI, CLI và cửa sổ

Mã bài G01 · Nâng cao · M3 · GUI Tkinter và dự án · 25 phút · 20 XP · thứ tự 14 · mức N

MỤC TIÊU BÀI

Tạo cửa sổ chính, cửa sổ phụ và chạy vòng lặp sự kiện.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Cho app_title = "Student system" và size = "800x600". Tạo cửa sổ chính root = tk.Tk(), đặt tiêu đề app_title và kích thước size; tạo cửa sổ phụ details bằng tk.Toplevel(root) với tiêu đề "Details", kích thước "400x300"; dòng cuối gọi root.mainloop().

Thực hành: title và geometry là method nên phải có ngoặc và đối số; cửa sổ phụ là tk.Toplevel(root); root.mainloop() ở dòng cuối.

Gọi (đề cho)	Kết quả mong đợi
`app_title = "Student system", size = "800x600"`	`Cửa sổ chính 800x600 tên Student system; cửa sổ phụ » Details 400x300`
`app_title = "Quản lý điểm", size = "640x480"`	`root.title() trả "Quản lý điểm", kích thước 640x480`

JSON của bài không khai step_4.contracts (hợp đồng hàm) — chỗ này KHÔNG được suy diễn thêm.

LỜI GIẢI MẪU (qa.json · G01)

```
import tkinter as tk

app_title = "Student system"
size = "800x600"

root = tk.Tk()
root.title(app_title)
root.geometry(size)

details = tk.Toplevel(root)
details.title("Details")
details.geometry("400x300")

root.mainloop()
```

BỘ CHẤM — 7 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Tiêu đề cửa sổ chính	`root.title()` `→ 'Student system'`
2	Kích thước 800x600	`root.geometry().split('+')[0]` `→ '800x600'`
3	Cửa sổ phụ là Toplevel, không phải Tk thứ hai	`type(details).__name__` `→ 'Toplevel'`
4	Cửa sổ phụ thuộc root	`details.master is root` `→ True`
5	Cửa sổ phụ: Details, 400x300	`(details.title(), details.geometry().split('+')[0])` `→ ('Details', '400x300')`
6	Đã gọi mainloop()	`_studio.tk.mainloop_called` `→ True`
7	Dữ liệu khác: "Quản lý điểm", 640x480	`(lambda w: (w.title(), w.geometry().split('+')[0]))(_studio.run(vars={'app_ » title': 'Quản lý điểm', 'size': '640x480'}).ns['root'])` `→ ('Quản lý điểm', '640x480')`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. title và geometry là method nên phải có ngoặc và đối số; cửa sổ phụ là tk.Toplevel(root); root.mainloop() ở dòng cuối.

Cấu trúc. 1. root = tk.Tk(); root.title(app_title); root.geometry(size) 2. details = tk.Toplevel(root); details.title("Details"); details.geometry("400x300") 3. Dòng cuối: root.mainloop()

Lời giải từng phần. root = tk.Tk() root.title(app_title) ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — tkinter · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy tkinter thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar…), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ.

- Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn.
- Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

NGỘ NHẬN THƯỜNG GẶP: "Muốn thêm cửa sổ thì gọi tk.Tk() lần nữa"

Mỗi tk.Tk() khởi động một bộ thông dịch Tcl riêng: StringVar hay ảnh (PhotoImage) gắn với cửa sổ này không dùng được ở cửa sổ kia, lỗi rất khó hiểu. Cửa sổ phụ dùng tk.Toplevel(root). Cũng đừng viết root.title = "..." — đó là GHI ĐÈ method bằng một chuỗi: tiêu đề không đổi, còn root.title() sau đó nổ TypeError.

CÂU DỰ ĐOÁN — đáp án và giải thích

Trên máy thật, dòng print đặt SAU root.mainloop() chạy khi nào?

```
root = tk.Tk()
root.mainloop()
print("Bye")
```

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Ngay lập tức, cùng lúc cửa sổ hiện ra	Chỉ Studio mô phỏng mới cho mainloop trả về ngay.
B	Không bao giờ chạy	Đóng cửa sổ thì mainloop trả về và code phía sau chạy tiếp.
C ĐÚNG	**Khi người dùng đóng cửa sổ chính — mainloop chặn tới lúc đó**	Event loop chạy bên trong mainloop().

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Toplevel dùng để tạo gì?

A ĐÚNG	**Một cửa sổ phụ thuộc ứng dụng Tk hiện tại**	Đóng cửa sổ chính thì Toplevel đóng theo.
B	Cửa sổ chính thứ hai, độc lập hoàn toàn	Cửa sổ chính là tk.Tk(); Toplevel phụ thuộc root.
C	Một khung nằm bên trong cửa sổ	Đó là Frame; Toplevel là một cửa sổ riêng.

Giải thích (nguyên văn JSON): Đóng cửa sổ chính thì Toplevel đóng theo.

2. Khác biệt cốt lõi giữa chương trình CLI và GUI là gì?

A ĐÚNG	**GUI chờ sự kiện trong event loop rồi gọi hàm xử lý; CLI chạy lệnh theo thứ tự**	Vì thế GUI cần mainloop().
B	CLI không nhận được dữ liệu từ người dùng	CLI nhận qua input() hoặc đối số dòng lệnh.
C	GUI luôn chạy nhanh hơn CLI	Tốc độ không phải khác biệt cốt lõi; GUI thường tốn bộ nhớ hơn.

Giải thích (nguyên văn JSON): Vì thế GUI cần mainloop().

3. Viết root.title = "Student system" (không ngoặc) thì sao?

A	Tiêu đề đổi như root.title("Student system")	Không có method nào được gọi.
B	SyntaxError ngay khi chạy	Cú pháp gán hợp lệ — lỗi chỉ lộ ra lúc dùng.
C ĐÚNG	**Tiêu đề không đổi; root.title giờ là chuỗi nên root.title() nổ TypeError**	Đó là phép gán, không gọi method.

Giải thích (nguyên văn JSON): Đó là phép gán, không gọi method.

CÂU PHẢN VÍ DỤ — đáp án

Với app_title = "Quản lý điểm", root.title() trả gì?

Nhãn	Lựa chọn
A ĐÚNG	`Quản lý điểm`
B	`tk`
C	`Student system`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao root.mainloop() phải nằm ở cuối chương trình?

Mẫu đạt: mainloop() chặn tại chỗ để nhận và xử lý sự kiện cho tới khi cửa sổ chính đóng. Code đặt sau nó (tạo widget, cửa sổ phụ) chỉ chạy khi người dùng đã đóng cửa sổ — quá muộn. Vì vậy tạo xong mọi cửa sổ, widget rồi mới gọi mainloop(). Trường hợp đặc biệt: quên mainloop thì chạy bằng file, cửa sổ hiện rồi tắt ngay.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	tk.Tk() → tạo cửa sổ chính root

Bước 2	root.title(app_title) → đặt chữ trên thanh tiêu đề
Bước 3	tk.Toplevel(root) → tạo cửa sổ phụ thuộc root
Bước 4	root.mainloop() → chờ sự kiện tới khi cửa sổ đóng

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Dữ liệu mới: "Quản lý thư viện", 640x480. Lắp bốn lời gọi.

Yêu cầu thêm: Đổi app_title và size (vd. "Quản lý điểm", "640x480"); bộ chấm chạy lại chương trình rồi đọc tiêu đề và kích thước của cửa sổ mới.

```
import tkinter as tk

app_title = "Quản lý thư viện"
size = "640x480"

root = {{slot1}}
root.title(app_title)
{{slot2}}
details = {{slot3}}
details.title("Details")
{{slot4}}
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`tk.Tk()`
[Slot 2]	`root.geometry(size)`
[Slot 3]	`tk.Toplevel(root)`
[Slot 4]	`root.mainloop()`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 2	`root.geometry("640*480")`	Kích thước dùng chữ x: "640x480" — dấu * làm Tk báo TclError.
ô 2	`root.geometry = size`	Ghi đè method bằng chuỗi; kích thước không đổi.
ô 1	`tk.Tk`	Thiếu ngoặc: root là class Tk chứ chưa phải cửa sổ.
ô 4	`root.mainloop`	Thiếu ngoặc: chỉ nhắc tên method, vòng lặp sự kiện không chạy.

Ngân hàng còn 4 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 3, ô 2, ô 4, ô 1.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [2, 3, 0, 1].

Bên trái	NỐI VỚI (đáp án)
tk.Tk()	**tạo cửa sổ chính root**
root.title(app_title)	**đặt chữ trên thanh tiêu đề**
tk.Toplevel(root)	**tạo cửa sổ phụ thuộc root**
root.mainloop()	**chờ sự kiện tới khi cửa sổ đóng**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`tk.Tk()`
ô 2	`root.geometry(size)`
ô 3	`tk.Toplevel(root)`
ô 4	`root.mainloop()`

Bẫy ở ô 2 — `root.geometry("640*480")`: Kích thước dùng chữ x: "640x480" — dấu * làm Tk báo TclError.
Bẫy ở ô 2 — `root.geometry = size`: Ghi đè method bằng chuỗi; kích thước không đổi.
Bẫy ở ô 1 — `tk.Tk`: Thiếu ngoặc: root là class Tk chứ chưa phải cửa sổ.
Bẫy ở ô 4 — `root.mainloop`: Thiếu ngoặc: chỉ nhắc tên method, vòng lặp sự kiện không chạy.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 7 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 1).
Lỗi: gán title thay vì gọi
Sửa thành:

```
root.title(app_title)
```

```
    1 | import tkinter as tk
    2 |
    3 | app_title = "Student system"
    4 | size = "800x600"
    5 |
    6 | root = tk.Tk()
>>  7 | root.title = app_title
```

```
 8 | root.geometry(size)
 9 |
10 | details = tk.Toplevel(root)
11 | details.title("Details")
12 | details.geometry("400x300")
13 |
14 | root.mainloop()
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: cửa sổ phụ bằng tk.Tk()

Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
10	`details = tk.Tk()`	`details = tk.Toplevel(root)`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao root.mainloop() phải nằm ở cuối chương trình?

Câu hỏi của màn: Vì sao root.mainloop() phải nằm ở cuối chương trình?

Lời giải thích mẫu: mainloop() chặn tại chỗ để nhận và xử lý sự kiện cho tới khi cửa sổ chính đóng. Code đặt sau nó (tạo widget, cửa sổ phụ) chỉ chạy khi người dùng đã đóng cửa sổ — quá muộn. Vì vậy tạo xong mọi cửa sổ, widget rồi mới gọi mainloop(). Trường hợp đặc biệt: quên mainloop thì chạy bằng file, cửa sổ hiện rồi tắt ngay.

Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — gán title thay vì gọi

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
7	`root.title(app_title)`	`root.title = app_title`

KIỂU SAI 2 — cửa sổ phụ bằng tk.Tk()

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
10	`details = tk.Toplevel(root)`	`details = tk.Tk()`

KIỂU SAI 3 — quên mainloop()

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
14	`root.mainloop()`	(xoá dòng)
15		(xoá dòng)

Nguồn trang này: pe-v4 · 014-G01.json · 014-G01.json · qa.json[G01]

G02 · Widget và callback

Mã bài G02 · Nâng cao · M3 · GUI Tkinter và dự án · 30 phút · 20 XP · thứ tự 15 · mức N

MỤC TIÊU BÀI

Nối nút với callback và đọc Entry đúng lúc bấm.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hoàn thành form đăng nhập (đã có root, frame, user_entry): tạo password_entry che ký tự bằng show="*", Label status với chữ "Chưa đăng nhập", và Button chữ "Đăng nhập" với command=login. Hàm login() đọc hai Entry LÚC ĐƯỢC BẤM: tên bỏ khoảng trắng hai đầu; mật khẩu bằng PASSWORD thì status.config(text="Xin chào <tên>") và messagebox.showinfo("Đăng nhập", "Chào mừng <tên>!"); sai thì status.config(text="Sai mật khẩu").

Thực hành: login() gọi user_entry.get() và password_entry.get() ngay trong hàm, so với PASSWORD rồi đổi chữ status bằng config(text=...). Nút dùng command=login (không ngoặc).

Gọi (đề cho)	Kết quả mong đợi
`Tên "Mai", mật khẩu "usth2026"`	`status: Xin chào Mai; hộp thông báo "Chào mừng Mai!"`
`Mật khẩu sai hoặc rỗng`	`status: Sai mật khẩu; không bật hộp thông báo`

JSON của bài không khai step_4.contracts (hợp đồng hàm) — chỗ này KHÔNG được suy diễn thêm.

LỜI GIẢI MẪU (qa.json · G02)

```
import tkinter as tk
from tkinter import messagebox

PASSWORD = "usth2026"

root = tk.Tk()
root.title("Đăng nhập")
frame = tk.Frame(root)
frame.pack()

tk.Label(frame, text="Tên").pack()
user_entry = tk.Entry(frame, width=24)
user_entry.pack()

tk.Label(frame, text="Mật khẩu").pack()
password_entry = tk.Entry(frame, width=24, show="*")
password_entry.pack()

status = tk.Label(frame, text="Chưa đăng nhập")
status.pack()


def login():
    user = user_entry.get().strip()
    if password_entry.get() == PASSWORD:
        status.config(text=f"Xin chào {user}")
        messagebox.showinfo("Đăng nhập", f"Chào mừng {user}!")
    else:
        status.config(text="Sai mật khẩu")


tk.Button(frame, text="Đăng nhập", command=login).pack()

root.mainloop()
```

BỘ CHẤM — 9 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Chưa bấm: status "Chưa đăng nhập"	`status.cget('text')` `→ 'Chưa đăng nhập'`
2	Có nút "Đăng nhập"	`_studio.tk.find(type='Button', text='Đăng nhập') is not None` `→ True`
3	Ô mật khẩu che bằng *	`password_entry.cget('show')` `→ '*'`

4	Mai + mật khẩu đúng	`(user_entry.delete(0, 'end'), user_entry.insert(0, 'Mai'), »`
		`password_entry.delete(0, 'end'), password_entry.insert(0, 'usth2026'), »`
		`_studio.tk.find(type='Button', text='Đăng nhập').invoke(), »`
		`status.cget('text'))[-1]`
		`→ 'Xin chào Mai'`

5	Mật khẩu sai	`(user_entry.delete(0, 'end'), user_entry.insert(0, 'Mai'), »`
		`password_entry.delete(0, 'end'), password_entry.insert(0, 'usth'), »`
		`_studio.tk.find(type='Button', text='Đăng nhập').invoke(), »`
		`status.cget('text'))[-1]`
		`→ 'Sai mật khẩu'`

6	Tên khác có khoảng trắng " An "	`(user_entry.delete(0, 'end'), user_entry.insert(0, ' An '), » password_entry.delete(0, 'end'), password_entry.insert(0, 'usth2026'), » _studio.tk.find(type='Button', text='Đăng nhập').invoke(), » status.cget('text'))[-1]` → `'Xin chào An'`
7	Mật khẩu rỗng	`(user_entry.delete(0, 'end'), user_entry.insert(0, 'Mai'), » password_entry.delete(0, 'end'), password_entry.insert(0, ''), » _studio.tk.find(type='Button', text='Đăng nhập').invoke(), » status.cget('text'))[-1]` → `'Sai mật khẩu'`
8	Đúng mật khẩu → showinfo("Đăng nhập", "Chào mừng Lan!")	`((user_entry.delete(0, 'end'), user_entry.insert(0, 'Lan'), » password_entry.delete(0, 'end'), password_entry.insert(0, 'usth2026'), » _studio.tk.find(type='Button', text='Đăng nhập').invoke(), » status.cget('text'))[-1], (lambda m: ('info' in str(m[0]), m[1], » m[2]))(tuple(_studio.tk.messages[-1])))[1]` → `(True, 'Đăng nhập', 'Chào mừng Lan!')`
9	Sai mật khẩu → không bật hộp thông báo	`(lambda n: ((user_entry.delete(0, 'end'), user_entry.insert(0, 'Lan'), » password_entry.delete(0, 'end'), password_entry.insert(0, 'sai'), » _studio.tk.find(type='Button', text='Đăng nhập').invoke(), » status.cget('text'))[-1], len(_studio.tk.messages) - » n)[1])(len(_studio.tk.messages))` → `0`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. login() gọi user_entry.get() và password_entry.get() ngay trong hàm, so với PASSWORD rồi đổi chữ status bằng config(text=...). Nút dùng command=login (không ngoặc).

Cấu trúc. 1. password_entry = tk.Entry(frame, width=24, show="*"); password_entry.pack() 2. status = tk.Label(frame, text="Chưa đăng nhập") rồi status.pack() ở dòng riêng. 3. def login(): user = user_entry.get().strip(); đúng mật khẩu → status "Xin chào ..." + showinfo; sai → "Sai mật khẩu". 4. tk.Button(frame, text="Đăng nhập", command=login).pack()

Lời giải từng phần. def login(): user = user_entry.get().strip() if password_entry.get() == PASSWORD: ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — tkinter · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy tkinter thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar...), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ.
- Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn.
- Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

NGỘ NHẬN THƯỜNG GẶP: "status = tk.Label(...).pack() cho gọn"

pack() xếp widget rồi trả về None, nên status thành None và status.config(...) trong callback nổ AttributeError — đúng lúc người dùng bấm nút, không phải lúc chạy chương trình. Giữ widget vào biến trước, pack ở dòng sau. Tương tự, command nhận TÊN hàm: command=login, không phải login().

CÂU DỰ ĐOÁN — đáp án và giải thích

Bấm nút này thì chuyện gì xảy ra?

```
button = tk.Button(root, text="Open", command=clicked())
```

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	clicked chạy mỗi lần bấm như bình thường	Đó là command=clicked (không ngoặc).
B	Báo lỗi mỗi lần bấm	command rỗng thì bấm không làm gì, cũng không báo lỗi.
C ĐÚNG	**Không có gì: clicked() đã chạy một lần lúc tạo nút, command nhận None**	Dấu ngoặc là lời gọi ngay.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Ở command=clicked có dùng dấu ngoặc gọi hàm không?

A ĐÚNG	**Không — truyền tên hàm để Tkinter gọi khi bấm**	Có ngoặc là gọi ngay lúc tạo nút.
B	Có — phải viết command=clicked()	Khi đó clicked chạy một lần lúc dựng giao diện.
C	Tuỳ phiên bản Python	Quy tắc callback không đổi theo phiên bản.

Giải thích (nguyên văn JSON): Có ngoặc là gọi ngay lúc tạo nút.

2. Entry mật khẩu cần tuỳ chọn nào để hiện dấu * thay cho ký tự?

A ĐÚNG	**show="*"**	Entry vẫn giữ chữ thật; get() trả đúng mật khẩu đã gõ.
B	text="*"	Entry không dùng text để che chữ.

C	password=True	Tkinter không có tuỳ chọn này.

Giải thích (nguyên văn JSON): Entry vẫn giữ chữ thật; get() trả đúng mật khẩu đã gõ.

3. entry.bind("<Return>", handler) khác command=handler ở điểm nào?

A ĐÚNG	**bind gọi handler(event) — handler cần một tham số nhận event**	command gọi handler() không đối số.
B	bind chỉ dùng được cho Button	Mọi widget đều bind được sự kiện.
C	Không khác gì	bind luôn truyền đối tượng event vào hàm.

Giải thích (nguyên văn JSON): command gọi handler() không đối số.

CÂU PHẢN VÍ DỤ — đáp án

Gõ tên "An", mật khẩu "abc" rồi bấm Đăng nhập — status hiện gì?

Nhãn	Lựa chọn
A	`Chưa đăng nhập`
B	`Xin chào An`
C ĐÚNG	`Sai mật khẩu`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao login() phải gọi user_entry.get() BÊN TRONG hàm, không đọc sẵn ở ngoài?

Mẫu đạt: Dòng ngoài hàm chạy một lần lúc dựng giao diện, khi Entry còn rỗng, nên giá trị đọc được là "". Callback chạy mỗi lần bấm nút, lúc người dùng đã gõ xong, nên phải gọi get() trong callback để lấy chữ hiện tại. Trường hợp đặc biệt: mật khẩu rỗng vẫn phải ra "Sai mật khẩu".

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	tk.Entry(frame, show="*") → ô mật khẩu che ký tự
Bước 2	command=login → nút giữ hàm, chưa gọi
Bước 3	Bấm nút → Tkinter gọi login()
Bước 4	login() → get() hai Entry rồi status.config(text=...)

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Dữ liệu mới: nút "Gửi" chào theo tên trong Entry. Lắp ba chỗ.

Yêu cầu thêm: Bộ chấm gõ tên và mật khẩu khác (đúng, sai, rỗng) vào hai Entry, bấm nút bằng invoke() rồi đọc status.cget("text") và hộp thông báo.

```
import tkinter as tk

root = tk.Tk()
name_entry = tk.Entry(root)
name_entry.pack()
greeting = {{slot1}}
greeting.pack()


def send():
    name = {{slot2}}
    greeting.config(text="Hello " + name)


tk.Button(root, text="Gửi", command={{slot3}}).pack()
root.mainloop()
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`tk.Label(root, text="")`
[Slot 2]	`name_entry.get()`
[Slot 3]	`send`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 3	`send()`	Gọi send ngay lúc dựng giao diện; nút nhận None.
ô 1	`tk.Label(root, text="").pack()`	pack() trả về None nên greeting thành None.
ô 2	`name_entry`	Đó là widget, không phải chữ đã gõ — cần get().

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 1, ô 2, ô 3.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [0, 2, 3, 1].

Bên trái	NỐI VỚI (đáp án)
tk.Entry(frame, show="*")	**ô mật khẩu che ký tự**
command=login	**nút giữ hàm, chưa gọi**
Bấm nút	**Tkinter gọi login()**
login()	**get() hai Entry rồi status.config(text=...)**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`tk.Label(root, text="")`
ô 2	`name_entry.get()`
ô 3	`send`

Bẫy ở ô 3 — `send()`: Gọi send ngay lúc dựng giao diện; nút nhận None.
Bẫy ở ô 1 — `tk.Label(root, text="").pack()`: pack() trả về None nên greeting thành None.
Bẫy ở ô 2 — `name_entry`: Đó là widget, không phải chữ đã gõ — cần get().

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 32 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 25).
Lỗi: command=login() gọi ngay
Sửa thành:

```
tk.Button(frame, text="Đăng nhập", command=login).pack()
```

```
    25 |     if password_entry.get() == PASSWORD:
    26 |         status.config(text=f"Xin chào {user}")
    27 |         messagebox.showinfo("Đăng nhập", f"Chào mừng {user}!")
    28 |     else:
    29 |         status.config(text="Sai mật khẩu")
    30 |
    31 |
>>  32 | tk.Button(frame, text="Đăng nhập", command=login()).pack()
    33 |
    34 | root.mainloop()
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: status = Label(...).pack()
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
19	`status = tk.Label(frame, text="Chưa đăng nhập").pack()`	`status = tk.Label(frame, text="Chưa đăng nhập")`
20	`(không có dòng)`	`status.pack()`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao login() phải gọi user_entry.get() BÊN TRONG hàm, không đọc sẵn ở ngoài?

Câu hỏi của màn: Vì sao login() phải gọi user_entry.get() BÊN TRONG hàm, không đọc sẵn ở ngoài?
Lời giải thích mẫu: Dòng ngoài hàm chạy một lần lúc dựng giao diện, khi Entry còn rỗng, nên giá trị đọc được là "". Callback chạy mỗi lần bấm nút, lúc người dùng đã gõ xong, nên phải gọi get() trong callback để lấy chữ hiện tại. Trường hợp đặc biệt: mật khẩu rỗng vẫn phải ra "Sai mật khẩu".
Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — command=login() gọi ngay

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
32	`tk.Button(frame, text="Đăng nhập", command=login).pack()`	`tk.Button(frame, text="Đăng nhập", command=login()).pack()`

KIỂU SAI 2 — status = Label(...).pack()

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
19	`status = tk.Label(frame, text="Chưa đăng nhập")`	`status = tk.Label(frame, text="Chưa đăng nhập").pack()`
20	`status.pack()`	`(xoá dòng)`

KIỂU SAI 3 — đọc mật khẩu lúc dựng giao diện

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
—	`(không có dòng)`	`password = password_entry.get()`
—	`(không có dòng)`	
25	`    if password_entry.get() == PASSWORD:`	`    if password == PASSWORD:`

Nguồn trang này: pe-v4 · 015-G02.json · 015-G02.json · qa.json[G02]

G03 · Dữ liệu nhập và lựa chọn

Mã bài G03 · Nâng cao · M3 · GUI Tkinter và dự án · 35 phút · 20 XP · thứ tự 16 · mức N

MỤC TIÊU BÀI

Đọc Entry, Combobox, Radiobutton, Checkbutton bằng get() và kiểm dữ liệu trước khi dùng.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Form nhập điểm đã có name_var, mark_var (StringVar gắn hai Entry), course_box (ttk.Combobox), retake_var (BooleanVar của Checkbutton "Thi lại"), marks_list (Listbox) và status.

(1) Tạo hai Radiobutton "Giữa kỳ", "Cuối kỳ" dùng CHUNG exam_var (value bằng đúng chữ trên nút).

(2) Viết add_mark(): họ tên rỗng → messagebox.showwarning, dừng; điểm đổi sang float (chấp nhận "8,5"), không phải số hoặc ngoài 0..10 → messagebox.showerror, dừng; hợp lệ → thêm dòng "<tên> | <môn> | <kỳ thi> | <điểm>" (điểm là float vừa đổi, in thẳng: 8.5, 7.0, 9.25; thêm " | thi lại" nếu đánh dấu) vào cuối marks_list và đổi status thành "Đã nhập <số dòng> điểm".

Thực hành: Mỗi giá trị đọc qua get(); điểm qua replace(",", ".") rồi float() trong try/except; kiểm 0..10; chỉ insert khi mọi thứ hợp lệ.

Gọi (đề cho)	Kết quả mong đợi
Mai, "8.5", Python, Giữa kỳ, không thi lại	Dòng "Mai | Python | Giữa kỳ | 8.5"; status "Đã nhập 1 điểm"
Điểm "abc" hoặc "10.5"	showerror; Listbox giữ nguyên
Họ tên rỗng	showwarning; Listbox giữ nguyên

JSON của bài không khai step_4.contracts (hợp đồng hàm) — chỗ này KHÔNG được suy diễn thêm.

LỜI GIẢI MẪU (qa.json · G03)

```
import tkinter as tk
from tkinter import ttk, messagebox

root = tk.Tk()
root.title("Nhập điểm")

name_var = tk.StringVar()
mark_var = tk.StringVar()
exam_var = tk.StringVar(value="Giữa kỳ")
retake_var = tk.BooleanVar(value=False)

tk.Label(root, text="Họ tên").pack()
tk.Entry(root, textvariable=name_var).pack()
tk.Label(root, text="Điểm (0-10)").pack()
tk.Entry(root, textvariable=mark_var).pack()

course_box = ttk.Combobox(root, values=["Python", "Data", "Web"], state="readonly")
course_box.set("Python")
course_box.pack()

for text in ["Giữa kỳ", "Cuối kỳ"]:
    tk.Radiobutton(root, text=text, value=text, variable=exam_var).pack()

tk.Checkbutton(root, text="Thi lại", variable=retake_var).pack()

marks_list = tk.Listbox(root, height=6)
marks_list.pack()
status = tk.Label(root, text="Chưa có điểm")
status.pack()


def add_mark():
    name = name_var.get().strip()
    if name == "":
        messagebox.showwarning("Thiếu họ tên", "Hãy nhập họ tên")
        return
    raw = mark_var.get().strip().replace(",", ".")
    try:
        mark = float(raw)
    except ValueError:
        messagebox.showerror("Điểm không hợp lệ", f"'{raw}' không phải là số")
        return
    if not 0 <= mark <= 10:
        messagebox.showerror("Điểm không hợp lệ", "Điểm phải trong khoảng 0..10")
        return
    line = f"{name} | {course_box.get()} | {exam_var.get()} | {mark}"
    if retake_var.get():
        line += " | thi lại"
    marks_list.insert(tk.END, line)
    status.config(text=f"Đã nhập {marks_list.size()} điểm")


tk.Button(root, text="Thêm", command=add_mark).pack()

root.mainloop()
```

BỘ CHẤM — 9 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Chưa nhập: Listbox rỗng, status "Chưa có điểm"	`(marks_list.size(), status.cget('text'))` `→ (0, 'Chưa có điểm')`

2	Mai, 8.5, Python, Giữa kỳ	`(name_var.set('Mai'), mark_var.set('8.5'), course_box.set('Python'), »`
		`_studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), »`
		`retake_var.set(False), _studio.tk.find(type='Button', »`
		`text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1]`
		`→ 'Mai | Python | Giữa kỳ | 8.5'`

3	Radiobutton Cuối kỳ + Data + thi lại	`(name_var.set('An'), mark_var.set('7'), course_box.set('Data'), »`
		`_studio.tk.find(type='Radiobutton', text='Cuối kỳ').invoke(), »`
		`retake_var.set(True), _studio.tk.find(type='Button', »`
		`text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1]`
		`→ 'An | Data | Cuối kỳ | 7.0 | thi lại'`

4	Dấu phẩy "9,25", tên có khoảng trắng	`(name_var.set(' Lan '), mark_var.set('9,25'), course_box.set('Web'), »`
		`_studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), »`
		`retake_var.set(False), _studio.tk.find(type='Button', »`
		`text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1]`
		`→ 'Lan | Web | Giữa kỳ | 9.25'`

5	Biên 10 và 0 hợp lệ	`[(name_var.set('Huy'), mark_var.set('10'), course_box.set('Python'), »`
		`_studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), »`
		`retake_var.set(False), _studio.tk.find(type='Button', »`
		`text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1], »`
		`(name_var.set('Huy'), mark_var.set('0'), course_box.set('Python'), »`
		`_studio.tk.find(type='Radiobutton', text='Cuối kỳ').invoke(), »`
		`retake_var.set(False), _studio.tk.find(type='Button', »`
		`text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1]]`
		`→ ['Huy | Python | Giữa kỳ | 10.0', 'Huy | Python | Cuối kỳ | 0.0']`

6	"abc" → showerror, không thêm dòng	`(lambda n, m: ((name_var.set('Mai'), mark_var.set('abc'), »`
		`course_box.set('Python'), _studio.tk.find(type='Radiobutton', »`
		`text='Giữa kỳ').invoke(), retake_var.set(False), »`
		`_studio.tk.find(type='Button', text='Thêm').invoke(), »`
		`marks_list.get(marks_list.size() - 1))[-1], (marks_list.size() - n, »`
		`len(_studio.tk.messages) - m, 'error' in »`
		`str(_studio.tk.messages[-1][0]) if _studio.tk.messages else »`
		`False))[1])(marks_list.size(), len(_studio.tk.messages))`
		`→ (0, 1, True)`

7	"10.5" ngoài 0..10 → showerror, không thêm dòng	`(lambda n, m: ((name_var.set('Mai'), mark_var.set('10.5'), »`
		`course_box.set('Python'), _studio.tk.find(type='Radiobutton', »`
		`text='Giữa kỳ').invoke(), retake_var.set(False), »`
		`_studio.tk.find(type='Button', text='Thêm').invoke(), »`
		`marks_list.get(marks_list.size() - 1))[-1], (marks_list.size() - n, »`
		`len(_studio.tk.messages) - m, 'error' in »`
		`str(_studio.tk.messages[-1][0]) if _studio.tk.messages else »`
		`False))[1])(marks_list.size(), len(_studio.tk.messages))`
		`→ (0, 1, True)`

8	Họ tên rỗng → showwarning, không thêm dòng	`(lambda n, m: ((name_var.set('  '), mark_var.set('8'), »`
		`course_box.set('Python'), _studio.tk.find(type='Radiobutton', »`
		`text='Giữa kỳ').invoke(), retake_var.set(False), »`
		`_studio.tk.find(type='Button', text='Thêm').invoke(), »`
		`marks_list.get(marks_list.size() - 1))[-1], (marks_list.size() - n, »`
		`len(_studio.tk.messages) - m, 'warning' in »`
		`str(_studio.tk.messages[-1][0]) if _studio.tk.messages else »`
		`False))[1])(marks_list.size(), len(_studio.tk.messages))`
		`→ (0, 1, True)`

9	status đếm đúng số dòng trong Listbox	`status.cget('text')` `→ 'Đã nhập %d điểm' % marks_list.size()`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Mỗi giá trị đọc qua get(); điểm qua replace(",", ".") rồi float() trong try/except; kiểm 0..10; chỉ insert khi mọi thứ hợp lệ.

Cấu trúc. 1. for text in ["Giữa kỳ", "Cuối kỳ"]: tk.Radiobutton(root, text=text, value=text, variable=exam_var).pack() 2. name = name_var.get().strip(); rỗng → showwarning, return. 3. mark = float(mark_var.get().strip().replace(",", ".")) trong try; ValueError hoặc ngoài 0..10 → showerror, return. 4. line = f"{name} | {course_box.get()} | {exam_var.get()} | {mark}"; retake_var.get() → thêm " | thi lại"; insert; cập nhật status.

Lời giải từng phần. raw = mark_var.get().strip().replace(",", ".") try: mark = float(raw) except ValueError: messagebox.showerror("Điểm không hợp lệ", f"'{raw}' không phải là số") return

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — tkinter · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy tkinter thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

• Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar…), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ.

• Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn.

- Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

NGỘ NHẬN THƯỜNG GẶP: "Viết if retake_var: là kiểm ô đã tích"

retake_var là OBJECT BooleanVar — object luôn được coi là đúng, nên nhánh "thi lại" chạy cả khi ô chưa tích. Phải đọc giá trị: if retake_var.get():. Tương tự float(mark_var) nổ TypeError vì truyền object, không phải chuỗi. Và Radiobutton mỗi nút một biến riêng thì không còn là một nhóm: chọn "Cuối kỳ" không đổi exam_var.

CÂU DỰ ĐOÁN — đáp án và giải thích

Entry chứa "8.5" — mark_var.get() trả gì?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Số thực 8.5	StringVar luôn trả str.
B ĐÚNG	**Chuỗi "8.5" — phải float() mới thành số 8.5**	Entry chỉ biết chữ; đổi kiểu là việc của bạn.
C	Số nguyên 8	Không có phép đổi kiểu tự động nào.

TỰ KIỂM TRA — 4 câu trắc nghiệm

1. Entry chứa "8.5" có tự là float không?

A	Có, Tkinter tự nhận ra số	Entry không biết kiểu dữ liệu.
B ĐÚNG	**Không — phải chuyển kiểu và xử lý dữ liệu không hợp lệ**	get() luôn trả chuỗi.
C	Có, nếu dùng StringVar	StringVar cũng trả chuỗi.

Giải thích (nguyên văn JSON): get() luôn trả chuỗi.

2. Hai Radiobutton "Giữa kỳ", "Cuối kỳ" phải cấu hình thế nào để thành một nhóm?

A ĐÚNG	**Cùng variable=exam_var, mỗi nút một value khác nhau**	Chọn nút nào thì exam_var nhận value của nút đó.
B	Mỗi nút một StringVar riêng	Khi đó chúng độc lập, không loại trừ nhau.
C	Cùng text, khác variable	text chỉ là chữ hiển thị.

Giải thích (nguyên văn JSON): Chọn nút nào thì exam_var nhận value của nút đó.

3. float("8,5") cho kết quả gì?

A	85.0	Dấu phẩy không bị bỏ qua.
B ĐÚNG	**ValueError — Python dùng dấu chấm thập phân**	Thay "," bằng "." trước khi đổi kiểu.
C	8.5	float không hiểu dấu phẩy thập phân.

Giải thích (nguyên văn JSON): Thay "," bằng "." trước khi đổi kiểu.

4. marks_list.get(tk.ACTIVE) trả gì?

A	Mục cuối cùng	Mục cuối là get(tk.END).
B	Mọi mục trong Listbox	Muốn hết thì get(0, tk.END).
C ĐÚNG	**Mục đang active (có con trỏ), không nhất thiết là mọi mục được chọn**	Mục được chọn: curselection().

Giải thích (nguyên văn JSON): Mục được chọn: curselection().

CÂU PHẢN VÍ DỤ — đáp án

Không thay "," bằng "." thì float("9,25") làm gì?

Nhãn	Lựa chọn
A	`Trả 9.25`
B	`Trả 925.0`
C ĐÚNG	`ValueError`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao add_mark phải dừng (return) ngay khi điểm không hợp lệ, trước khi insert vào Listbox?

Mẫu đạt: Dữ liệu từ giao diện là chuỗi người dùng gõ, có thể là "abc" hoặc "11". Đổi kiểu và kiểm khoảng trước; sai thì báo bằng messagebox rồi return để Listbox và status không nhận dòng hỏng. Trường hợp đặc biệt: "8,5" hợp lệ sau khi thay dấu phẩy; 0 và 10 nằm trong khoảng.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái

Bước 1	name_var.get().strip() → họ tên đã bỏ khoảng trắng
Bước 2	float(raw.replace(",", ".")) → "8,5" thành 8.5
Bước 3	Không phải số hoặc ngoài 0..10 → showerror rồi return
Bước 4	marks_list.insert(tk.END, line) → thêm dòng cuối Listbox

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Dữ liệu mới: chọn ca học bằng Radiobutton và đánh dấu "Có laptop". Lắp ba chỗ.

Yêu cầu thêm: Bộ chấm điền tên/điểm khác, chọn môn bằng course_box.set, bấm Radiobutton "Cuối kỳ", tích hoặc bỏ "Thi lại", rồi bấm Thêm và đọc dòng cuối Listbox.

```
import tkinter as tk

root = tk.Tk()
shift_var = tk.StringVar(value="Sáng")
laptop_var = tk.BooleanVar(value=False)

for text in ["Sáng", "Chiều"]:
    tk.Radiobutton(root, text=text, value=text, variable={{slot1}}).pack()
tk.Checkbutton(root, text="Có laptop", variable=laptop_var).pack()
summary = tk.Label(root, text="")
summary.pack()


def show():
    note = "mang laptop" if {{slot2}} else "không laptop"
    summary.config(text="Ca " + {{slot3}} + ", " + note)


tk.Button(root, text="Xem", command=show).pack()
root.mainloop()
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`shift_var`
[Slot 2]	`laptop_var.get()`
[Slot 3]	`shift_var.get()`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 2	`laptop_var`	Object BooleanVar luôn được coi là đúng — phải gọi get().
ô 1	`tk.StringVar(value="Sáng")`	Mỗi nút một biến mới: hai Radiobutton không còn chung nhóm.
ô 3	`str(shift_var)`	str() của StringVar là TÊN biến Tcl (PY_VAR0), không phải giá trị.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 3, ô 2, ô 1.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [3, 1, 2, 0].

Bên trái	NỐI VỚI (đáp án)
name_var.get().strip()	**họ tên đã bỏ khoảng trắng**
float(raw.replace(",", "."))	**"8,5" thành 8.5**
Không phải số hoặc ngoài 0..10	**showerror rồi return**
marks_list.insert(tk.END, line)	**thêm dòng cuối Listbox**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`shift_var`
ô 2	`laptop_var.get()`
ô 3	`shift_var.get()`

Bẫy ở ô 2 — `laptop_var`: Object BooleanVar luôn được coi là đúng — phải gọi get().
Bẫy ở ô 1 — `tk.StringVar(value="Sáng")`: Mỗi nút một biến mới: hai Radiobutton không còn chung nhóm.
Bẫy ở ô 3 — `str(shift_var)`: str() của StringVar là TÊN biến Tcl (PY_VAR0), không phải giá trị.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 47 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 40).
Lỗi: if retake_var: (quên get())
Sửa thành:

```
    if retake_var.get():
    40 |     except ValueError:
```

```
    41 |         messagebox.showerror("Điểm không hợp lệ", f"'{raw}' không phải là số")
    42 |         return
    43 |     if not 0 <= mark <= 10:
    44 |         messagebox.showerror("Điểm không hợp lệ", "Điểm phải trong khoảng 0..10")
    45 |         return
    46 |     line = f"{name} | {course_box.get()} | {exam_var.get()} | {mark}"
>>  47 |     if retake_var:
    48 |         line += " | thi lại"
    49 |     marks_list.insert(tk.END, line)
    50 |     status.config(text=f"Đã nhập {marks_list.size()} điểm")
    51 |
    52 |
    53 | tk.Button(root, text="Thêm", command=add_mark).pack()
    54 |
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: mỗi Radiobutton một biến riêng

Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
22	`tk.Radiobutton(root, text=text, value=text, » variable=tk.StringVar(value="Giữa kỳ")).pack()`	`tk.Radiobutton(root, text=text, value=text, » variable=exam_var).pack()`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao add_mark phải dừng (return) ngay khi điểm không hợp lệ, trước khi insert vào Listbox?

Câu hỏi của màn: Vì sao add_mark phải dừng (return) ngay khi điểm không hợp lệ, trước khi insert vào Listbox?

Lời giải thích mẫu: Dữ liệu từ giao diện là chuỗi người dùng gõ, có thể là "abc" hoặc "11". Đổi kiểu và kiểm khoảng trước; sai thì báo bằng messagebox rồi return để Listbox và status không nhận dòng hỏng. Trường hợp đặc biệt: "8,5" hợp lệ sau khi thay dấu phẩy; 0 và 10 nằm trong khoảng.

Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — if retake_var: (quên get())

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
47	`if retake_var.get():`	`if retake_var:`

KIỂU SAI 2 — mỗi Radiobutton một biến riêng

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
22	`tk.Radiobutton(root, text=text, value=text, » variable=exam_var).pack()`	`tk.Radiobutton(root, text=text, value=text, » variable=tk.StringVar(value="Giữa kỳ")).pack()`

KIỂU SAI 3 — không bắt ValueError khi đổi kiểu

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
38	`try:`	`mark = float(raw)`
39	`mark = float(raw)`	(xoá dòng)
40	`except ValueError:`	(xoá dòng)
41	`messagebox.showerror("Điểm không hợp lệ", » f"'{raw}' không phải là số")`	(xoá dòng)
42	`return`	(xoá dòng)

Nguồn trang này: pe-v4 · 016-G03.json · 016-G03.json · qa.json[G03]

G04 · Layout: pack, place, grid

Mã bài G04 · Nâng cao · M3 · GUI Tkinter và dự án · 30 phút · 20 XP · thứ tự 17 · mức N

MỤC TIÊU BÀI

Bố trí form bằng grid trong Frame, khung bằng pack, nhãn góc bằng place.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết build_form(parent, fields): với field thứ row (đếm từ 0) đặt Label(text=field) ở grid(row=row, column=0, sticky="w") và một Entry ở grid(row=row, column=1, sticky="ew"); lưu Entry vào dict entries[field]; cuối cùng đặt Button "Lưu" ở grid(row=len(fields), column=0, columnspan=2); trả về entries.

Trong root: form = Frame pack(fill="x") rồi entries = build_form(form, FIELDS); toolbar = Frame pack(side="bottom", fill="x") chứa nút "Mở" pack(side="left") và "Thoát" pack(side="right"); badge = Label "v1.0" đặt bằng place(relx=1.0, rely=0.0, anchor="ne").

Thực hành: Một parent — một kiểu bố trí: form dùng grid cho con của nó, root dùng pack cho hai Frame, badge dùng place.

Gọi (đề cho)	Kết quả mong đợi
`FIELDS = ["Họ tên", "Ngày sinh", "Điểm"]`	`Entry ở (0, 1), (1, 1), (2, 1); nút Lưu ở hàng 3, » columnspan=2`
`5 field`	`Entry ở hàng 0..4; nút Lưu ở hàng 5`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`build_form(parent, fields)`	Dựng form nhập bằng grid trong parent. · parent: Frame chưa có con nào; fields: list[str] · dict tên field → Entry, theo thứ tự fields · Field thứ i: Label (i, 0) sticky="w", Entry (i, 1) sticky="ew"; Button "Lưu" (len(fields), 0) columnspan=2.

LỜI GIẢI MẪU (qa.json · G04)

```
import tkinter as tk

FIELDS = ["Họ tên", "Ngày sinh", "Điểm"]


def build_form(parent, fields):
    entries = {}
    for row, text in enumerate(fields):
        tk.Label(parent, text=text).grid(row=row, column=0, sticky="w", padx=4, pady=2)
        entry = tk.Entry(parent)
        entry.grid(row=row, column=1, sticky="ew", padx=4, pady=2)
        entries[text] = entry
    tk.Button(parent, text="Lưu").grid(row=len(fields), column=0, columnspan=2, pady=4)
    return entries


root = tk.Tk()
root.title("Hồ sơ sinh viên")
root.geometry("360x220")

form = tk.Frame(root)
form.pack(fill="x", padx=8, pady=8)
entries = build_form(form, FIELDS)

toolbar = tk.Frame(root)
toolbar.pack(side="bottom", fill="x")
tk.Button(toolbar, text="Mở").pack(side="left")
tk.Button(toolbar, text="Thoát").pack(side="right")

badge = tk.Label(root, text="v1.0")
badge.place(relx=1.0, rely=0.0, anchor="ne")

root.mainloop()
```

BỘ CHẤM — 9 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	form pack(fill="x") trong root	`(form.manager, form.layout.get('fill'))` → `('pack', 'x')`
2	Mỗi Entry ở cột 1, mỗi field một hàng	`[(e.manager, e.layout.get('row'), e.layout.get('column')) for e in » entries.values()]` → `[('grid', 0, 1), ('grid', 1, 1), ('grid', 2, 1)]`
3	Label ở cột 0, cùng hàng với Entry	`[(w.manager, w.layout.get('row'), w.layout.get('column')) for w in » (_studio.tk.find(type='Label', text=t) for t in ['Họ tên', 'Ngày » sinh', 'Điểm'])]` → `[('grid', 0, 0), ('grid', 1, 0), ('grid', 2, 0)]`

4	Entry giãn ngang: sticky có e và w	`[{'e', 'w'} <= set(''.join(e.layout.get('sticky', ''))) for e in » entries.values()]` `→ [True, True, True]`
5	Nút Lưu: hàng 3, columnspan=2	`(lambda b: (b.manager, b.layout.get('row'), » b.layout.get('columnspan')))(_studio.tk.find(type='Button', » text='Lưu'))` `→ ('grid', 3, 2)`
6	toolbar ở đáy; Mở bên trái, Thoát bên phải	`(toolbar.manager, toolbar.layout.get('side'), » _studio.tk.find(type='Button', text='Mở').layout.get('side'), » _studio.tk.find(type='Button', text='Thoát').layout.get('side'))` `→ ('pack', 'bottom', 'left', 'right')`
7	badge đặt bằng place ở góc trên phải	`(badge.manager, badge.layout.get('relx'), badge.layout.get('rely'), » badge.layout.get('anchor'))` `→ ('place', 1.0, 0.0, 'ne')`
8	5 field khác: Entry hàng 0..4, nút Lưu hàng 5	`(lambda es: ([(e.layout.get('row'), e.layout.get('column')) for e in » es.values()], _studio.tk.find_all(type='Button', » text='Lưu')[-1].layout.get('row')))(build_form(tk.Frame(root), ['Mã » SV', 'Lớp', 'Email', 'SĐT', 'Ghi chú']))` `→ ([(0, 1), (1, 1), (2, 1), (3, 1), (4, 1)], 5)`
9	1 field: Label hàng 0, nút Lưu ngay hàng 1	`(lambda es: (list(es), _studio.tk.find(type='Label', text='Ghi chú » riêng').layout.get('row'), _studio.tk.find_all(type='Button', » text='Lưu')[-1].layout.get('row')))(build_form(tk.Frame(root), ['Ghi » chú riêng']))` `→ (['Ghi chú riêng'], 0, 1)`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Một parent — một kiểu bố trí: form dùng grid cho con của nó, root dùng pack cho hai Frame, badge dùng place.

Cấu trúc. 1. for row, text in enumerate(fields): Label grid(row=row, column=0, sticky="w"); Entry grid(row=row, column=1, sticky="ew"). 2. entries[text] = entry; sau vòng lặp: Button "Lưu" grid(row=len(fields), column=0, columnspan=2); return entries. 3. form = tk.Frame(root); form.pack(fill="x"); entries = build_form(form, FIELDS). 4. toolbar pack(side="bottom", fill="x"): "Mở" pack(side="left"), "Thoát" pack(side="right"); badge.place(relx=1.0, rely=0.0, anchor="ne").

Lời giải từng phần. for row, text in enumerate(fields): tk.Label(parent, text=text).grid(row=row, column=0, sticky="w") ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — tkinter · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy tkinter thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar…), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ.
- Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn.
- Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

NGỘ NHẬN THƯỜNG GẶP: "Thêm một Label bằng pack() vào Frame đang dùng grid cũng chẳng sao"

pack và grid cùng tính kích thước của parent và giành nhau quyền quyết định — Tk chặn bằng TclError "cannot use geometry manager pack inside ... which already has slaves managed by grid". Muốn phối hợp, tạo Frame riêng cho mỗi kiểu. Một nhầm lẫn khác: gọi grid() không có row/column và tưởng nó tự xếp như pack — Label và Entry sẽ không còn thẳng hàng.

CÂU DỰ ĐOÁN — đáp án và giải thích

Có thể pack frame ngoài, rồi grid các widget con BÊN TRONG frame đó không?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A ĐÚNG	**Có — frame (con của root) và widget con của frame có parent khác nhau**	Luật chỉ áp dụng trong cùng một parent.
B	Không — cả chương trình chỉ được dùng một kiểu	Mỗi parent chọn kiểu riêng cho các con của nó.
C	Được, nhưng phải dùng place cho frame	pack frame là bình thường.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Trong cùng một Frame, đã có widget dùng grid; thêm widget dùng pack thì…

A ĐÚNG	**Tk báo TclError: không được trộn pack và grid trong cùng parent**	Tách sang Frame khác nếu cần pack.
B	Widget mới tự chuyển sang grid	Tk không tự đổi geometry manager.
C	Chạy được, chỉ lệch vị trí	Tk chặn hẳn bằng lỗi.

Giải thích (nguyên văn JSON): Tách sang Frame khác nếu cần pack.

2. Entry ở cột 1 muốn giãn hết bề ngang ô thì dùng gì?

A	grid(..., side="left")	side cũng là của pack.
B	grid(..., fill="x")	fill là tuỳ chọn của pack, grid không nhận.
C ĐÚNG	**grid(..., sticky="ew")**	Bám cả cạnh trái (w) và phải (e).

Giải thích (nguyên văn JSON): Bám cả cạnh trái (w) và phải (e).

3. place(relx=1.0, rely=0.0, anchor="ne") đặt widget ở đâu?

A	Góc dưới trái	Đó là relx=0.0, rely=1.0, anchor="sw".
B ĐÚNG	**Góc trên phải: góc ne của widget bám vào điểm (100% ngang, 0% dọc)**	relx/rely là tỉ lệ theo parent.
C	Toạ độ (1, 0) pixel	Pixel là x/y; relx/rely là tỉ lệ.

Giải thích (nguyên văn JSON): relx/rely là tỉ lệ theo parent.

CÂU PHẢN VÍ DỤ — đáp án

build_form(frame, ["Mã SV", "Lớp", "Email", "SĐT", "Ghi chú"]) đặt nút Lưu ở hàng nào?

Nhãn	Lựa chọn
A ĐÚNG	`5`
B	`0`
C	`4`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Vì sao form dùng grid trong một Frame riêng thay vì grid thẳng vào root?

Mẫu đạt: root còn toolbar dùng pack. Nếu grid các Label/Entry thẳng vào root thì root có con dùng cả grid lẫn pack → TclError. Đặt form vào Frame riêng: root chỉ pack hai Frame, còn bên trong form tự do dùng grid. Trường hợp đặc biệt: place không tranh chấp, nên badge đặt bằng place trong root vẫn được.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	form.pack(fill="x") → khung form giãn ngang trong root
Bước 2	enumerate(fields) → mỗi field một số hàng 0, 1, 2
Bước 3	Label cột 0, Entry cột 1 sticky="ew" → thẳng hàng
Bước 4	Nút Lưu grid(row=len(fields), columnspan=2) → hàng cuối

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Dữ liệu mới: bàn phím 2×2 trong một Frame được pack. Lắp bốn chỗ.

Yêu cầu thêm: Bộ chấm gọi build_form với danh sách field khác (5 field, 1 field) và đọc hàng/cột của từng widget; trộn pack với grid trong cùng parent sẽ làm chương trình dừng ngay bằng TclError.

```
import tkinter as tk

root = tk.Tk()
pad = tk.Frame(root)
pad.{{slot1}}
for index, text in enumerate(["7", "8", "4", "5"]):
    button = tk.Button(pad, text=text, width=4)
    button.grid(row={{slot2}}, column={{slot3}})
tk.Label(root, text="Bàn phím số").{{slot4}}
root.mainloop()
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`pack(padx=10, pady=10)`
[Slot 2]	`index // 2`
[Slot 3]	`index % 2`
[Slot 4]	`pack(side="bottom")`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 1	`grid(row=0, column=0)`	pad dùng grid nhưng Label cùng root lại pack → TclError.
ô 4	`grid(row=1, column=0)`	root đã có pad dùng pack → trộn với grid → TclError.
ô 2	`index`	Mỗi nút một hàng: thành một cột dọc bốn nút.
ô 3	`0`	Mọi nút dồn vào cột 0.

Ngân hàng còn 4 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 4, ô 2, ô 3, ô 1.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [1, 3, 0, 2].

Bên trái	NỐI VỚI (đáp án)
form.pack(fill="x")	**khung form giãn ngang trong root**
enumerate(fields)	**mỗi field một số hàng 0, 1, 2**
Label cột 0, Entry cột 1 sticky="ew"	**thẳng hàng**
Nút Lưu grid(row=len(fields), columnspan=2)	**hàng cuối**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`pack(padx=10, pady=10)`
ô 2	`index // 2`
ô 3	`index % 2`
ô 4	`pack(side="bottom")`

Bẫy ở ô 1 — `grid(row=0, column=0)`: pad dùng grid nhưng Label cùng root lại pack → TclError.
Bẫy ở ô 4 — `grid(row=1, column=0)`: root đã có pad dùng pack → trộn với grid → TclError.
Bẫy ở ô 2 — `index`: Mỗi nút một hàng: thành một cột dọc bốn nút.
Bẫy ở ô 3 — `0`: Mọi nút dồn vào cột 0.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 23 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 16).
Lỗi: pack thêm Label vào form đang grid
JSON không khai `fix` — không có dòng sửa mẫu.

```
    16 |
    17 | root = tk.Tk()
    18 | root.title("Hồ sơ sinh viên")
    19 | root.geometry("360x220")
    20 |
    21 | form = tk.Frame(root)
    22 | form.pack(fill="x", padx=8, pady=8)
>>  23 | tk.Label(form, text="Thông tin").pack()
    24 | entries = build_form(form, FIELDS)
    25 |
    26 | toolbar = tk.Frame(root)
    27 | toolbar.pack(side="bottom", fill="x")
    28 | tk.Button(toolbar, text="Mở").pack(side="left")
    29 | tk.Button(toolbar, text="Thoát").pack(side="right")
    30 |
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: form grid trong root, toolbar lại pack
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
22	`form.grid(row=0, column=0, sticky="ew")`	`form.pack(fill="x", padx=8, pady=8)`

selfExplain-4 · TỰ GIẢI THÍCH — Vì sao form dùng grid trong một Frame riêng thay vì grid thẳng vào root?

Câu hỏi của màn: Vì sao form dùng grid trong một Frame riêng thay vì grid thẳng vào root?
Lời giải thích mẫu: root còn toolbar dùng pack. Nếu grid các Label/Entry thẳng vào root thì root có con dùng cả grid lẫn pack → TclError. Đặt form vào Frame riêng: root chỉ pack hai Frame, còn bên trong form tự do dùng grid. Trường hợp đặc biệt: place không tranh chấp, nên badge đặt bằng place trong root vẫn được.
Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 3 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — pack thêm Label vào form đang grid

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
—	(không có dòng)	`tk.Label(form, text="Thông tin").pack()`

KIỂU SAI 2 — form grid trong root, toolbar lại pack

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
22	`form.pack(fill="x", padx=8, pady=8)`	`form.grid(row=0, column=0, sticky="ew")`

KIỂU SAI 3 — grid() không row/column, tưởng tự xếp như pack

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
9	`tk.Label(parent, text=text).grid(row=row, » column=0, sticky="w", padx=4, pady=2)`	`tk.Label(parent, text=text).grid(sticky="w")`
11	`entry.grid(row=row, column=1, sticky="ew", » padx=4, pady=2)`	`entry.grid(sticky="ew")`

Nguồn trang này: pe-v4 · 017-G04.json · 017-G04.json · qa.json[G04]

G05 · OOP và tách logic khỏi giao diện

Mã bài G05 · Nâng cao · M3 · GUI Tkinter và dự án · 35 phút · 20 XP · thứ tự 18 · mức N

MỤC TIÊU BÀI

Gói giao diện vào class App, tách logic thành hàm thuần.

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm THUẦN (không đụng widget, không messagebox): validate_mark(raw) đổi chuỗi thành float, không phải số hoặc ngoài 0..10 thì raise ValueError; summary(marks) trả "Chưa có điểm" khi list rỗng, ngược lại "<n> điểm, trung bình <tb, 2 chữ số thập phân>".

Hoàn thành class App: __init__ tạo self.marks = [], self.mark_var rồi gọi create_widgets(); create_widgets tạo Entry gắn self.mark_var, nút "Thêm điểm" với command=self.add_mark và self.result = Label hiện summary(self.marks); add_mark gọi validate_mark — sai thì messagebox.showerror, đúng thì append vào self.marks và cập nhật self.result.

Thực hành: Hai hàm thuần không chạm widget; App giữ widget trong self, bắt ValueError của validate_mark trong add_mark.

Gọi (đề cho)	Kết quả mong đợi
`validate_mark("8.5") / validate_mark("11")`	`8.5 / ValueError`
`Gõ 8 rồi bấm "Thêm điểm"`	`self.marks = [8.0]; nhãn "1 điểm, trung bình 8.00"`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`validate_mark(raw)`	Đổi chuỗi người dùng gõ thành điểm hợp lệ. · raw: str (có thể có khoảng trắng hai đầu) · float trong 0..10; không phải số hoặc ngoài khoảng → ValueError · Hàm thuần: không đọc/ghi widget, không bật messagebox.
`summary(marks)`	Tạo chữ tóm tắt cho nhãn kết quả. · marks: list[float] · "Chưa có điểm" nếu rỗng; ngược lại "<n> điểm, trung bình <tb 2 số lẻ>" · Hàm thuần, không sửa marks.
`App(root)`	Gói giao diện: Entry gắn self.mark_var, nút "Thêm điểm", nhãn self.result. · root: cửa sổ Tk · object có self.marks (list riêng), self.mark_var, self.result · add_mark(): hợp lệ → thêm vào self.marks và cập nhật nhãn; sai → showerror, không thêm.

LỜI GIẢI MẪU (qa.json · G05)

```
import tkinter as tk
from tkinter import messagebox


def validate_mark(raw):
    mark = float(raw)
    if not 0 <= mark <= 10:
        raise ValueError("Outside 0..10")
    return mark


def summary(marks):
    if not marks:
        return "Chưa có điểm"
    average = sum(marks) / len(marks)
    return f"{len(marks)} điểm, trung bình {average:.2f}"


class App:
    def __init__(self, root):
        self.root = root
        self.marks = []
        self.mark_var = tk.StringVar()
        self.create_widgets()

    def create_widgets(self):
        tk.Entry(self.root, textvariable=self.mark_var).pack()
        tk.Button(self.root, text="Thêm điểm", command=self.add_mark).pack()
        self.result = tk.Label(self.root, text=summary(self.marks))
        self.result.pack()

    def add_mark(self):
        try:
            mark = validate_mark(self.mark_var.get())
        except ValueError as error:
            messagebox.showerror("Điểm không hợp lệ", str(error))
            return
        self.marks.append(mark)
        self.result.config(text=summary(self.marks))


if __name__ == "__main__":
    root = tk.Tk()
    root.title("Điểm trung bình")
```

```
app = App(root)
root.mainloop()
```

BỘ CHẤM — 11 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	validate_mark("8.5") → 8.5	`validate_mark("8.5")` `→ 8.5`
2	Biên 0 và 10, có khoảng trắng	`[validate_mark(" 0 "), validate_mark("10")]` `→ [0.0, 10.0]`
3	Dữ liệu sai → ValueError	`[_studio.call(validate_mark, raw).error for raw in ['abc', '10.5', '-1', '']]` `→ ['ValueError', 'ValueError', 'ValueError', 'ValueError']`

4	validate_mark không tự bật messagebox	`(lambda n: (_studio.call(validate_mark, '11'), len(_studio.tk.messages) - »`
		`n)[1])(len(_studio.tk.messages))`
		`→ 0`

5	summary([]) và summary([8, 7])	`(summary([]), summary([8, 7]))` `→ ('Chưa có điểm', '2 điểm, trung bình 7.50')`
6	summary làm tròn 2 chữ số	`summary([9, 8.5, 6.25])` `→ '3 điểm, trung bình 7.92'`
7	App giữ nhãn trong self.result	`app.result.cget('text')` `→ 'Chưa có điểm'`

8	Bấm "Thêm điểm" với 8	`(app.marks.clear(), app.mark_var.set('8'), _studio.tk.find(type='Button', »`
		`text='Thêm điểm').invoke(), list(app.marks), »`
		`app.result.cget('text'))[3:]`
		`→ ([8.0], '1 điểm, trung bình 8.00')`

9	"abc" → showerror, không thêm	`(lambda n, m: (app.mark_var.set('abc'), _studio.tk.find(type='Button', »`
		`text='Thêm điểm').invoke(), len(app.marks) - n, »`
		`len(_studio.tk.messages) - m)[2:])(len(app.marks), »`
		`len(_studio.tk.messages))`
		`→ (0, 1)`

10	App mới: self.marks riêng, nhãn ban đầu	`(lambda a: (a.marks, a.result.cget('text')))(App(tk.Tk()))` `→ ([], 'Chưa có điểm')`

11	add_mark gọi trực tiếp hai lần	`(lambda a: (a.mark_var.set('7.5'), a.add_mark(), a.mark_var.set('9'), »`
		`a.add_mark(), a.result.cget('text'))[-1])(App(tk.Tk()))`
		`→ '2 điểm, trung bình 8.25'`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Hai hàm thuần không chạm widget; App giữ widget trong self, bắt ValueError của validate_mark trong add_mark.

Cấu trúc. 1. validate_mark: mark = float(raw); ngoài 0..10 → raise ValueError(...); return mark. 2. summary: list rỗng → "Chưa có điểm"; còn lại f"{len(marks)} điểm, trung bình {sum(marks) / len(marks):.2f}". 3. create_widgets: tk.Button(self.root, text="Thêm điểm", command=self.add_mark); self.result = tk.Label(...). 4. add_mark: try validate_mark(self.mark_var.get()) except ValueError → showerror, return; append; self.result.config(text=summary(self.marks)).

Lời giải từng phần. def add_mark(self): try: mark = validate_mark(self.mark_var.get()) except ValueError as error: ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — tkinter · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy tkinter thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar...), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ.
- Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn.
- Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

NGỘ NHẬN THƯỜNG GẶP: "Cho validate_mark tự bật messagebox là tiện nhất"

Khi đó validate_mark chỉ dùng được trong GUI: gọi từ CLI hay từ test sẽ bật hộp thoại và trả None thay vì báo lỗi rõ ràng, nơi gọi phải đoán vì sao có None. Hàm thuần raise ValueError; App (tầng giao diện) bắt lỗi và chọn cách hiển thị. Đừng quên: command=self.add_mark() vẫn là gọi ngay, và result = tk.Label(...) không có self thì method khác không thấy.

CÂU DỰ ĐOÁN — đáp án và giải thích

Ô nhập đang là "abc", người dùng bấm "Thêm điểm". Chuyện gì xảy ra?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	self.marks thêm 0	Dữ liệu sai không bao giờ được thêm.

B ĐÚNG	**validate_mark raise ValueError; add_mark bắt lỗi, bật showerror; self.marks giữ nguyên**	Hàm thuần báo lỗi, tầng giao diện quyết định cách hiện.
C	Chương trình dừng hẳn với ValueError	add_mark bắt ValueError bằng try/except nên giao diện vẫn chạy.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. self.entry giúp gì so với một biến local mất sau hàm?

A	Giúp widget tự lưu dữ liệu ra file	self chỉ là thuộc tính của object.
B	Bắt buộc để widget hiện lên	Widget hiện nhờ pack/grid, không nhờ self.
C ĐÚNG	**Giữ widget để các method khác truy cập**	add_mark đọc self.mark_var và đổi self.result được vì chúng là thuộc tính của object.

Giải thích (nguyên văn JSON): add_mark đọc self.mark_var và đổi self.result được vì chúng là thuộc tính của object.

2. Vì sao validate_mark nên raise ValueError thay vì gọi messagebox.showerror?

A	Vì messagebox không chạy trong hàm	messagebox gọi được ở đâu cũng được — vấn đề là phụ thuộc GUI.
B ĐÚNG	**Để hàm dùng lại được ngoài GUI và nơi gọi tự quyết cách báo lỗi**	Logic tách khỏi giao diện.
C	Vì raise nhanh hơn	Không phải chuyện tốc độ.

Giải thích (nguyên văn JSON): Logic tách khỏi giao diện.

3. Khai marks = [] ở thân class (không trong __init__) thì hai App khác nhau...

A	Báo lỗi khi chạy	Chạy được, chỉ sai âm thầm.
B	Mỗi App có list riêng	Chỉ đúng khi tạo self.marks = [] trong __init__.
C ĐÚNG	**Dùng CHUNG một list — điểm của cửa sổ này hiện ở cửa sổ kia**	Thuộc tính class được mọi object chia sẻ.

Giải thích (nguyên văn JSON): Thuộc tính class được mọi object chia sẻ.

CÂU PHẢN VÍ DỤ — đáp án

summary([9, 8.5, 6.25]) trả gì?

Nhãn	Lựa chọn
A ĐÚNG	`3 điểm, trung bình 7.92`
B	`Chưa có điểm`
C	`3 điểm, trung bình 7.91`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Tách validate_mark/summary khỏi class App được lợi gì khi kiểm thử và khi đổi giao diện?

Mẫu đạt: validate_mark và summary chỉ nhận dữ liệu và trả kết quả (hoặc raise), nên kiểm được bằng gọi hàm, không cần mở cửa sổ hay bấm nút. App chỉ lo đọc widget, gọi hàm và hiện kết quả. Đổi sang giao diện khác (CLI, web) vẫn dùng lại nguyên hai hàm. Trường hợp đặc biệt: list rỗng → "Chưa có điểm", "abc" → ValueError.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	App(root) → __init__ tạo self.marks, self.mark_var
Bước 2	create_widgets() → Entry, nút, self.result
Bước 3	Bấm "Thêm điểm" → add_mark() gọi validate_mark
Bước 4	ValueError → showerror; hợp lệ → append rồi summary()

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Dữ liệu mới: ứng dụng đổi °C sang °F. Hàm thuần to_fahrenheit đã có; lắp bốn chỗ trong class App.

Yêu cầu thêm: Bộ chấm gọi thẳng validate_mark/summary với dữ liệu mới (biên 0 và 10, "abc", list ba điểm), rồi tạo App mới trên tk.Tk() mới để kiểm self.marks riêng và nút "Thêm điểm".

```
import tkinter as tk


def to_fahrenheit(raw):
    celsius = float(raw)
    return round(celsius * 9 / 5 + 32, 1)


class App:
```

```
    def __init__(self, root):
        self.celsius = tk.StringVar(value="0")
        tk.Entry(root, textvariable=self.celsius).pack()
        tk.Button(root, text="Đổi", command={{slot1}}).pack()
        {{slot2}} = tk.Label(root, text="")
        self.answer.pack()

    def convert(self):
        value = to_fahrenheit({{slot3}})
        {{slot4}}


root = tk.Tk()
app = App(root)
root.mainloop()
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`self.convert`
[Slot 2]	`self.answer`
[Slot 3]	`self.celsius.get()`
[Slot 4]	`self.answer.config(text=str(value) + " °F")`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 4	`print(value)`	In ra console; nhãn trên cửa sổ không đổi.
ô 3	`self.celsius`	Truyền object StringVar; float() cần chuỗi → TypeError.
ô 1	`self.convert()`	Gọi convert ngay trong __init__; nút nhận None.
ô 2	`answer`	Biến cục bộ mất sau __init__; convert không thấy được.

Ngân hàng còn 4 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 3, ô 1, ô 4, ô 2.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [0, 1, 3, 2].

Bên trái	NỐI VỚI (đáp án)
App(root)	**__init__ tạo self.marks, self.mark_var**
create_widgets()	**Entry, nút, self.result**
Bấm "Thêm điểm"	**add_mark() gọi validate_mark**
ValueError	**showerror; hợp lệ → append rồi summary()**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`self.convert`
ô 2	`self.answer`
ô 3	`self.celsius.get()`
ô 4	`self.answer.config(text=str(value) + " °F")`

Bẫy ở ô 4 — `print(value)`: In ra console; nhãn trên cửa sổ không đổi.
Bẫy ở ô 3 — `self.celsius`: Truyền object StringVar; float() cần chuỗi → TypeError.
Bẫy ở ô 1 — `self.convert()`: Gọi convert ngay trong __init__; nút nhận None.
Bẫy ở ô 2 — `answer`: Biến cục bộ mất sau __init__; convert không thấy được.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 28 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 21).
Lỗi: command=self.add_mark() gọi ngay
Sửa thành:

```
        tk.Button(self.root, text="Thêm điểm", command=self.add_mark).pack()
```

```
    21 |         self.root = root
    22 |         self.marks = []
    23 |         self.mark_var = tk.StringVar()
    24 |         self.create_widgets()
    25 |
    26 |     def create_widgets(self):
    27 |         tk.Entry(self.root, textvariable=self.mark_var).pack()
>>  28 |         tk.Button(self.root, text="Thêm điểm", command=self.add_mark()).pack()
    29 |         self.result = tk.Label(self.root, text=summary(self.marks))
    30 |         self.result.pack()
    31 |
    32 |     def add_mark(self):
    33 |         try:
    34 |             mark = validate_mark(self.mark_var.get())
    35 |         except ValueError as error:
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: nhãn kết quả là biến cục bộ

Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
29	`result = tk.Label(self.root, » text=summary(self.marks))`	`self.result = tk.Label(self.root, » text=summary(self.marks))`
30	`result.pack()`	`self.result.pack()`

selfExplain-4 · TỰ GIẢI THÍCH — Tách validate_mark/summary khỏi class App được lợi gì khi kiểm thử và khi đổi giao diện?

Câu hỏi của màn: Tách validate_mark/summary khỏi class App được lợi gì khi kiểm thử và khi đổi giao diện?

Lời giải thích mẫu: validate_mark và summary chỉ nhận dữ liệu và trả kết quả (hoặc raise), nên kiểm được bằng gọi hàm, không cần mở cửa sổ hay bấm nút. App chỉ lo đọc widget, gọi hàm và hiện kết quả. Đổi sang giao diện khác (CLI, web) vẫn dùng lại nguyên hai hàm. Trường hợp đặc biệt: list rỗng → "Chưa có điểm", "abc" → ValueError.

Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 4 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — command=self.add_mark() gọi ngay

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
28	`tk.Button(self.root, text="Thêm điểm", » command=self.add_mark).pack()`	`tk.Button(self.root, text="Thêm điểm", » command=self.add_mark()).pack()`

KIỂU SAI 2 — nhãn kết quả là biến cục bộ

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
29	`self.result = tk.Label(self.root, » text=summary(self.marks))`	`result = tk.Label(self.root, » text=summary(self.marks))`
30	`self.result.pack()`	`result.pack()`

KIỂU SAI 3 — hàm logic tự gọi messagebox, trả None

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
6	`mark = float(raw)`	`try:`
—	(không có dòng)	`mark = float(raw)`
—	(không có dòng)	`except ValueError:`
—	(không có dòng)	`messagebox.showerror("Lỗi", "Điểm phải là số")`
—	(không có dòng)	`return None`
8	`raise ValueError("Outside 0..10")`	`messagebox.showerror("Lỗi", "Điểm phải trong » 0..10")`
—	(không có dòng)	`return None`

KIỂU SAI 4 — marks khai ở thân class (dùng chung)

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
—	(không có dòng)	`marks = []`
—	(không có dòng)	
22	`self.marks = []`	(xoá dòng)

Nguồn trang này: pe-v4 · 018-G05.json · 018-G05.json · qa.json[G05]

G06 · Event loop và worker nền

Mã bài G06 · Nâng cao · M3 · GUI Tkinter và dự án · 40 phút · 20 XP · thứ tự 19 · mức N

MỤC TIÊU BÀI

Giữ giao diện phản hồi: việc dài ở worker, kết quả về qua queue + after().

ĐỀ BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hoàn thành ứng dụng "Tác vụ nền" (slow_total(n) cộng 1..n đã có): worker(n, q) chạy ở thread nền, tính slow_total(n) rồi q.put(kết quả) — KHÔNG đụng widget.

start_task(): n_var không phải số nguyên không âm → status "n phải là số nguyên không âm"; hợp lệ → status "Đang tính...", start một Thread(target=worker, args=(n, q)) (không join) rồi root.after(100, poll_result).

poll_result(): lấy kết quả bằng q.get_nowait(); queue rỗng (Empty) → root.after(100, poll_result) hẹn lại; có kết quả → status "Tổng = <kết quả>".

Thực hành: Callback chỉ làm việc ngắn: kiểm n, đổi chữ, start thread, hẹn after. Worker chỉ put vào queue. Chỉ poll_result (thread GUI) đổi widget.

Gọi (đề cho)	Kết quả mong đợi
`n = 10, bấm Tính`	`Ngay sau khi bấm: "Đang tính..."; sau khi event loop chạy: » "Tổng = 55"`
`n = "abc"`	`"n phải là số nguyên không âm"; không tạo thread`

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`worker(n, q)`	Việc nền: tính tổng và gửi về. · n: int ≥ 0; q: Queue · None · q.put(slow_total(n)); không đọc/ghi widget.
`poll_result()`	Thread GUI kiểm queue. · không · None · Có kết quả → status "Tổng = ..."; rỗng → root.after(100, poll_result).

LỜI GIẢI MẪU (qa.json · G06)

```
import threading
import tkinter as tk
from queue import Queue, Empty


def slow_total(n):
    total = 0
    for number in range(1, n + 1):
        total += number
    return total


def worker(n, q):
    q.put(slow_total(n))


root = tk.Tk()
root.title("Tác vụ nền")
q = Queue()
n_var = tk.StringVar(value="1000000")
tk.Entry(root, textvariable=n_var).pack()
status = tk.Label(root, text="Sẵn sàng")
status.pack()


def poll_result():
    try:
        total = q.get_nowait()
    except Empty:
        root.after(100, poll_result)
        return
    status.config(text=f"Tổng = {total}")


def start_task():
    raw = n_var.get().strip()
    if not raw.isdigit():
        status.config(text="n phải là số nguyên không âm")
        return
    status.config(text="Đang tính...")
    threading.Thread(target=worker, args=(int(raw), q), daemon=True).start()
    root.after(100, poll_result)


tk.Button(root, text="Tính", command=start_task).pack()

root.mainloop()
```

BỘ CHẤM — 8 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Chưa bấm: status "Sẵn sàng"	`status.cget('text')` `→ 'Sẵn sàng'`
2	slow_total(10) = 55	`slow_total(10)` `→ 55`

3	worker đưa KẾT QUẢ vào queue	`(lambda box: (worker(4, box), »`
		`box.get_nowait())[1])(__import__('queue').Queue())`
		`→ 10`

4	Bấm Tính: một thread được start, KHÔNG join trong callback	`(lambda r: (r.thread_count, r.threads[0].started, r.threads[0].joined, »`
		`_studio.tk.run_after(n=5))[:3])((n_var.set('10'), »`
		`_studio.call(_studio.tk.find(type='Button', text='Tính').invoke))[1])`
		`→ (1, True, False)`

5	n = 10: "Đang tính..." rồi "Tổng = 55"	`(lambda r: (r[2], r[4]))((n_var.set('10'), _studio.tk.find(type='Button', »`
		`text='Tính').invoke(), status.cget('text'), _studio.tk.run_after(n=5), »`
		`status.cget('text')))`
		`→ ('Đang tính...', 'Tổng = 55')`

6	n = 100: "Tổng = 5050"	`(n_var.set('100'), _studio.tk.find(type='Button', text='Tính').invoke(), »`
		`status.cget('text'), _studio.tk.run_after(n=5), status.cget('text'))[4]`
		`→ 'Tổng = 5050'`

7	n = "abc": báo lỗi, không tạo thread	`(lambda r: (r.thread_count, status.cget('text')))((n_var.set('abc'), »`
		`_studio.call(_studio.tk.find(type='Button', text='Tính').invoke))[1])`
		`→ (0, 'n phải là số nguyên không âm')`

8	Queue rỗng → poll_result tự hẹn lại, nhận kết quả ở lượt sau	`(poll_result(), q.put(42), _studio.tk.run_after(n=5), »`
		`status.cget('text'))[-1]`
		`→ 'Tổng = 42'`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Callback chỉ làm việc ngắn: kiểm n, đổi chữ, start thread, hẹn after. Worker chỉ put vào queue. Chỉ poll_result (thread GUI) đổi widget.

Cấu trúc. 1. def worker(n, q): q.put(slow_total(n)) 2. start_task: raw = n_var.get().strip(); not raw.isdigit() → báo lỗi, return; status "Đang tính..."; Thread(...).start(); root.after(100, poll_result). 3. poll_result: try total = q.get_nowait() except Empty: root.after(100, poll_result); return — rồi status.config(text=f"Tổng = {total}").

Lời giải từng phần. def poll_result(): try: total = q.get_nowait() except Empty: root.after(100, poll_result) return ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — tkinter · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy tkinter thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar...), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ.
- Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn.
- Chạy file trên máy (python ten_file.py) để thấy giao diện thật.
- Thread cũng là mô phỏng: worker chạy xong ngay khi start(), còn after() chỉ xếp lời gọi vào hàng đợi — bộ chấm chạy hàng đợi đó thay cho event loop.
- Trên máy thật worker chạy song song và cửa sổ vẫn bấm, kéo được trong lúc chờ; hãy thử với n lớn để thấy khác biệt.

NGỘ NHẬN THƯỜNG GẶP: "Cho worker gọi status.config(...) luôn cho nhanh"

Tkinter chỉ an toàn khi widget được gọi từ thread chạy mainloop. Gọi từ worker có lúc chạy, có lúc lỗi "main thread is not in main loop" hoặc treo — tuỳ máy, rất khó tái hiện. Worker put vào queue, thread GUI poll bằng after(). Cũng đừng "chờ cho chắc" bằng join() trong callback: cửa sổ đứng tới khi worker xong, mất hết lợi ích của thread.

CÂU DỰ ĐOÁN — đáp án và giải thích

Trong mô hình worker + queue + after(), ai cập nhật Label?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	Queue tự cập nhật Label	Queue chỉ chứa dữ liệu.
B ĐÚNG	**Thread GUI, khi poll_result được event loop gọi qua after()**	Worker chỉ put kết quả vào queue.
C	Worker thread, ngay khi tính xong	Gọi widget từ thread khác là không an toàn.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. Trong mô hình này ai cập nhật Label?

A	Worker thread	Worker chỉ gửi thông điệp vào queue.
B	Hệ điều hành	Tkinter vẽ lại widget khi thread GUI đổi cấu hình.

C ĐÚNG	**Thread GUI khi callback poll_result chạy qua event loop**	after() hẹn poll_result; event loop gọi nó trên thread GUI.

Giải thích (nguyên văn JSON): after() hẹn poll_result; event loop gọi nó trên thread GUI.

2. Vì sao poll_result dùng q.get_nowait() chứ không dùng q.get()?

A	get() không có trong queue.Queue	get() có, nhưng chặn.
B ĐÚNG	**get() chặn khi queue rỗng — chặn thread GUI là cửa sổ treo**	get_nowait() báo Empty ngay để hẹn lại.
C	get_nowait() nhanh hơn khi queue có dữ liệu	Khác biệt là hành vi khi rỗng.

Giải thích (nguyên văn JSON): get_nowait() báo Empty ngay để hẹn lại.

3. Gọi worker_thread.join() ngay trong start_task thì...

A	Worker chạy nhanh hơn	join không tăng tốc worker.
B ĐÚNG	**Cửa sổ đứng tới khi worker xong — như làm việc dài trong callback**	join chặn nơi gọi, ở đây là thread GUI.
C	Không ảnh hưởng gì	join chặn thread GUI.

Giải thích (nguyên văn JSON): join chặn nơi gọi, ở đây là thread GUI.

CÂU PHẢN VÍ DỤ — đáp án

n = 100, bấm Tính rồi event loop chạy poll_result — status hiện gì?

Nhãn	Lựa chọn
A	`Tổng = 100`
B	`Đang tính...`
C ĐÚNG	`Tổng = 5050`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Mô tả đường đi của kết quả từ lúc bấm "Tính" tới lúc Label đổi chữ, và ai làm bước nào.

Mẫu đạt: Thread GUI chạy start_task: kiểm n, đổi status "Đang tính...", start worker rồi hẹn poll_result bằng after(). Worker tính slow_total(n) rồi put kết quả vào queue. Event loop gọi poll_result trên thread GUI: queue rỗng thì hẹn lại, có kết quả thì status.config. Trường hợp đặc biệt: n = "abc" thì không tạo thread nào.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	Bấm Tính → start_task() start worker, hẹn poll_result
Bước 2	Worker → slow_total(n) rồi q.put(kết quả)
Bước 3	poll_result() rỗng → after(100, poll_result) hẹn lại
Bước 4	poll_result() có kết quả → status.config trên thread GUI

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Luyện có hỗ trợ): Dữ liệu mới: worker đếm số chẵn rồi gửi về qua queue box. Lắp ba chỗ.

Yêu cầu thêm: Bộ chấm bấm Tính với n khác (10, 100, "abc"), đọc nhãn ngay sau khi bấm, rồi chạy hàng đợi after() thay event loop; còn gọi poll_result() khi queue rỗng để xem nó có tự hẹn lại không.

```
import threading
import tkinter as tk
from queue import Queue, Empty

root = tk.Tk()
box = Queue()
label = tk.Label(root, text="...")
label.pack()


def count_even(limit):
    box.put(sum(1 for k in range(limit) if k % 2 == 0))


def check_box():
    try:
        value = {{slot1}}
    except Empty:
        {{slot2}}
        return
    label.config(text=f"Có {value} số chẵn")

def start():
    {{slot3}}
```

```
    root.after(100, check_box)


tk.Button(root, text="Đếm", command=start).pack()
root.mainloop()
```

Ô trống	MẢNH ĐÚNG
[Slot 1]	`box.get_nowait()`
[Slot 2]	`root.after(100, check_box)`
[Slot 3]	`threading.Thread(target=count_even, args=(1000,)).start()`

Thuộc ô	MẢNH BẪY	Vì sao sai
ô 1	`box.get()`	Chặn thread GUI tới khi có dữ liệu — cửa sổ đứng hình.
ô 2	`pass`	Không hẹn lượt kiểm tra sau: kết quả đến muộn sẽ không bao giờ được hiện.
ô 3	`count_even(1000)`	Chạy việc dài ngay trong callback; giao diện đứng tới khi xong.

Ngân hàng còn 3 mảnh KHÔNG phải bẫy (chính là mảnh đúng của các ô): ô 1, ô 3, ô 2.

ĐÁP ÁN MINI-GAME (5 màn, nguồn out-advanced · studio-lesson/v2)

match-0 · NỐI CẶP — Nối mỗi dòng lệnh với việc nó làm

Nối đúng là từng CẶP theo thứ tự `pairs` trong JSON; cột phải hiển thị xáo theo `rightOrder` = [3, 2, 1, 0].

Bên trái	NỐI VỚI (đáp án)
Bấm Tính	**start_task() start worker, hẹn poll_result**
Worker	**slow_total(n) rồi q.put(kết quả)**
poll_result() rỗng	**after(100, poll_result) hẹn lại**
poll_result() có kết quả	**status.config trên thread GUI**

fill-1 · ĐIỀN Ô TRỐNG — Luyện có hỗ trợ

Màn này dùng lại step_3.scaffold (mảnh đúng + bẫy xem khối LẮP MẢNH).

Ô	MẢNH ĐÚNG
ô 1	`box.get_nowait()`
ô 2	`root.after(100, check_box)`
ô 3	`threading.Thread(target=count_even, args=(1000,)).start()`

Bẫy ở ô 1 — `box.get()`: Chặn thread GUI tới khi có dữ liệu — cửa sổ đứng hình.
Bẫy ở ô 2 — `pass`: Không hẹn lượt kiểm tra sau: kết quả đến muộn sẽ không bao giờ được hiện.
Bẫy ở ô 3 — `count_even(1000)`: Chạy việc dài ngay trong callback; giao diện đứng tới khi xong.

spotError-2 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 41 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 34).
Lỗi: tính ngay trong callback, không thread
Sửa thành:

```
    threading.Thread(target=worker, args=(int(raw), q), daemon=True).start()
```

```
    34 |
    35 | def start_task():
    36 |     raw = n_var.get().strip()
    37 |     if not raw.isdigit():
    38 |         status.config(text="n phải là số nguyên không âm")
    39 |         return
    40 |     status.config(text="Đang tính...")
>>  41 |     worker(int(raw), q)
    42 |     root.after(100, poll_result)
    43 |
    44 |
    45 | tk.Button(root, text="Tính", command=start_task).pack()
    46 |
    47 | root.mainloop()
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

debug-3 · SỬA CODE — Sửa code cho chạy đúng

Lỗi cần sửa: join() worker trong callback
Đáp án Studio nhận = LỜI GIẢI MẪU ở trên. Các dòng khác nhau giữa code cho sẵn và lời giải mẫu:

Dòng	Code cho sẵn (SAI)	Lời giải mẫu (ĐÚNG)
41	`task = threading.Thread(target=worker, » args=(int(raw), q), daemon=True)`	`threading.Thread(target=worker, args=(int(raw), q), » daemon=True).start()`
—	`task.start()`	(xoá dòng)
—	`task.join()`	(xoá dòng)

selfExplain-4 · TỰ GIẢI THÍCH — Mô tả đường đi của kết quả từ lúc bấm "Tính" tới lúc Label đổi chữ, và ai làm bước nào.

Câu hỏi của màn: Mô tả đường đi của kết quả từ lúc bấm "Tính" tới lúc Label đổi chữ, và ai làm bước nào.

Lời giải thích mẫu: Thread GUI chạy start_task: kiểm n, đổi status "Đang tính...", start worker rồi hẹn poll_result bằng after(). Worker tính slow_total(n) rồi put kết quả vào queue. Event loop gọi poll_result trên thread GUI: queue rỗng thì hẹn lại, có kết quả thì status.config. Trường hợp đặc biệt: n = "abc" thì không tạo thread nào.

Studio chỉ chấm độ dài (tối thiểu 8 từ), nội dung do giảng viên đối chiếu.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 4 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — tính ngay trong callback, không thread

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
41	`threading.Thread(target=worker, args=(int(raw), q), » daemon=True).start()`	`worker(int(raw), q)`

KIỂU SAI 2 — join() worker trong callback

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
41	`threading.Thread(target=worker, args=(int(raw), q), » daemon=True).start()`	`task = threading.Thread(target=worker, » args=(int(raw), q), daemon=True)`
—	(không có dòng)	`task.start()`
—	(không có dòng)	`task.join()`

KIỂU SAI 3 — worker cập nhật Label trực tiếp

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
14	`q.put(slow_total(n))`	`status.config(text=f"Tổng = {slow_total(n)}")`

KIỂU SAI 4 — queue rỗng thì bỏ qua, không hẹn lại

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
30	`root.after(100, poll_result)`	(xoá dòng)

Nguồn trang này: pe-v4 · 019-G06.json · 019-G06.json · qa.json[G06]

R02 · Dự án PW5–PW9: lưu, shell, thread và GUI

Mã bài R02 · Nâng cao · M3 · GUI Tkinter và dự án · 60 phút · 40 XP · thứ tự 20 · mức N · BÀI BOSS

MỤC TIÊU BÀI

Ghép lưu file nén, process con, worker thread và Tkinter thành một ứng dụng.

ĐỀ BÀI TẬP ĐỘC LẬP · Boss — tự viết, nộp lần đầu trước khi xem lời giải

Sổ điểm mini PW5–PW9. report.py đã cấp: đọc "tên,điểm" từ stdin, in báo cáo; không có dòng dữ liệu nào thì in lỗi ra stderr và thoát mã 1. Viết lần lượt:

1) validate(name, raw_mark) → (tên đã strip, float 0..10), nhận cả "7,5"; tên trống, không phải số hoặc ngoài khoảng → ValueError.

2) save_marks(marks, path) lưu dict bằng pickle NÉN gzip; load_marks(path) nạp lại, file chưa có → {}.

3) run_report(marks) chạy [sys.executable, "report.py"] bằng subprocess, gửi mỗi sinh viên một dòng "tên,điểm" qua stdin, trả stdout (str); con thoát mã khác 0 → RuntimeError(stderr của con).

4) start_save(marks, path, done) start một Thread lưu BẢN SAO marks; xong thì put ("ok", số sinh viên) vào done (OSError → ("error", thông báo)); trả Thread, không join.

5) App (giao diện đã dựng sẵn): __init__ nạp self.marks = load_marks(path); add → validate, self.marks[tên] = điểm, status "Có <n> sinh viên" (sai → showerror); save → status "Đang lưu...", self.saver = start_save(...), after(100, self.poll_save); poll_save → get_nowait, rỗng thì hẹn lại, có thì status "Đã lưu <n> sinh viên"; report → self.output hiện run_report(self.marks) (lỗi → showerror); wait_for_save → join self.saver nếu có.

Thực hành: Viết và thử từng hàm thuần trước (validate → save/load → run_report → start_save), rồi mới nối vào các method của App.

Gọi (đề cho)	Kết quả mong đợi
`validate(" Mai ", "8.5")`	`("Mai", 8.5)`
`run_report({"Mai": 8.5, "An": 7.0})`	`"Mai: 8.5\nAn: 7.0\nAverage: 7.75\n"`
Bấm Lưu với 2 sinh viên	"Đang lưu..." rồi (event loop) "Đã lưu 2 sinh viên"; file » nén gzip

HỢP ĐỒNG HÀM (chữ ký mà bộ chấm đòi)

Chữ ký	Nhận / trả / tác dụng
`validate(name, raw_mark)`	Kiểm và chuẩn hoá một dòng nhập trước khi đưa vào dữ liệu. · name, raw_mark: str như người dùng gõ (raw_mark có thể dùng dấu phẩy "7,5") · (tên đã strip, float 0..10); tên trống / không phải số / ngoài khoảng → ValueError · Hàm thuần.
`save_marks(marks, path) / load_marks(path)`	PW5–PW6: lưu dict điểm bằng pickle, nén gzip; nạp lại. · marks: dict tên → điểm; path: tên file · load_marks: dict; file chưa có → {} · Ghi/đọc file nhị phân nén gzip (2 byte đầu 1f 8b).
`run_report(marks)`	PW7: chạy report.py trong process con, gửi dữ liệu qua stdin. · marks: dict · str — stdout của process con; con thoát mã khác 0 → RuntimeError(stderr) · subprocess.run([sys.executable, "report.py"], input=..., capture_output=True, text=True, timeout=...)
`start_save(marks, path, done)`	PW8: lưu ở worker thread để giao diện không đứng. · marks: dict (worker nhận BẢN SAO); path; done: Queue · Thread đã start (KHÔNG join) · Worker gọi save_marks; xong → done.put(("ok", số sinh viên)); OSError → done.put(("error", thông báo)).
`App(root, path=DATA_FILE)`	PW9: giao diện Tkinter gọi các hàm trên. · root: Tk; path: file dữ liệu · self.marks (nạp từ path), self.status, self.output, self.saver · add: validate → self.marks; save: "Đang lưu..." + start_save + after(poll_save); poll_save: "Đã lưu <n> sinh viên"; report: self.output = run_report(...); wait_for_save: join worker trước khi thoát.

LỜI GIẢI MẪU (qa.json · R02)

```
import gzip
import pickle
import subprocess
import sys
import threading
import tkinter as tk
from queue import Queue, Empty
from tkinter import messagebox

DATA_FILE = "marks.dat"


def validate(name, raw_mark):
    name = name.strip()
    if not name:
        raise ValueError("Tên trống")
    mark = float(raw_mark.strip().replace(",", "."))
    if not 0 <= mark <= 10:
```

```
        raise ValueError("Điểm phải trong 0..10")
    return name, mark


def save_marks(marks, path):
    with gzip.open(path, "wb") as f:
        pickle.dump(marks, f)


def load_marks(path):
    try:
        with gzip.open(path, "rb") as f:
            return pickle.load(f)
    except FileNotFoundError:
        return {}


def run_report(marks):
    lines = "".join(f"{name},{mark}\n" for name, mark in marks.items())
    result = subprocess.run([sys.executable, "report.py"], input=lines,
                            capture_output=True, text=True, timeout=10)
    if result.returncode != 0:
        raise RuntimeError(result.stderr.strip())
    return result.stdout


def save_worker(marks, path, done):
    try:
        save_marks(marks, path)
    except OSError as error:
        done.put(("error", str(error)))
    else:
        done.put(("ok", len(marks)))


def start_save(marks, path, done):
    saver = threading.Thread(target=save_worker, args=(dict(marks), path, done))
    saver.start()
    return saver


class App:
    def __init__(self, root, path=DATA_FILE):
        self.root = root
        self.path = path
        self.marks = load_marks(path)
        self.done = Queue()
        self.saver = None
        self.name_var = tk.StringVar()
        self.mark_var = tk.StringVar()
        self.create_widgets()

    def create_widgets(self):
        form = tk.Frame(self.root)
        form.pack(fill="x")
        tk.Label(form, text="Tên").grid(row=0, column=0)
        tk.Entry(form, textvariable=self.name_var).grid(row=0, column=1)
        tk.Label(form, text="Điểm").grid(row=1, column=0)
        tk.Entry(form, textvariable=self.mark_var).grid(row=1, column=1)
        buttons = tk.Frame(self.root)
        buttons.pack(fill="x")
        tk.Button(buttons, text="Thêm", command=self.add).pack(side="left")
        tk.Button(buttons, text="Lưu", command=self.save).pack(side="left")
        tk.Button(buttons, text="Báo cáo", command=self.report).pack(side="left")
        self.status = tk.Label(self.root, text=f"Đã nạp {len(self.marks)} sinh viên")
        self.status.pack()
        self.output = tk.Label(self.root, text="")
        self.output.pack()

    def add(self):
        try:
            name, mark = validate(self.name_var.get(), self.mark_var.get())
        except ValueError as error:
            messagebox.showerror("Dữ liệu không hợp lệ", str(error))
            return
        self.marks[name] = mark
        self.status.config(text=f"Có {len(self.marks)} sinh viên")

    def save(self):
        self.status.config(text="Đang lưu...")
        self.saver = start_save(self.marks, self.path, self.done)
        self.root.after(100, self.poll_save)

    def poll_save(self):
        try:
            kind, info = self.done.get_nowait()
        except Empty:
            self.root.after(100, self.poll_save)
```

```
            return
        if kind == "ok":
            self.status.config(text=f"Đã lưu {info} sinh viên")
        else:
            self.status.config(text=f"Lỗi lưu: {info}")

    def report(self):
        try:
            text = run_report(self.marks)
        except (RuntimeError, subprocess.TimeoutExpired) as error:
            messagebox.showerror("Báo cáo", str(error))
            return
        self.output.config(text=text)

    def wait_for_save(self):
        if self.saver is not None:
            self.saver.join()


if __name__ == "__main__":
    root = tk.Tk()
    root.title("Sổ điểm mini")
    app = App(root)
    root.mainloop()
    app.wait_for_save()
```

BỘ CHẤM — 17 ca, phải ĐẠT HẾT mới tính hoàn thành

#	Ca kiểm tra — chấm cái gì	Biểu thức gọi → giá trị mong đợi
1	Khởi động chưa có marks.dat → "Đã nạp 0 sinh viên"	`app.status.cget('text')` `→ 'Đã nạp 0 sinh viên'`
2	validate chuẩn hoá tên và điểm	`[validate(" Mai ", "8.5"), validate("An", "7,25")]` `→ [('Mai', 8.5), ('An', 7.25)]`

3	validate từ chối dữ liệu sai bằng ValueError	`[_studio.call(validate, n, m).error for n, m in [('An', 'abc'), ('  ', »`
		`'7'), ('Binh', '11'), ('Chi', '-0.5')]]`
		`→ ['ValueError', 'ValueError', 'ValueError', 'ValueError']`

4	save_marks rồi load_marks phục hồi đúng dict	`(save_marks({'Lan': 9.25, 'Huy': 6.0}, 't1.dat'), load_marks('t1.dat'))[1]` `→ {'Lan': 9.25, 'Huy': 6.0}`
5	File lưu được nén gzip (2 byte đầu 1f 8b)	`(save_marks({'A': 1.0}, 't2.dat'), open('t2.dat', 'rb').read(2))[1]` `→ b'\x1f\x8b'`
6	Chưa có file → load_marks trả {}	`load_marks('chua_co.dat')` `→ {}`
7	run_report: process con in báo cáo	`run_report({'Mai': 8.5, 'An': 7.0})` `→ 'Mai: 8.5\nAn: 7.0\nAverage: 7.75\n'`
8	Process con lỗi (không có dữ liệu) → RuntimeError	`_studio.call(run_report, {}).error` `→ 'RuntimeError'`

9	start_save: một worker, ghi file, gửi ("ok", 2)	`(lambda box: (lambda r: (r.thread_count, load_marks('t3.dat'), »`
		`box.get_nowait()))(_studio.call(start_save, {'Mai': 8.0, 'An': 6.5}, »`
		`'t3.dat', box)))(__import__('queue').Queue())`
		`→ (1, {'Mai': 8.0, 'An': 6.5}, ('ok', 2))`

10	Bấm Thêm: Mai 8.5 rồi An 7	`(app.marks.clear(), app.name_var.set('Mai'), app.mark_var.set('8.5'), »`
		`_studio.tk.find(type='Button', text='Thêm').invoke(), »`
		`app.status.cget('text'), app.name_var.set('An'), »`
		`app.mark_var.set('7'), _studio.tk.find(type='Button', »`
		`text='Thêm').invoke(), app.status.cget('text'), dict(app.marks))[-2:]`
		`→ ('Có 2 sinh viên', {'Mai': 8.5, 'An': 7.0})`

11	Dữ liệu sai → showerror, không thêm	`(lambda n, m: (app.name_var.set('Binh'), app.mark_var.set('12'), »`
		`_studio.tk.find(type='Button', text='Thêm').invoke(), »`
		`app.status.cget('text'), len(app.marks) - n, len(_studio.tk.messages) »`
		`- m)[-2:])(len(app.marks), len(_studio.tk.messages))`
		`→ (0, 1)`

12	Bấm Lưu: worker nền, không join trong callback, "Đang lưu..."	`(lambda r: (r.thread_count, r.threads[0].joined, app.status.cget('text'), »`
		`_studio.tk.run_after(n=5))[:3])(_studio.call(_studio.tk.find(type='Butt »`
		`on', text='Lưu').invoke))`
		`→ (1, False, 'Đang lưu...')`

13	Event loop chạy poll_save → "Đã lưu 1 sinh viên", file có dữ liệu	`(app.marks.clear(), app.marks.update({'Kim': 9.0}), »`
		`_studio.tk.find(type='Button', text='Lưu').invoke(), »`
		`_studio.tk.run_after(n=5), app.status.cget('text'), »`
		`load_marks(app.path))[-2:]`
		`→ ('Đã lưu 1 sinh viên', {'Kim': 9.0})`

14	Bấm Báo cáo → hiện stdout của process con	`(app.marks.clear(), app.marks.update({'Lan': 9.0, 'Huy': 6.0}), »`
		`_studio.tk.find(type='Button', text='Báo cáo').invoke(), »`
		`app.output.cget('text'))[-1]`
		`→ 'Lan: 9.0\nHuy: 6.0\nAverage: 7.50\n'`

15	Trước khi thoát: wait_for_save() join worker đang lưu	`(lambda r: (app.wait_for_save(), r.threads[0].joined, »`
		`_studio.tk.run_after(n=5))[1])(_studio.call(_studio.tk.find(type='Butto »`
		`n', text='Lưu').invoke))`
		`→ True`

16	Khởi động lại: App mới nạp file đã lưu	`(save_marks({'Tam': 5.5}, 'boot.dat'), (lambda a: (a.marks, » a.status.cget('text')))(App(tk.Tk(), 'boot.dat')))[1]` → `({'Tam': 5.5}, 'Đã nạp 1 sinh viên')`
17	Process con chạy bằng sys.executable	`[c['args'][0] == __import__('sys').executable for c in » _studio.run(entry="run_report({'Mai': 8.0})").subprocess_calls]` → `[True]`

GỢI Ý THEO BẬC (học viên mở được, dùng để dẫn chứ không đưa đáp án)

Nguyên lý. Viết và thử từng hàm thuần trước (validate → save/load → run_report → start_save), rồi mới nối vào các method của App.

Cấu trúc. 1. validate: strip tên, rỗng → ValueError; float(raw.strip().replace(",", ".")); ngoài 0..10 → ValueError. 2. save_marks: with gzip.open(path, "wb") as f: pickle.dump(marks, f). load_marks: gzip.open(path, "rb") + pickle.load; FileNotFoundError → {}. 3. run_report: subprocess.run([sys.executable, "report.py"], input=..., capture_output=True, text=True, timeout=10); returncode != 0 → RuntimeError. 4. start_save: Thread(target=..., args=(dict(marks), path, done)).start(); return thread. 5. App.save: status "Đang lưu..."; self.saver = start_save(...); self.root.after(100, self.poll_save). wait_for_save: join nếu có.

Lời giải từng phần. def run_report(marks): lines = "".join(f"{name},{mark}\n" for name, mark in marks.items()) result = subprocess.run([sys.executable, "report.py"], input=lines, capture_output=True, text=True, timeout=10) ...

MÔI TRƯỜNG CHẤM LÀ MÔ PHỎNG — tkinter · ĐỌC TRƯỚC KHI NÓI HỌC VIÊN SAI

Bộ chấm KHÔNG chạy tkinter thật. Khác biệt giữa bản mô phỏng và máy thật là nguyên nhân thường gặp hơn cả lỗi code — nguyên văn ghi chú của bài:

- Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar...), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ.
- Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn.
- Chạy file trên máy (python ten_file.py) để thấy giao diện thật.
- Thread và subprocess cũng là mô phỏng: worker chạy xong ngay khi start(); process con chỉ chạy được [sys.executable, "report.py"] hoặc [sys.executable, "-c", code], có stdin/stdout/stderr riêng — lệnh shell (dir, ls...) không chạy.
- Chạy trên máy: lưu report.py cùng thư mục với bài và chạy lệnh từ thư mục đó (đường dẫn "report.py" là tương đối); trên Windows dữ liệu qua pipe dùng bảng mã mặc định của hệ điều hành, nên thử với tên không dấu.

Bài có CỬA TỰ KIỂM TRA (gate): học viên phải nộp trước, lời giải mẫu trong Studio chỉ mở sau khi bài được ghi nhận hoàn thành.

FILE CẤP SẴN CHO BÀI · report.py (module) — học viên không phải viết file này

```
"""Báo cáo điểm — chạy như PROCESS CON.

Đọc stdin, mỗi dòng "tên,điểm"; in báo cáo ra stdout. Không có dòng dữ liệu nào thì
in lỗi ra stderr và thoát mã 1. Chữ in ra không dấu để pipe trên Windows không vấp bảng mã.
"""
import sys

rows = []
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    name, mark = line.rsplit(",", 1)
    rows.append((name, float(mark)))

if not rows:
    print("No data", file=sys.stderr)
    sys.exit(1)

for name, mark in rows:
    print(f"{name}: {mark}")
average = sum(mark for name, mark in rows) / len(rows)
print(f"Average: {average:.2f}")
```

NGỘ NHẬN THƯỜNG GẶP: Bốn cái bẫy của PW5–PW9

Quên text=True nên stdout là bytes (và input= str nổ TypeError); gọi "python" thay vì sys.executable nên chạy nhầm Python khác (hoặc không có); lưu pickle mà không nén dù đề yêu cầu nén; join() worker ngay trong nút Lưu làm cửa sổ đứng. Ngoài ra: worker không được tự đổi Label — chỉ put vào queue.

CÂU DỰ ĐOÁN — đáp án và giải thích

Vì sao nút Lưu không được join() worker ngay trong callback?

Nhãn	Lựa chọn	Vì sao đúng / vì sao sai
A	join làm file bị lưu hai lần	join chỉ chờ, không chạy lại target.
B ĐÚNG	**join chặn thread GUI tới khi lưu xong — cửa sổ đứng, mất ý nghĩa của worker**	Chờ worker để lúc thoát (wait_for_save).

C	Không sao, join chạy rất nhanh	Lưu file lớn hoặc ổ chậm có thể mất nhiều giây.

TỰ KIỂM TRA — 3 câu trắc nghiệm

1. subprocess.run([...], capture_output=True) không có text=True thì result.stdout là gì?

A	str như thường	Mặc định pipe trả bytes.
B	None	capture_output=True đã thu stdout.
C ĐÚNG	**bytes, ví dụ b"Mai: 8.5\n..."**	text=True mới giải mã thành str.

Giải thích (nguyên văn JSON): text=True mới giải mã thành str.

2. Cần học threading trước khi tạo nút Tkinter đầu tiên không?

A	Có, Tkinter bắt buộc dùng thread	Tkinter chạy tốt trên một thread.
B ĐÚNG	**Không; cần khi kết hợp nút với tác vụ nền dài**	Nút đầu tiên chỉ cần callback; thread chỉ cần khi callback phải chờ lâu.
C	Có, mỗi nút một thread	Chỉ việc dài mới cần worker.

Giải thích (nguyên văn JSON): Nút đầu tiên chỉ cần callback; thread chỉ cần khi callback phải chờ lâu.

3. Vì sao gọi [sys.executable, "report.py"] thay vì ["python", "report.py"]?

A	sys.executable chạy được lệnh shell	Nó chỉ là đường dẫn tới trình thông dịch.
B	"python" chạy chậm hơn	Không phải chuyện tốc độ.
C ĐÚNG	**sys.executable là đúng file Python đang chạy chương trình; "python" có thể là bản khác hoặc không tồn tại**	Trên Windows "python" có thể là stub của Store.

Giải thích (nguyên văn JSON): Trên Windows "python" có thể là stub của Store.

CÂU PHẢN VÍ DỤ — đáp án

run_report({}) (không có sinh viên nào) cho kết quả gì?

Nhãn	Lựa chọn
A	`Chuỗi rỗng ""`
B ĐÚNG	`RuntimeError với stderr của report.py`
C	`Average: 0.00`

Schema không có trường giải thích cho câu phản ví dụ. Chỗ lý giải gần nhất đã có trong bài: khối NGỘ NHẬN và BẢNG TRACE bên dưới.

TỰ GIẢI THÍCH — lời giải thích mẫu

Câu hỏi: Kể đường đi của dữ liệu từ ô nhập tới file marks.dat và tới báo cáo, nêu thread/process nào làm từng bước.

Mẫu đạt: Thread GUI: add đọc hai StringVar, validate, ghi vào self.marks. Bấm Lưu: thread GUI start worker với bản sao marks; worker pickle + gzip ra file rồi put ("ok", n) vào queue; poll_save (thread GUI, qua after) đổi status. Báo cáo: process con report.py nhận "tên,điểm" qua stdin, in báo cáo ra stdout cho cha hiện lên Label. Trường hợp đặc biệt: chưa có file → {}, dữ liệu rỗng → con thoát mã 1 → RuntimeError.

Không có rubric từ khoá: Studio chỉ đòi độ dài tối thiểu 8 từ. Nội dung thì giảng viên đối chiếu với mẫu ở trên.

BẢNG TRACE (trạng thái trước → sau, dùng để giải thích cho học viên)

Bước	Trạng thái
Bước 1	Ô nhập → validate() → self.marks[tên] = điểm
Bước 2	Bấm Lưu → start_save() → worker pickle + gzip → put ("ok", n)
Bước 3	poll_save() qua after() → status "Đã lưu n sinh viên"
Bước 4	Bấm Báo cáo → subprocess report.py → stdout lên self.output

LẮP MẢNH (scaffold) — mảnh đúng cho từng ô và bẫy ở từng ô

Đề màn luyện có hỗ trợ (Lập kế hoạch trước khi tự viết): Đi lại đường dữ liệu của bài trước khi tự viết code.

Yêu cầu thêm: Bộ chấm gọi thẳng các hàm thuần với dữ liệu mới, kiểm file nén gzip, chạy process con với dữ liệu rỗng, rồi bấm Thêm/Lưu/Báo cáo trên App và chạy hàng đợi after() thay event loop.

JSON của bài không khai step_3.scaffold (bài này không có màn lắp mảnh) — chỗ này KHÔNG được suy diễn thêm.

ĐÁP ÁN MINI-GAME (2 màn, nguồn out-advanced · studio-lesson/v2)

trueFalse-0 · ĐÚNG / SAI — Khởi động trận Boss: mỗi đáp án đúng hay sai?

Câu hỏi: subprocess.run([...], capture_output=True) không có text=True thì result.stdout là gì?

Phán quyết	Phát biểu	Vì sao
SAI	str như thường	Mặc định pipe trả bytes.
SAI	None	capture_output=True đã thu stdout.

ĐÚNG	bytes, ví dụ b"Mai: 8.5\n..."	text=True mới giải mã thành str.

spotError-1 · SĂN LỖI — Săn lỗi: dòng nào làm chương trình sai?

Dòng sai: 39 (số dòng tuyệt đối, đoạn hiện ra bắt đầu từ dòng 32).
Lỗi: quên text=True
Sửa thành:

```
                          capture_output=True, text=True, timeout=10)
```

```
    32 |     except FileNotFoundError:
    33 |         return {}
    34 |
    35 |
    36 | def run_report(marks):
    37 |     lines = "".join(f"{name},{mark}\n" for name, mark in marks.items())
    38 |     result = subprocess.run([sys.executable, "report.py"], input=lines,
>>  39 |                             capture_output=True, timeout=10)
    40 |     if result.returncode != 0:
    41 |         raise RuntimeError(result.stderr.strip())
    42 |     return result.stdout
    43 |
    44 |
    45 | def save_worker(marks, path, done):
    46 |     try:
```

Dấu >> ở đầu dòng là của tài liệu này, không phải của bài.

CÁC KIỂU SAI ĐÃ BIẾT (qa.json · bad) — 4 biến thể

Mỗi biến thể là một bản code chỉ khác lời giải mẫu vài dòng. Bảng dưới là diff: trái = lời giải mẫu, phải = bản sai.

KIỂU SAI 1 — quên text=True

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
39	`capture_output=True, » text=True, timeout=10)`	`capture_output=True, » timeout=10)`

KIỂU SAI 2 — join() worker ngay trong start_save

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
—	(không có dòng)	`saver.join()`

KIỂU SAI 3 — lưu pickle không nén

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
24	`with gzip.open(path, "wb") as f:`	`with open(path, "wb") as f:`

KIỂU SAI 4 — gọi "python" thay vì sys.executable

Dòng	Lời giải mẫu (ĐÚNG)	Bản sai
38	`result = subprocess.run([sys.executable, » "report.py"], input=lines,`	`result = subprocess.run(["python", "report.py"], » input=lines,`

Nguồn trang này: pe-v4 · 020-R02.json · 020-R02.json · qa.json[R02]